Final Project of Gas Dynamics Course

Numerical Solution of Potential Flow over a NACA 0012 Airfoil

Students

Hamid Motallebi
Student ID:

Sheida Yarahmadi
Student ID:

Instructor

Dr. Vahid Esfahanian

Submission Date

February 2026

Faculty of Mechanical Engineering
College of Engineering
University of Tehran

Table of Contents

1 Project Theory **9**

1.1 Grid Generation and Coordinate Transformation . . . 9

1.1.1 Mapping Definition . . . 9

1.1.2 Transformation Metrics . . . 9

1.1.3 Elliptic Grid Generation . . . 10

1.2 Governing Flow Equations . . . 10

1.2.1 Full Potential Equation . . . 10

1.2.2 Density Relation . . . 10

1.3 Equation Discretization . . . 11

1.4 Boundary Conditions . . . 11

1.4.1 Airfoil Surface: . . . 11

1.4.2 Far-field: . . . 12

1.4.3 Branch Cut: . . . 12

2 Project Objective **13**

3 Response to Part A **14**

3.1 Problem Statement . . . 14

3.2 Description and Analysis . . . 14

3.2.1 Mesh Visualization . . . 14

3.2.2 Convergence Iterations . . . 21

4 Response to Part B **22**

4.1 Convergence Plot . . . 22

4.2 ϕ-Contour Plotting . . . 24

4.3 Leading-Edge ϕ Values . . . 26

4.4 Aerodynamic Coefficients . . . 27

5 Response for Part C **30**

5.1 Problem Objective . . . 30

5.2 Project Theory . . . 30

5.3 Integral Form (Finite Volume Method) . . . 31
5.4 Mapping to MATLAB Code Variables . . . 32
5.5 Final Discrete Equation (for Point P) . . . 32

6 Response for Part D **36**
6.1 Problem Objective . . . 36
6.2 Solution Procedure . . . 36
6.3 Numerical Results for NACA 0012 . . . 36
6.4 Documentation of Code Modifications from Part C to Part D . . . 37
6.4.1 Structural Change: From "Linear Script" to "Iterative Loop" . . . 37
6.4.2 Dynamization of Equation Coefficients . . . 38
6.4.3 Addition of "Critical Mach Detection" Block . . . 38
6.4.4 Error Handling . . . 38
6.5 Results of Part D Code . . . 39

7 Response for Part E **41**
7.1 Problem Objective . . . 41
7.2 Introduction and Objective . . . 41
7.3 Theoretical Foundations . . . 41
7.4 Methodology . . . 41
7.5 Results and Discussion . . . 42
7.5.1 Comparison at Mach 0.2 (Low Compressibility Regime) . . . 42
7.5.2 Comparison at Critical Mach Number $M \approx 0.70$ (High Compressibility Regime) . . . 43
7.6 Final Conclusion . . . 44

8 Response for Part F **45**
8.1 Problem Objective . . . 45
8.2 Introduction . . . 45
8.3 Verification of Governing Equation and Transformation Metrics . . . 45
8.4 Verification of Discretization and Solver Algorithm . . . 46
8.5 Verification of Boundary Conditions . . . 47

8.6 Verification of Aerodynamic Computations (Post-Processing) 47
8.7 Conclusion 48

A MATLAB Implementation Code for Parts A and B **49**

B MATLAB Implementation Code for Part C **60**

C MATLAB Implementation Code for Part D **70**

D MATLAB Implementation Code for Part E **79**

List of Figures

1 Complete Domain Geometry . . . 13
2 The TFI mesh for the 61×31 case . . . 15
3 The TFI mesh for the 61×31 case around an airfoil . . . 15
4 The elliptic mesh for the 61×31 case . . . 16
5 The elliptic mesh for the 61×31 case around an airfoil . . . 16
6 The TFI mesh for the 121×61 case . . . 17
7 The TFI mesh for the 121×61 case around an airfoil . . . 17
8 The elliptic mesh for the 121×61 case . . . 18
9 The elliptic mesh for the 121×61 case around an airfoil . . . 18
10 The TFI mesh for the 241×121 case . . . 19
11 The TFI mesh for the 241×121 case around an airfoil . . . 19
12 The elliptic mesh for the 241×121 case . . . 20
13 The elliptic mesh for the 242×121 case around an airfoil . . . 20
14 Convergence diagram for mesh size 61×31 . . . 22
15 Convergence diagram for mesh size 121×61 . . . 23
16 Convergence diagram for mesh size 241×121 . . . 23
17 ϕ contour for mesh size 61×31 . . . 24
18 ϕ contour for mesh size 121×61 . . . 25
19 ϕ contour for mesh size 241×121 . . . 25
20 Pressure coefficient diagram for mesh size 61×31 . . . 27
21 Pressure coefficient diagram for mesh size 121×61 . . . 28
22 Pressure coefficient diagram for mesh size 241×121 . . . 28
23 Mesh Generation for $M_\infty = 0.8$ Case . . . 34
24 Convergence Analysis for $M_\infty = 0.8$ . . . 34
25 ϕ contour for $M_\infty = 0.8$. . . . 35
26 Pressure coefficient plot for $M_\infty = 0.8$. . . . 35
27 Output table from code execution for calculating maximum local Mach number for various freestream Mach numbers . . . 39
28 Pressure coefficient versus location diagram for critical Mach . . . 40

29 Local Mach number plot versus location for critical Mach 40

30 Variation of Maximum Local Mach Number ($M_{\text{local,max}}$) vs. Free-stream Mach Number (M_∞) 42

31 Surface Pressure Coefficient and Local Mach Number Distribution at Critical Condition ($M_\infty \approx 0.70$) 43

List of Tables

1 Number of elliptic smoothing iterations for each mesh size 21
2 Number of SOR solution iterations per number of meshes 24
3 Value of ϕ on the Leading edge for different mesh sizes 26
4 Drag and lift coefficients for different mesh sizes 29
5 Maximum Mach number on the surface for various freestream Mach numbers . . 37

Numerical Solution of Potential Flow over a NACA 0012 Airfoil

Students: Hamid Motallebi ( )
Sheida Yarahmadi ( )

abstract

This report documents the solution of incompressible potential flow around a NACA0012 airfoil. The computational domain, as specified, is a far-field circle of radius $R = 41$ with zero angle of attack, and the far-field boundary condition is applied as $\phi = U_\infty x$. The airfoil, defined by the given geometric equation, is centered at the origin, and Laplace's equation $\nabla^2\phi = 0$ is solved over the entire domain. For domain discretization, a structured curvilinear grid is first generated using the Transfinite Interpolation method and subsequently improved via an elliptic smoothing scheme. Three mesh sizes, 61×31, 121×61, and 241×121, are constructed to investigate the effect of grid refinement on solution accuracy. In the numerical solution, finite-volume coefficients are obtained by calculating metrics at cell-face midpoints, and the impermeability condition $\nabla\phi \cdot \hat{n} = 0$ is enforced on the airfoil surface. The iterative procedure is carried out using the Successive Over-Relaxation (SOR) method with $\omega = 1.7$. The report structure is as follows: grid generation and evaluation of the elliptic scheme's convergence, the finite-volume solution and iteration convergence history for all meshes, reconstruction of the potential at the nodes and plotting of ϕ contours, estimation of the potential value at the leading edge and assessment of the order of accuracy using the method proposed by the ASME Journal of Fluids Engineering, and finally, calculation and visualization of the pressure coefficient C_p and extraction of the lift and drag coefficients for all three meshes, comparing them with the physical expectation at zero angle of attack.

Numerical Solution of Potential Flow over a NACA 0012 Airfoil

Students: Hamid Motallebi ( )

Sheida Yarahmadi ( )

1 Project Theory

In numerical analysis of fluid flow around complex-geometry bodies (such as the NACA 0012 airfoil), using regular Cartesian grids leads to large errors near boundaries. To accurately solve the governing equations and properly apply boundary conditions, the coordinate transformation method is used. In this method, the complex physical domain (x, y) is mapped to a regular rectangular computational domain (ξ, η).

1.1 Grid Generation and Coordinate Transformation

1.1.1 Mapping Definition

Assume the physical domain (Physical Domain) lies in the x-y plane, which includes the airfoil and far-field boundaries. We seek a mapping that transforms this domain into a computational domain (Computational Domain) in the ξ-η plane with uniform grid spacing ($\Delta\xi = 1$, $\Delta\eta = 1$):

$$\xi = \xi(x, y), \quad \eta = \eta(x, y)$$

In the computational domain: The ξ-axis (index i) typically runs along the airfoil surface and the branch cut. The η-axis (index j) runs perpendicular to the surface (from the airfoil to the far-field).

1.1.2 Transformation Metrics

To solve differential equations in computational space, partial derivatives with respect to x and y are needed. According to the chain rule:

$$\frac{\partial}{\partial x} = \xi_x \frac{\partial}{\partial \xi} + \eta_x \frac{\partial}{\partial \eta}, \quad \frac{\partial}{\partial y} = \xi_y \frac{\partial}{\partial \xi} + \eta_y \frac{\partial}{\partial \eta}$$

To compute the metric coefficients $(\xi_x, \xi_y, \eta_x, \eta_y)$, the transformation Jacobian is used. Differentials are related as:

$$\begin{bmatrix} dx \\ dy \end{bmatrix} = \begin{bmatrix} x_\xi & x_\eta \\ y_\xi & y_\eta \end{bmatrix} \begin{bmatrix} d\xi \\ d\eta \end{bmatrix}$$

The above matrix is the transformation Jacobian J. The Jacobian determinant represents the area element change during transformation:

$$J = x_\xi y_\eta - x_\eta y_\xi$$

Inverting this matrix gives the transformation metrics:

$$\xi_x = y_\eta / J, \quad \xi_y = -x_\eta / J, \quad \eta_x = -y_\xi / J, \quad \eta_y = x_\xi / J$$

1.1.3 Elliptic Grid Generation

To obtain a smooth and orthogonal grid with controllable point distribution, Poisson differential equations are solved. The main idea is that grid lines ($\xi = \text{const}$ and $\eta = \text{const}$) must satisfy the following equations in physical space:

$$\nabla^2 \xi = P(\xi, \eta), \quad \nabla^2 \eta = Q(\xi, \eta)$$

By swapping dependent and independent variables (transforming from ξ, η-space to x, y-space), we obtain grid generation equations in computational space that must be solved for x and y:

$$\alpha \frac{\partial^2 x}{\partial \xi^2} - 2\beta \frac{\partial^2 x}{\partial \xi \partial \eta} + \gamma \frac{\partial^2 x}{\partial \eta^2} = -J^2 (P x_\xi + Q x_\eta)$$
$$\alpha \frac{\partial^2 y}{\partial \xi^2} - 2\beta \frac{\partial^2 y}{\partial \xi \partial \eta} + \gamma \frac{\partial^2 y}{\partial \eta^2} = -J^2 (P y_\xi + Q y_\eta)$$

where geometric coefficients are:

$$\alpha = x_\eta^2 + y_\eta^2, \quad \beta = x_\xi x_\eta + y_\xi y_\eta, \quad \gamma = x_\xi^2 + y_\xi^2$$

Control functions P and Q are used to control grid clustering near the airfoil (for capturing steep velocity gradients). These equations are solved using finite difference methods and iterative approaches (such as SOR) to obtain the final grid coordinates $x_{i,j}$ and $y_{i,j}$.

1.2 Governing Flow Equations

The project aims to solve subsonic compressible potential flow.

1.2.1 Full Potential Equation

The flow is inviscid, irrotational, and compressible. Irrotational condition ($\nabla \times \vec{V} = 0$) allows defining velocity potential ϕ such that $\vec{V} = \nabla \phi$. Steady continuity equation (mass conservation):

$$\nabla \cdot (\rho \nabla \phi) = 0 \Rightarrow \nabla \cdot (\rho \vec{V}) = 0$$

This is the main equation to be solved.

1.2.2 Density Relation

In compressible flow, density (ρ) is not constant and depends on local flow velocity. From isentropic relations for perfect gas:

$$\frac{\rho}{\rho_\infty} = \left[1 - \frac{\gamma - 1}{2} M_\infty^2 \left(\frac{V^2}{V_\infty^2} - 1 \right) \right]^{1/(\gamma - 1)}$$

where $|\nabla \phi|^2 = u^2 + v^2$. If $M_\infty \to 0$, then $\rho \approx \rho_\infty$ and the equation reduces to Laplace's equation ($\nabla^2 \phi = 0$). For $M_\infty > 0$, the equation becomes nonlinear because ρ itself is a function of ϕ.

1.3 Equation Discretization

To numerically solve the continuity equation $\nabla \cdot (\rho \nabla \phi) = 0$ on a curvilinear grid, the finite volume method is used. The integral form of the equation over a computational control volume is:

$$\oint_{\partial\Omega} (\rho \vec{V}) \cdot \vec{n}\, dA = 0$$

In two-dimensional computational space (ξ, η), this integral becomes a sum of fluxes over the four faces of the cell:

$$F_{i+1/2,j} - F_{i-1/2,j} + G_{i,j+1/2} - G_{i,j-1/2} = 0$$

where fluxes F and G are contravariant density-momentum components. Using metrics defined in Section 2.2, contravariant velocities (U, V) are defined as:

$$U = \xi_x u + \xi_y v = \xi_x \phi_x + \xi_y \phi_y, \quad V = \eta_x u + \eta_y v = \eta_x \phi_x + \eta_y \phi_y$$

Substituting ϕ_x, ϕ_y in terms of computational space derivatives:

$$\phi_x = \xi_x \phi_\xi + \eta_x \phi_\eta, \quad \phi_y = \xi_y \phi_\xi + \eta_y \phi_\eta$$

The final discretized equation for each cell takes the tensor form:

$$\frac{\partial}{\partial \xi}\left[\frac{\rho}{J}(\alpha \phi_\xi - \beta \phi_\eta)\right] + \frac{\partial}{\partial \eta}\left[\frac{\rho}{J}(\gamma \phi_\eta - \beta \phi_\xi)\right] = 0$$

Here α, β, γ are the same metric coefficients defined in the grid generation section (with the note that β appears with a negative sign in the cross term, depending on the precise definition of the metric tensor g^{ij}).

Important Implementation Note:

Since equations must be discretized using central difference schemes, for stability in supersonic regions (if present), "artificial density" is required. However, since your project is defined as subsonic, simple averaging or central differencing for density on faces is sufficient.

1.4 Boundary Conditions

To complete the mathematical model, boundary conditions are applied in transformed space:

1.4.1 Airfoil Surface:

No-penetration condition. Normal velocity to the surface is zero. In computational space ($\eta = 1$):

$$V = 0 \Rightarrow \nabla \phi \cdot \vec{n} = 0 \Rightarrow \frac{\partial \phi}{\partial \eta} = 0$$

(If the grid is fully orthogonal. If not, $V = 0$ implies a relation between ϕ_η and ϕ_ξ).

1.4.2 Far-field:

Uniform flow with zero angle of attack $\alpha = 0$

$$\phi \to U_\infty x \quad \text{or} \quad \phi \to U_\infty r\cos\theta$$

At the outer boundary ($\eta = \eta_{\max}$), ϕ is fixed based on physical coordinates of boundary points.

1.4.3 Branch Cut:

Since the grid (C-grid or O-grid) wraps around the airfoil, a branch cut is created behind the airfoil (wake). On this line, potential continuity (or potential jump due to the Kutta condition) must be applied. For a symmetric airfoil at zero angle of attack (NACA 0012 at 0 AoA), lift is zero, thus $\Gamma = 0$ and potential is continuous across the branch cut:

$$\phi(\xi, 1)_{\text{upper}} = \phi(\xi, 1)_{\text{lower}}$$

In coding, values on the lower boundary ($j = 1$) and upper boundary ($j = j_{\max}$) in the region behind the airfoil are set equal (periodic condition).

Students: Hamid Motallebi ( )
Sheida Yarahmadi ( )

2 Project Objective

In this project, incompressible potential flow around the NACA0012 airfoil is solved. The equation for the NACA0012 airfoil is given below. Note that this formulation is slightly modified from the standard expression found in the literature, so that the upper and lower surfaces meet at a sharp "trailing edge".

$$y = \pm\frac{0.12}{0.2}\left[0.29690\sqrt{x} - 0.12600x - 0.35160x^2 + 0.28430x^3 - (0.10150 + 0.0021)x^4\right] \quad (1)$$

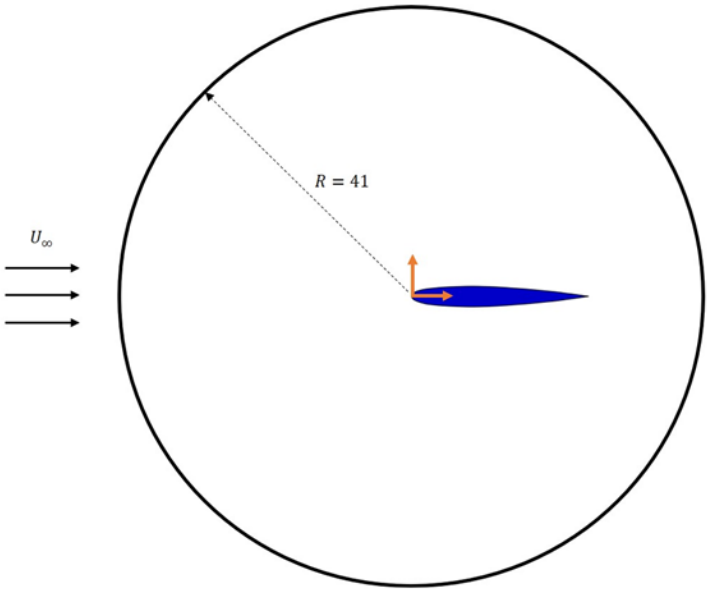


Figure 1: Complete Domain Geometry

To solve $\nabla^2\phi = 0$ over the domain, a structured curvilinear grid with $(m+1) \times (n+1)$ grid points is required, where:

- m: number of divisions along the airfoil and far-field circle (circumferential direction)
- n: number of divisions along the branch cut (radial direction)

The far-field boundary condition for the flow potential is given by:

$$\phi = U_\infty x$$

where $U_\infty = 1$ is taken as the reference freestream velocity.

3 Response to Part A

3.1 Problem Statement

Implementation of subsonic flow over an airfoil code in MATLAB software.

3.2 Description and Analysis

The mesh generation begins with the Transfinite Interpolation (TFI) method for this domain, and by applying an elliptic grid generation scheme, three meshes with sizes of 61×31, 121×61, and 241×121 are constructed. To achieve a more uniform grid distribution, the grid points on the branch cut are not fixed/locked.

3.2.1 Mesh Visualization

This section deals with plotting the mesh figures. Note that to examine different numbers of meshes, the ξ (ksi) and η (eta) step sizes must be changed manually at the beginning of the code. These two lines of code are given below:

```
eta = 0 : 1/30 : 1;
ksi = 0 : 1/60 : 1;
```

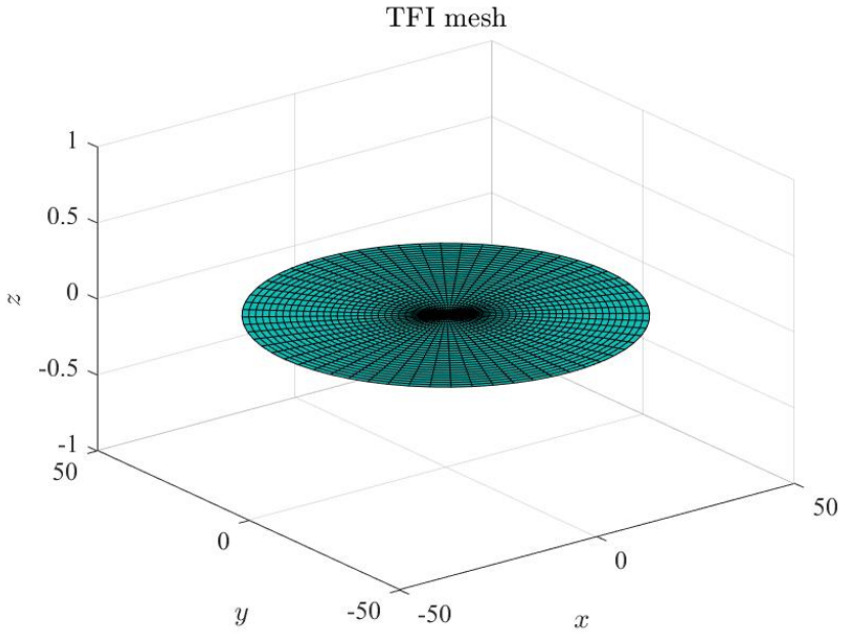


Figure 2: The TFI mesh for the 61×31 case

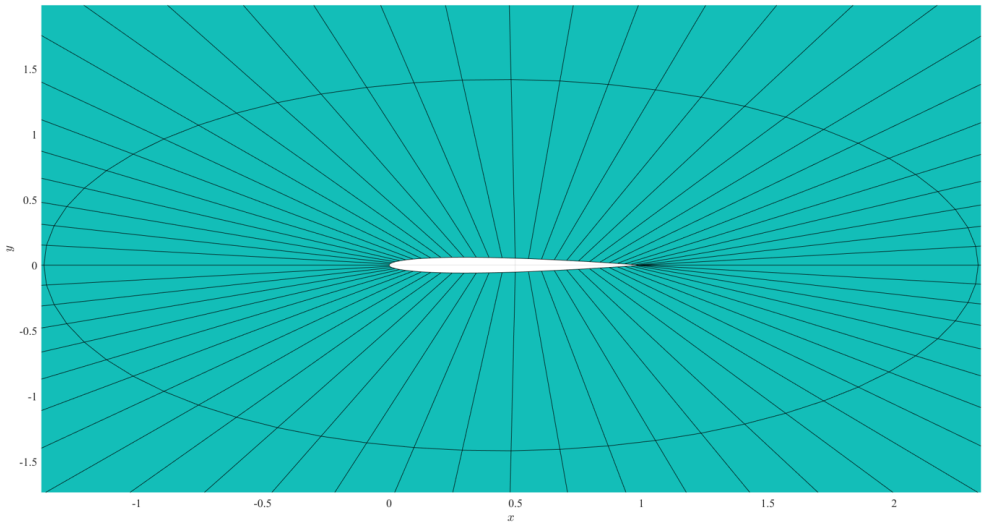


Figure 3: The TFI mesh for the 61×31 case around an airfoil

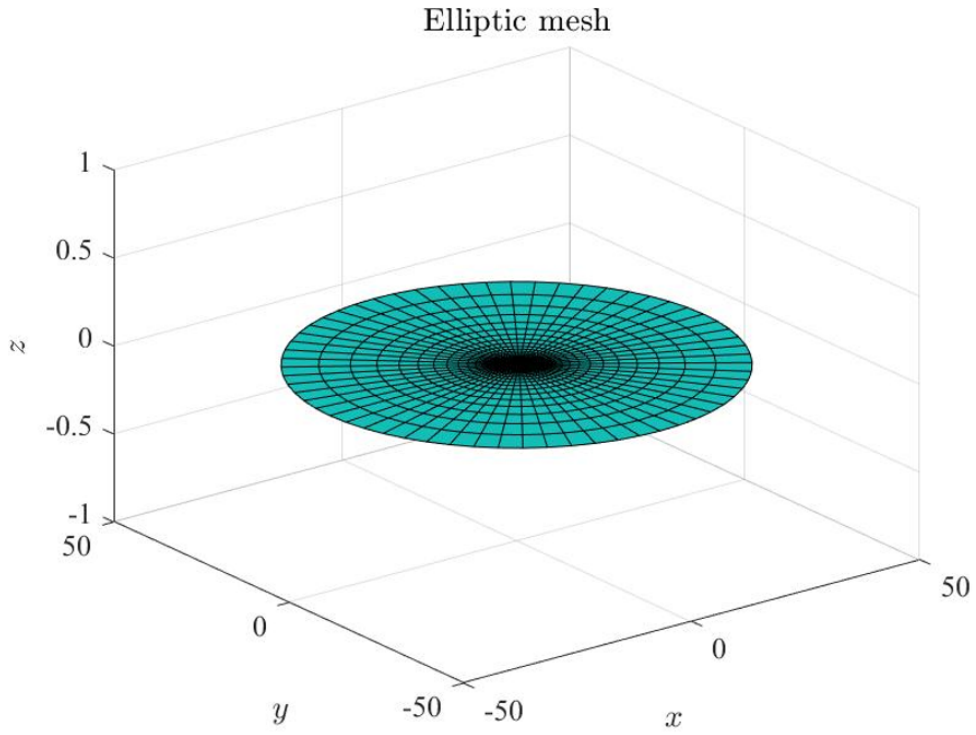


Figure 4: The elliptic mesh for the 61×31 case

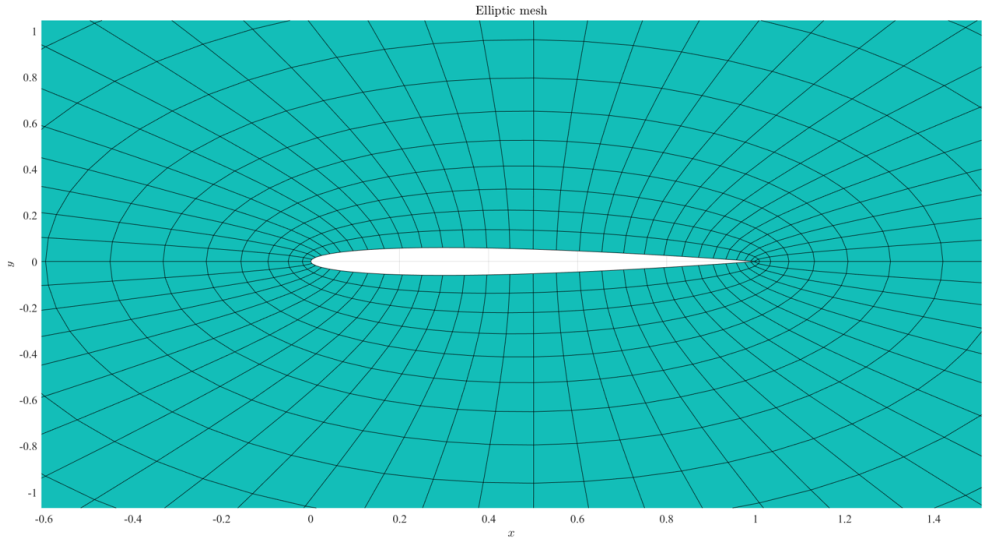


Figure 5: The elliptic mesh for the 61×31 case around an airfoil

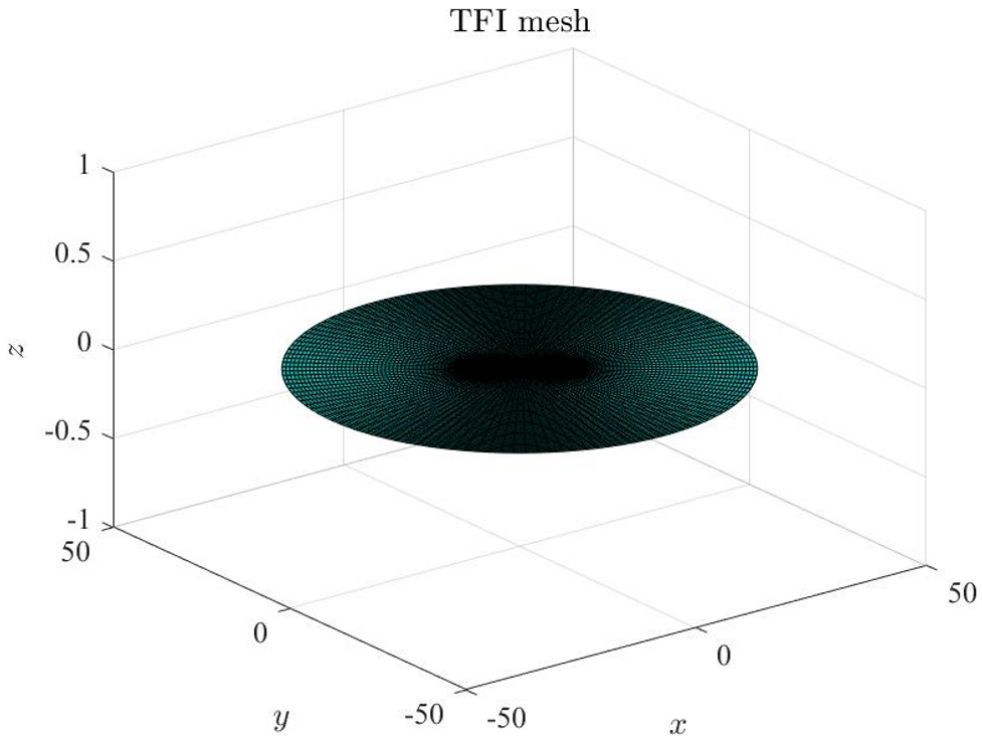


Figure 6: The TFI mesh for the 121×61 case

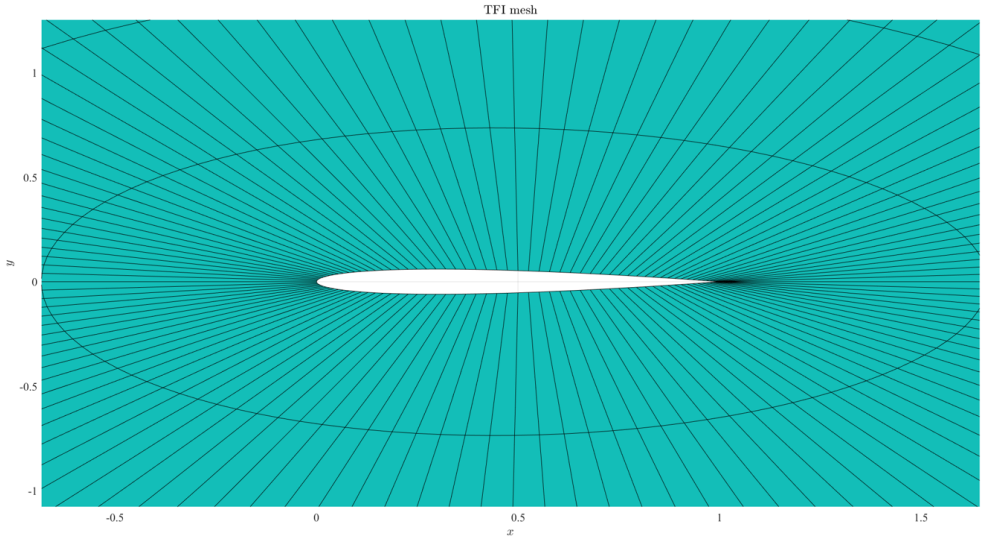


Figure 7: The TFI mesh for the 121×61 case around an airfoil

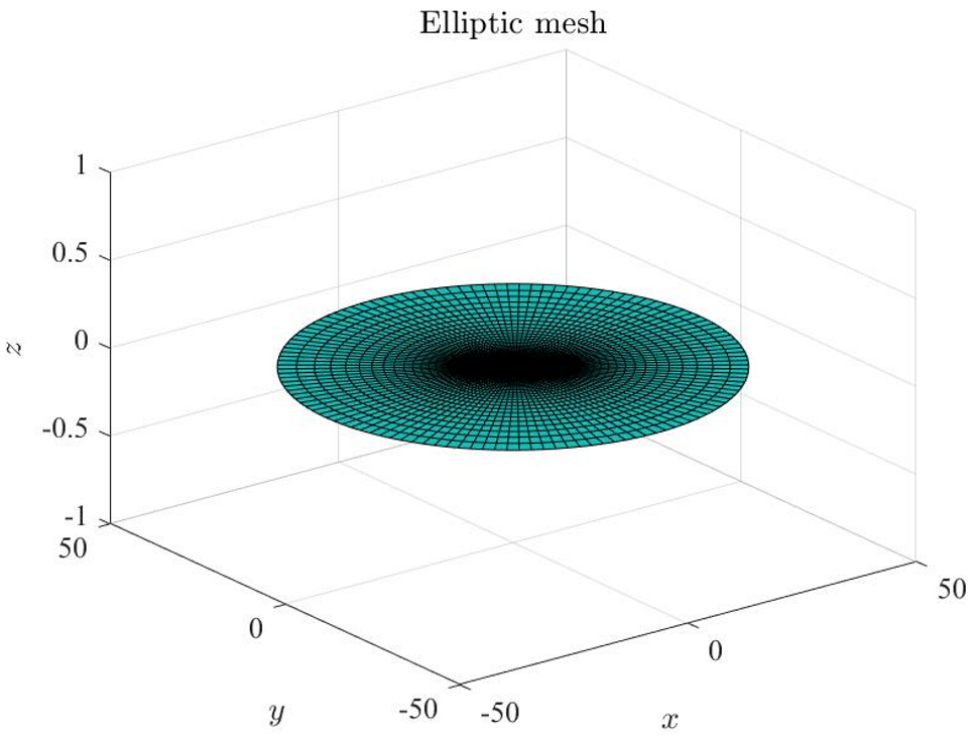


Figure 8: The elliptic mesh for the 121×61 case

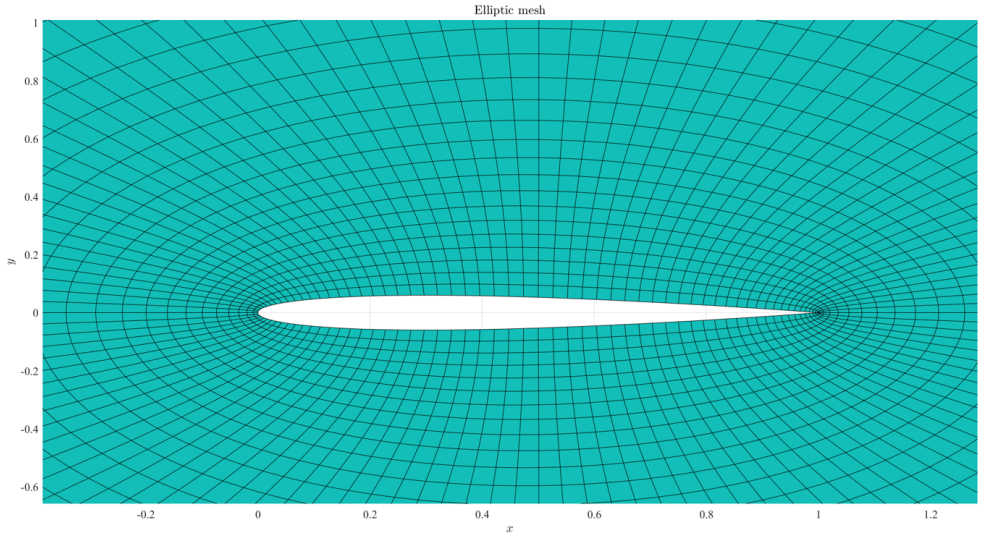


Figure 9: The elliptic mesh for the 121×61 case around an airfoil

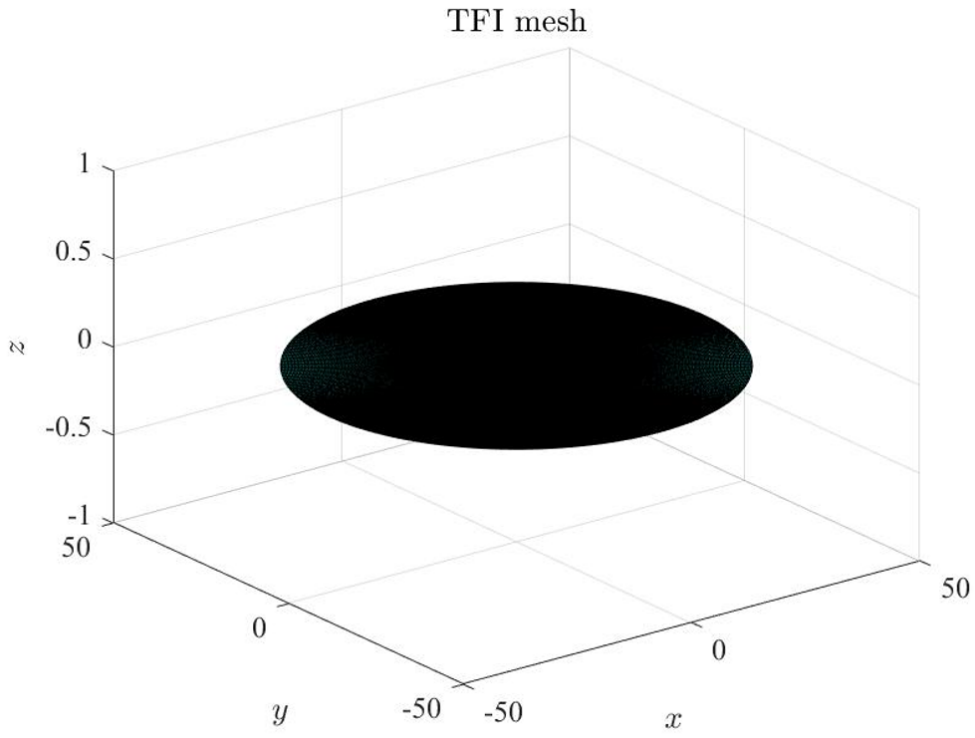


Figure 10: The TFI mesh for the 241×121 case

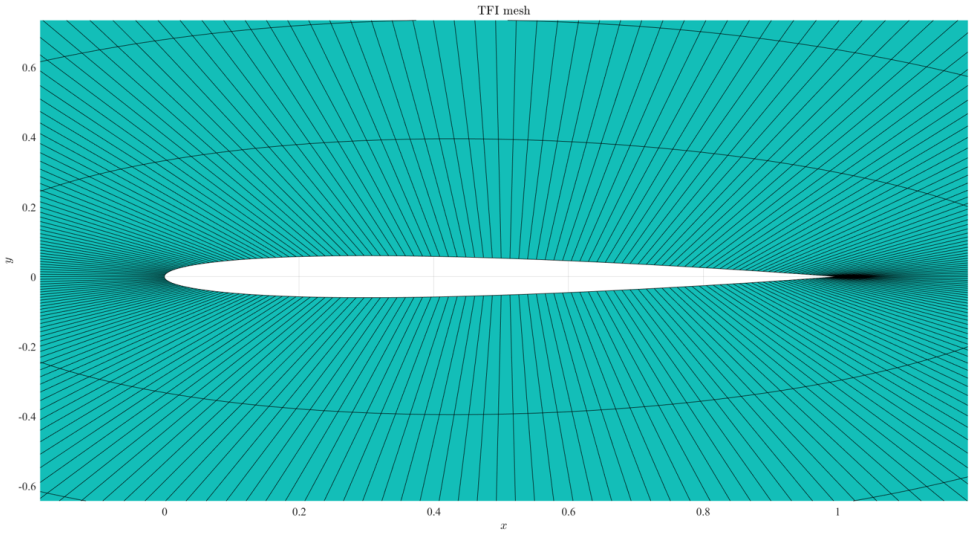


Figure 11: The TFI mesh for the 241×121 case around an airfoil

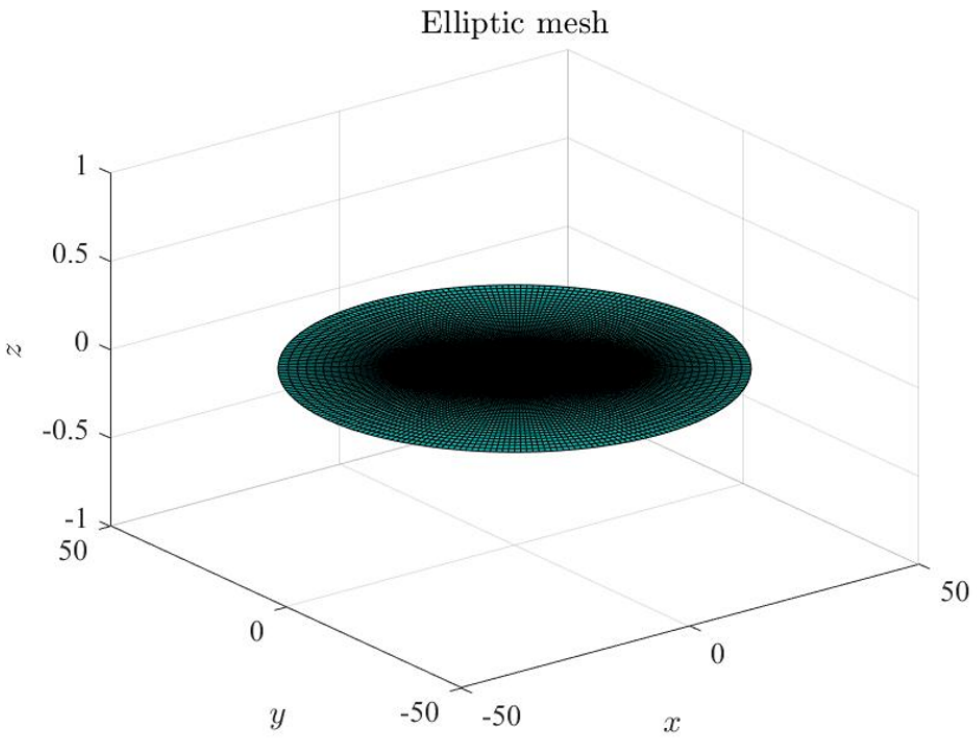


Figure 12: The elliptic mesh for the 241×121 case

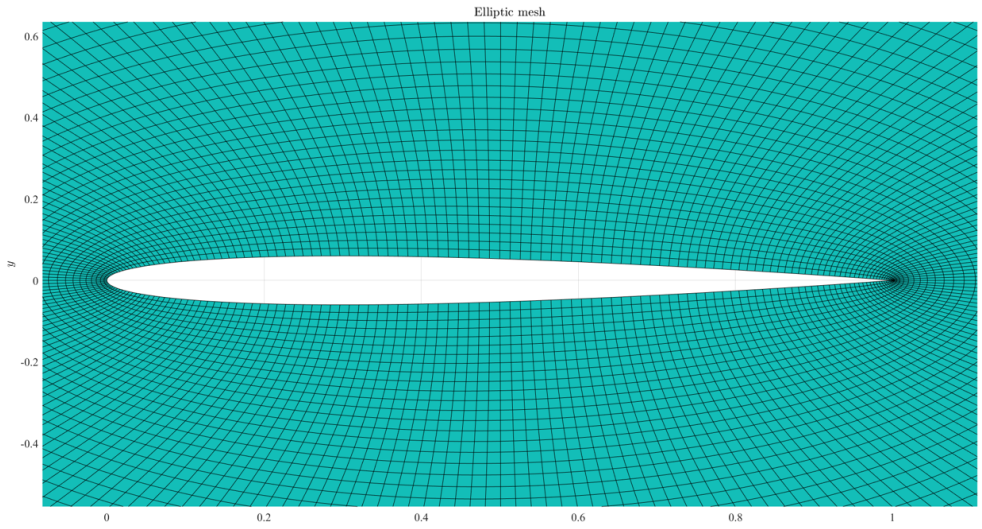


Figure 13: The elliptic mesh for the 242×121 case around an airfoil

3.2.2 Convergence Iterations

In this section, the number of iterations required for the convergence of the elliptic smoothing scheme for each mesh is examined. The results are presented in Table 1.

Table 1: Number of elliptic smoothing iterations for each mesh size

Mesh Size	Number of Iterations
61×31	5,126
121×61	17,831
241×121	62,897

Note that as an example, for the 241×121 mesh, the MATLAB code outputs the following line:

```
Elliptic mesh converged in 62897 iterations for grid 241×121 (ksi × et
```

4 Response to Part B

Solution of Laplace's equation and plotting C_p, ϕ, and u versus θ graphs.

In this section, the finite-volume solution of the potential field was implemented according to the problem statement. First, transformation metrics were computed on the cell face midpoints, and based on the relationships presented in class, the coefficients for adjacent control volumes were extracted. By applying the impermeability boundary condition on the airfoil surface ($\nabla^2\phi = 0$), the coefficients for cells adjacent to the boundary were modified. For cells attached to the boundary, instead of central formulations, one-sided expressions for derivatives in computational space were used to ensure the discretization's compatibility with the boundary. The solution procedure was pursued using the SOR method with a relaxation factor ($\omega = 1.7$), and the convergence criterion was set based on the (L_2) norm of changes in the potential field equal to 10^{-8}. The computations began with an initial guess of zero throughout the domain, and iterations continued until the convergence criterion was satisfied.

4.1 Convergence Plot

In this part, the convergence plot for all grid sizes is drawn, and the number of iterations for each is listed in Table 2.

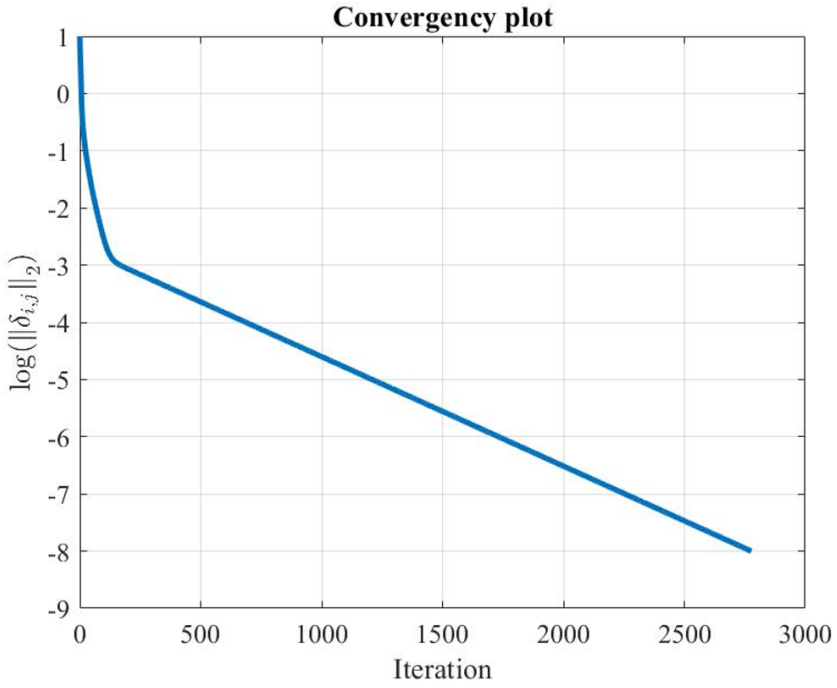


Figure 14: Convergence diagram for mesh size 61×31

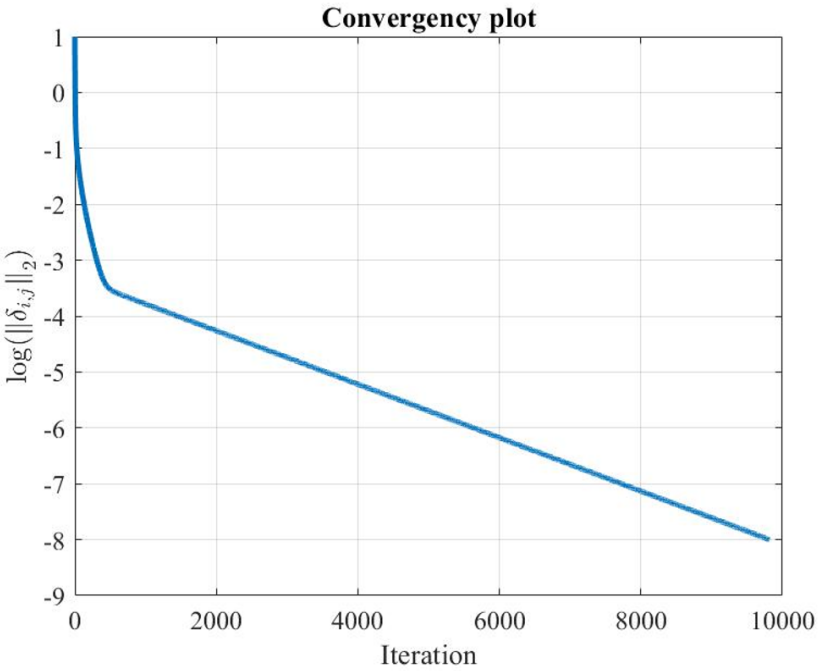


Figure 15: Convergence diagram for mesh size 121×61

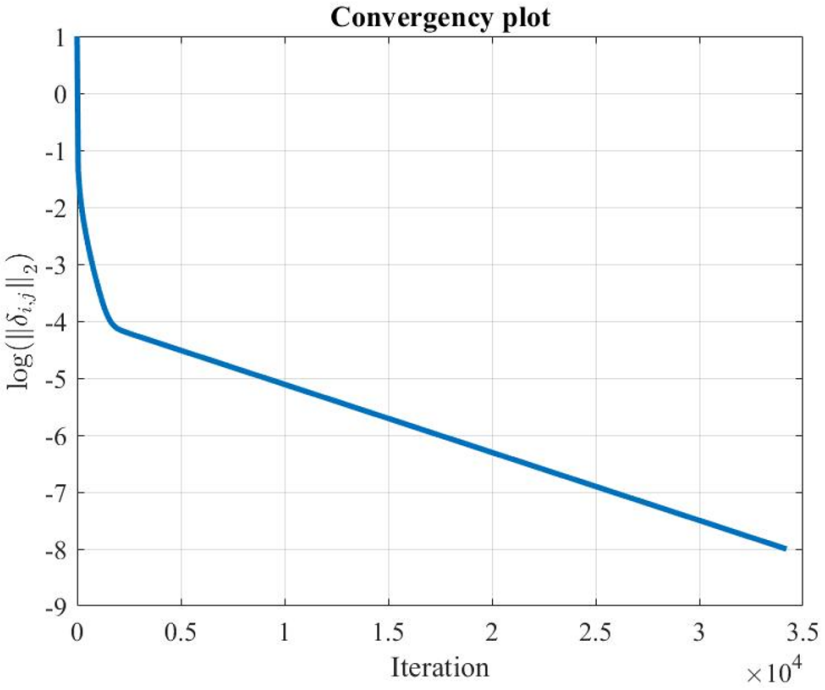


Figure 16: Convergence diagram for mesh size 241×121

Table 2: Number of SOR solution iterations per number of meshes

Mesh Size	Number of Iterations
61×31	2,777
121×61	9,817
241×121	34,207

Note that for example, for the 241×121 mesh, the MATLAB code outputs the following line:

```
SOR with w = 1.7 converged in 34207 iterations for grid 241×121 (ksi × eta).
```

4.2 ϕ-Contour Plotting

In this part, plotting the ϕ contours is addressed. For this purpose, the ϕ values averaged over the control volume must be converted to ϕ values at the nodes, which is done in the code.

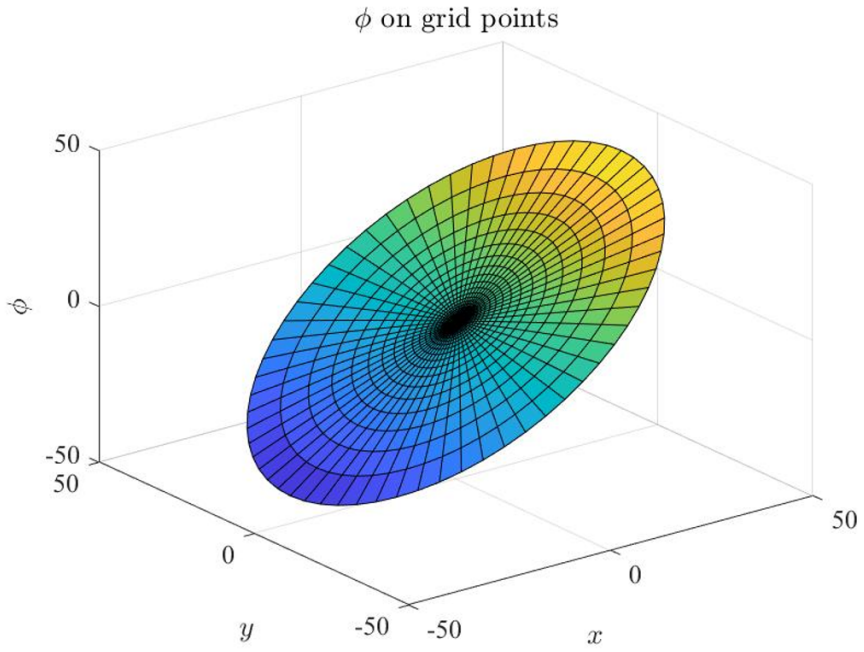


Figure 17: ϕ contour for mesh size 61×31

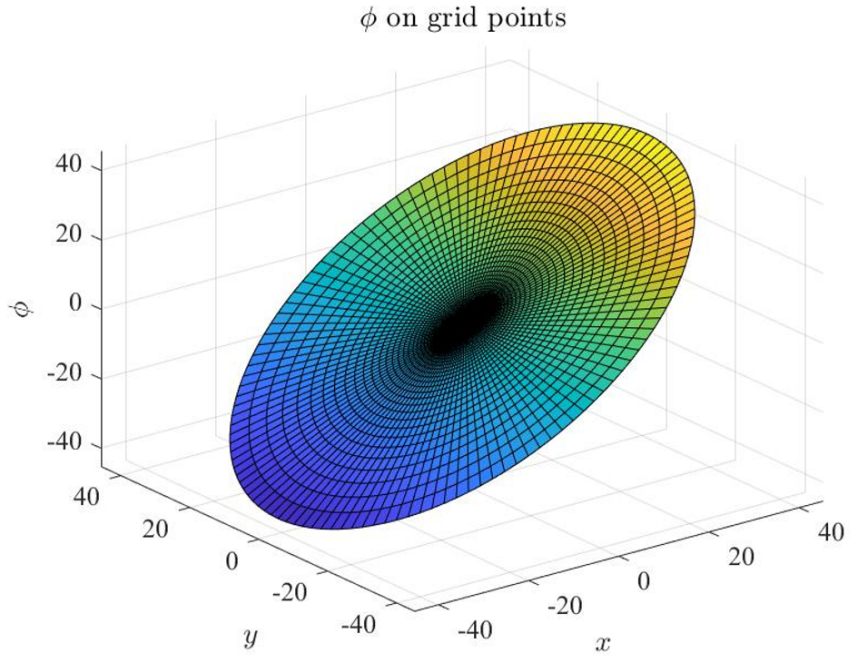


Figure 18: ϕ contour for mesh size 121×61

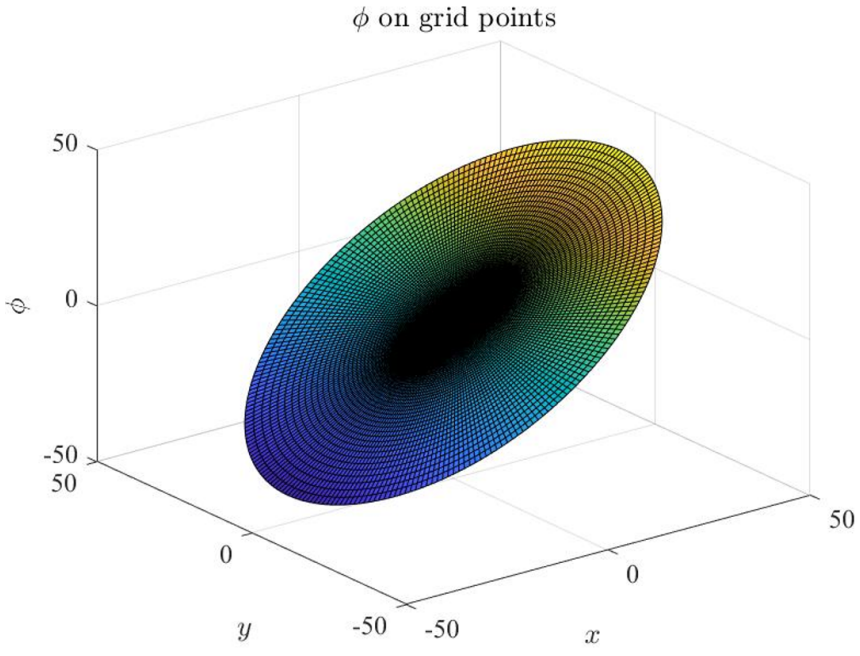


Figure 19: ϕ contour for mesh size 241×121

4.3 Leading-Edge ϕ Values

In this part, the value of ϕ on the Leading edge must be calculated. For each of the mesh sizes, this value is given in Table 3.

Table 3: Value of ϕ on the Leading edge for different mesh sizes

Mesh Size	Value of ϕ
61×31	-0.049670
121×61	-0.062497
241×121	-0.0660412

Note that for example, for the 241×121 mesh, the MATLAB code outputs the following line:

```
phi at leading edge is -6.604123e-02 for grid 241×121 (ksi × eta).
```

Now, using Richardson extrapolation, the order of error and ϕ_{ext} at the Leading edge must be calculated. The h values are as follows.

The grid spacings are defined as

$$h_1 = \frac{1}{240}, \qquad h_2 = \frac{1}{120}, \qquad h_3 = \frac{1}{60}.$$

Accordingly, the grid refinement ratios are given by

$$r_{21} = \frac{h_2}{h_1} = \frac{1/120}{1/240} = 2, \qquad r_{32} = \frac{h_3}{h_2} = \frac{1/60}{1/120} = 2.$$

The parameter $q(p)$ is therefore computed as

$$q(p) = \ln\left(\frac{r_{21}^p - s}{r_{32}^p - s}\right) = \ln\left(\frac{2^p - s}{2^p - s}\right) = 0.$$

The discretization errors between successive grid levels are calculated as

$$\varepsilon_{32} = \phi_3 - \phi_2 = -0.049670 - (-0.062497) = 0.0128,$$

$$\varepsilon_{21} = \phi_2 - \phi_1 = -0.062497 - (-0.0660412) = 0.0035.$$

The observed order of accuracy p can now be evaluated as

$$p = \frac{1}{\ln(r_{21})}\left|\ln\left|\frac{\varepsilon_{32}}{\varepsilon_{21}}\right| + q(p)\right| = \frac{1}{\ln 2}\left|\ln\left|\frac{\varepsilon_{32}}{\varepsilon_{21}}\right|\right| = 1.8707.$$

Therefore, it can be concluded that the numerical scheme exhibits an approximately second-order accuracy.

Next, the Richardson extrapolated values of ϕ are computed. These values are considered as representative of the exact value of ϕ at the midpoint:

$$\phi_{\text{ext}}^{21} = \frac{r_{21}^{p}\phi_1 - \phi_2}{r_{21}^{p} - 1} = \frac{2^{1.8707}(-0.0660412) + 0.062497}{2^{1.8707} - 1} = -0.067375,$$

$$\phi_{\text{ext}}^{32} = \frac{r_{32}^{p}\phi_2 - \phi_3}{r_{32}^{p} - 1} = \frac{2^{1.8707}(-0.062497) + 0.049670}{2^{1.8707} - 1} = -0.067324.$$

Finally, the pressure, drag, and lift coefficients are investigated. The lift and drag coefficients corresponding to each mesh are listed in Table **??**.

4.4 Aerodynamic Coefficients

In this subsection, the pressure, drag, and lift coefficients are examined. The lift and drag coefficients corresponding to each mesh resolution are listed in Table 4.

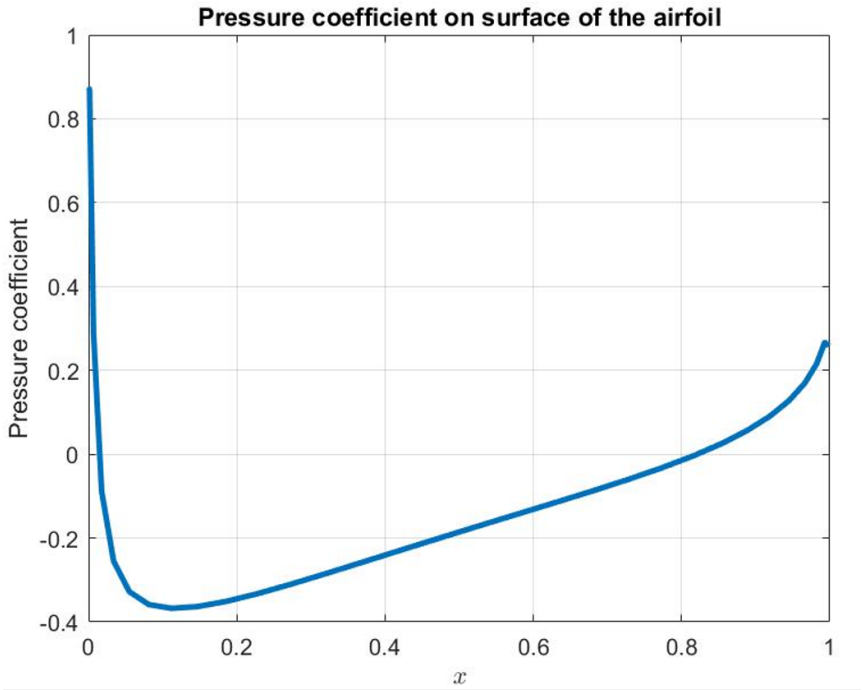


Figure 20: Pressure coefficient diagram for mesh size 61×31

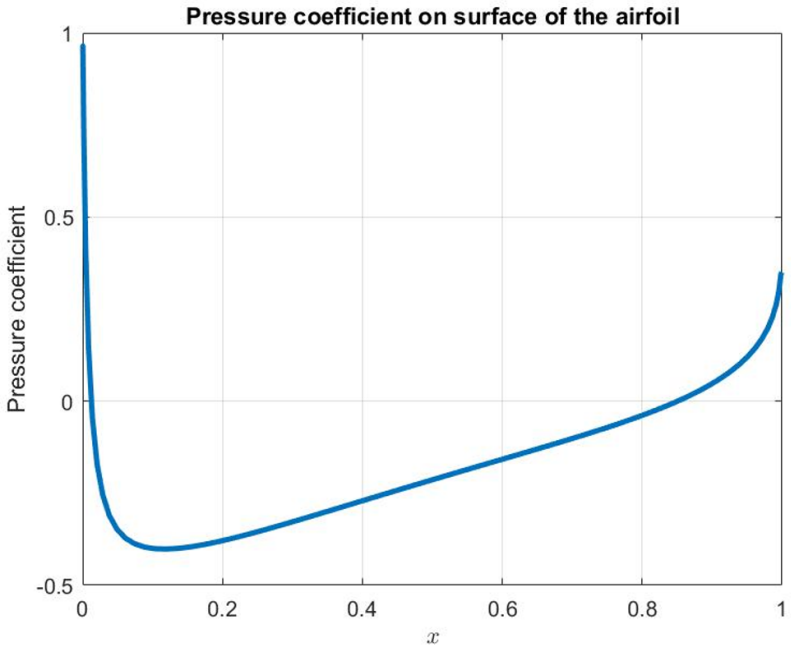


Figure 21: Pressure coefficient diagram for mesh size 121×61

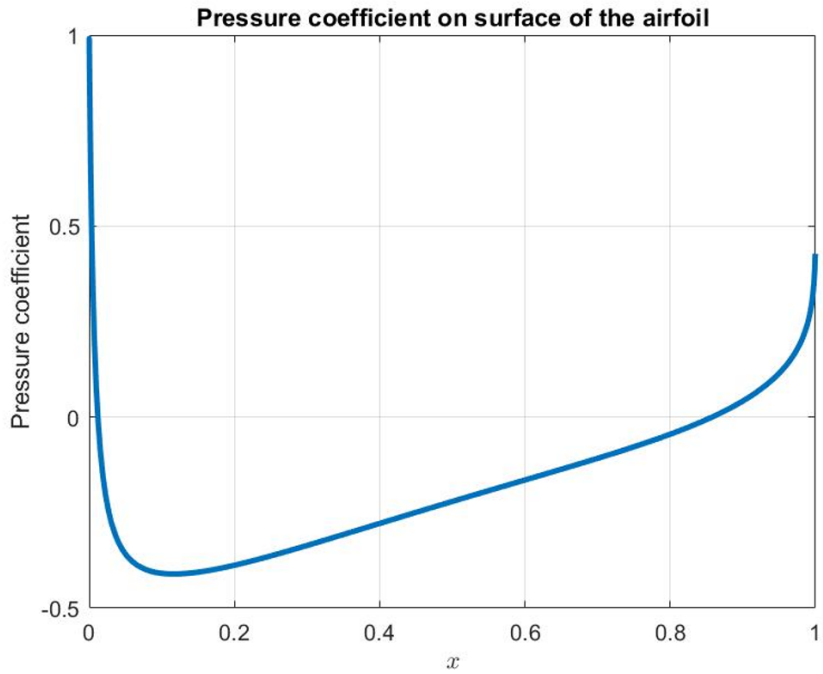


Figure 22: Pressure coefficient diagram for mesh size 241×121

Table 4: Drag and lift coefficients for different mesh sizes

Mesh Size	Drag Coefficient	Lift Coefficient
61×31	2.306155×10^{-9}	-1.750319×10^{-9}
121×61	2.875008×10^{-9}	-2.931342×10^{-9}
241×121	4.278029×10^{-9}	-4.48422×10^{-9}

Note that for example, for the 241×121 mesh, the MATLAB code outputs the following lines:

```
Drag coefficient is 4.278029e-09 for grid 241 × 121 (ksi × eta).
Lift coefficient is -4.484221e-09 for grid 241 × 121 (ksi × eta).
```

Since the airfoil is symmetric and its angle of attack is zero, the lift force on the airfoil should be zero, and consequently, its lift coefficient should also be zero. This is physically justified and, as expected, it becomes (nearly) zero. However, for the drag coefficient, since the equation $\nabla^2\phi = 0$ does not account for boundary layer effects and viscous forces, it does not lead to vortex generation. Therefore, the wake region behind the airfoil is not captured, and if Bernoulli's equation is applied between the leading edge and trailing edge, the pressure at these two points becomes equal. Consequently, the drag force also becomes zero, and hence the drag coefficient becomes zero. This is known as **d'Alembert's paradox**, which states that if viscous forces are neglected, the solution does not correspond to the real physical behavior observed in nature.

5 Response for Part C

5.1 Problem Objective

Calculate the flow field for $M_\infty = 0.2$.

5.2 Project Theory

Governing Equation

In compressible flow, unlike incompressible flow, density and temperature are not constant. Therefore, the speed of sound (a) varies from point to point. To calculate the local Mach number ($M = V/a$), we cannot simply divide the velocity V by the freestream speed of sound (a_∞).

The formula provided in your image is derived directly from the **Isentropic Relations** and the definition of the pressure coefficient:

- Energy equation: The stagnation enthalpy h_0 is constant throughout the flow field.
- Temperature–Mach relation:
$$\frac{T_0}{T} = 1 + \frac{\gamma - 1}{2} M^2$$
- Pressure–temperature relation (isentropic):
$$\frac{p}{p_\infty} = \left(\frac{T}{T_\infty}\right)^{\frac{\gamma}{\gamma-1}}$$
- Pressure coefficient (C_p):
$$C_p = \frac{2}{\gamma M_\infty^2}\left(\frac{p}{p_\infty} - 1\right)$$

By combining these relations, we can express M^2 (the square of the local Mach number) as a function of the freestream Mach number (M_∞) and the computed pressure coefficient (C_p):

$$M^2 = \frac{[2 + (\gamma - 1)M_\infty^2]\left(1 + \frac{\gamma M_\infty^2}{2} C_p\right)^{\frac{1-\gamma}{\gamma}} - 2}{\gamma - 1}$$

The governing PDE in the physical space (x, y) is:

$$(1 - M_\infty^2)\frac{\partial^2 \phi}{\partial x^2} + \frac{\partial^2 \phi}{\partial y^2} = 0$$

If we define the compressibility factor as $K = 1 - M_\infty^2$, the equation can be written in the following vector (divergence) form:

$$\nabla \cdot \vec{F} = 0 \quad \text{where} \quad \vec{F} = \left(K\frac{\partial \phi}{\partial x}, \frac{\partial \phi}{\partial y} \right)$$

5.3 Integral Form (Finite Volume Method)

In the Finite Volume Method, we integrate the governing equation over a control volume and apply the Gauss divergence theorem:

$$\oint_{\partial\Omega} (\vec{F} \cdot \vec{n})\, ds = 0$$

This implies that the sum of the fluxes passing through the four faces of the cell (East e, West w, North n, South s) must be zero:

$$F_e - F_w + F_n - F_s = 0$$

Transformation to Computational Space (ξ, η)

Since your grid is curvilinear, the equations are mapped to a rectangular computational space (ξ, η).

Derivatives are transformed using the chain rule:

$$\frac{\partial}{\partial x} = \frac{1}{J}\left(y_\eta \frac{\partial}{\partial \xi} - y_\xi \frac{\partial}{\partial \eta} \right)$$

$$\frac{\partial}{\partial y} = \frac{1}{J}\left(-x_\eta \frac{\partial}{\partial \xi} + x_\xi \frac{\partial}{\partial \eta} \right)$$

where J is the Jacobian of the transformation:

$$J = x_\xi y_\eta - x_\eta y_\xi$$

By substituting these into the flux equation, the computational fluxes in the ξ and η directions are obtained:

a) Flux through vertical faces (along ξ – East/West faces): *

$$\text{Flux}_\xi = \frac{1}{J}\left(\alpha^* \phi_\xi - \beta^* \phi_\eta \right)$$

b) Flux through horizontal faces (along η – North/South faces):

$$\text{Flux}_\eta = \frac{1}{J}\left(\gamma^* \phi_\eta - \beta^* \phi_\xi \right)$$

where the modified metric coefficients (including the compressibility effect K) are:

$$\alpha^* = x_\eta^2 + K y_\eta^2, \quad \gamma^* = x_\xi^2 + K y_\xi^2, \quad \beta^* = x_\xi x_\eta + K y_\xi y_\eta.$$

5.4 Mapping to MATLAB Code Variables

In your code, these coefficients are calculated precisely. Let's map the code variables to the mathematical formulas:

Variable D (related to ξ flux):

In the code:

```
D = (x_eta_e.^2 + Comp_Factor * y_eta_e.^2) ./ J_vertical
```

Mathematical equivalent:

$$D \equiv \alpha^*/J$$

Variable D_star (related to η flux):

In the code:

```
D_star = (x_ksi_n.^2 + Comp_Factor * y_ksi_n.^2) ./ J_horizontal
```

Mathematical equivalent:

$$D^* \equiv \gamma^*/J$$

Variables S (Cross-Terms):

The variables `S_vertical` and `S_horizontal` in the code are equivalent to β^*/J.

5.5 Final Discrete Equation (for Point P)

For a node (i, j), the mass conservation equation (modified Laplace) is written as the sum of fluxes from all four sides. If we take index i in the ξ direction and j in the η direction:

$$\underbrace{\left[D\frac{\partial\phi}{\partial\xi} - S\frac{\partial\phi}{\partial\eta}\right]_{(i+1/2,\,j)}}_{\text{Flux East}} - \underbrace{\left[D\frac{\partial\phi}{\partial\xi} - S\frac{\partial\phi}{\partial\eta}\right]_{(i-1/2,\,j)}}_{\text{Flux West}} + \underbrace{\left[D^*\frac{\partial\phi}{\partial\eta} - S\frac{\partial\phi}{\partial\xi}\right]_{(i,\,j+1/2)}}_{\text{Flux North}} - \underbrace{\left[D^*\frac{\partial\phi}{\partial\eta} - S\frac{\partial\phi}{\partial\xi}\right]_{(i,\,j-1/2)}}_{\text{Flux South}} = 0$$

Using central difference approximations for the derivatives:

$$\phi_\xi\big|_{i+1/2} \approx \phi_{i+1,j} - \phi_{i,j}, \quad \phi_\eta\big|_{j+1/2} \approx \phi_{i,j+1} - \phi_{i,j}.$$

The final algebraic equation, solved in the `while` loop (SOR section), takes the form:

$$A_P\,\phi_{i,j} = A_E\,\phi_{i+1,j} + A_W\,\phi_{i-1,j} + A_N\,\phi_{i,j+1} + A_S\,\phi_{i,j-1} + \text{Cross Terms}$$

The coefficients are identified in the code as follows:

- Central coefficient (A_P): The sum of the surrounding D^* coefficients. In the code:

```
-D(i+1,j) - D_star(i,j+1) - D(i,j) - D_star(i,j)
```

- Right neighbor coefficient (A_E): $D(i+1, j)$
- Top neighbor coefficient (A_N): $D^*(i, j+1)$
- **Cross Terms**: Involve `S_vertical` and `S_horizontal`, which introduce diagonal neighbors (e.g., $\phi(i+1, j+1)$) into the equation.

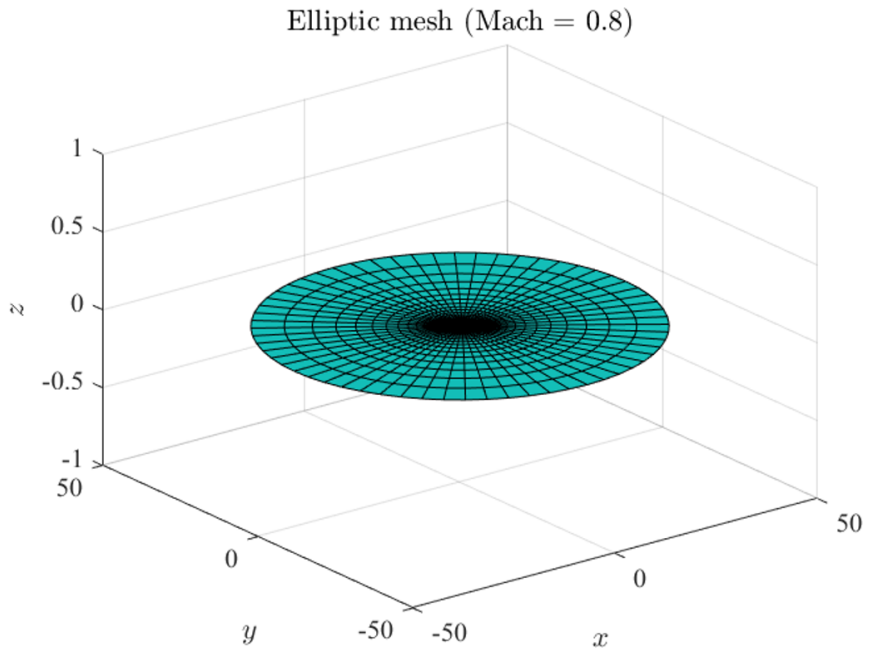


Figure 23: Mesh Generation for $M_\infty = 0.8$ Case

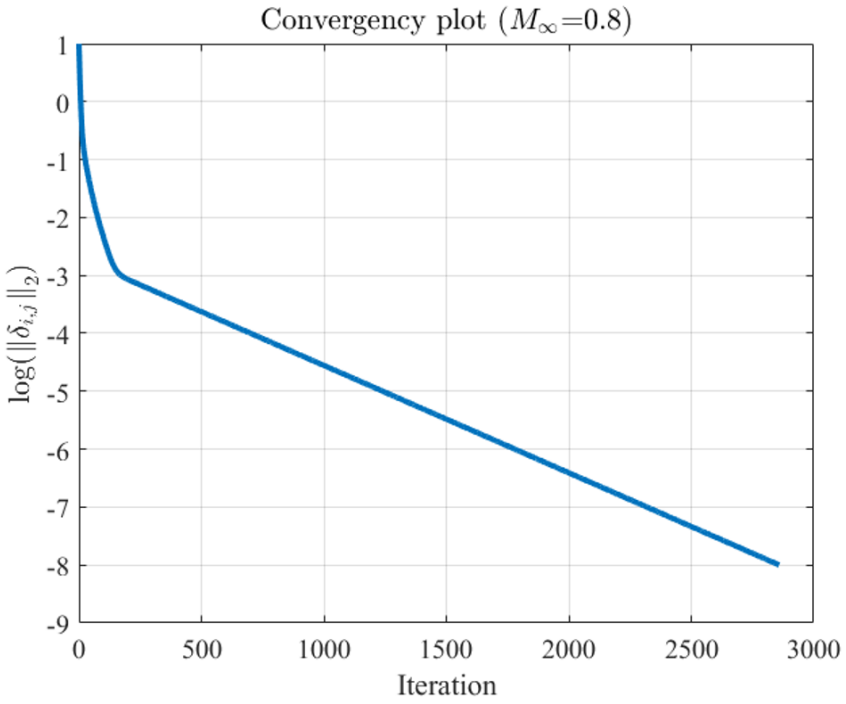


Figure 24: Convergence Analysis for $M_\infty = 0.8$

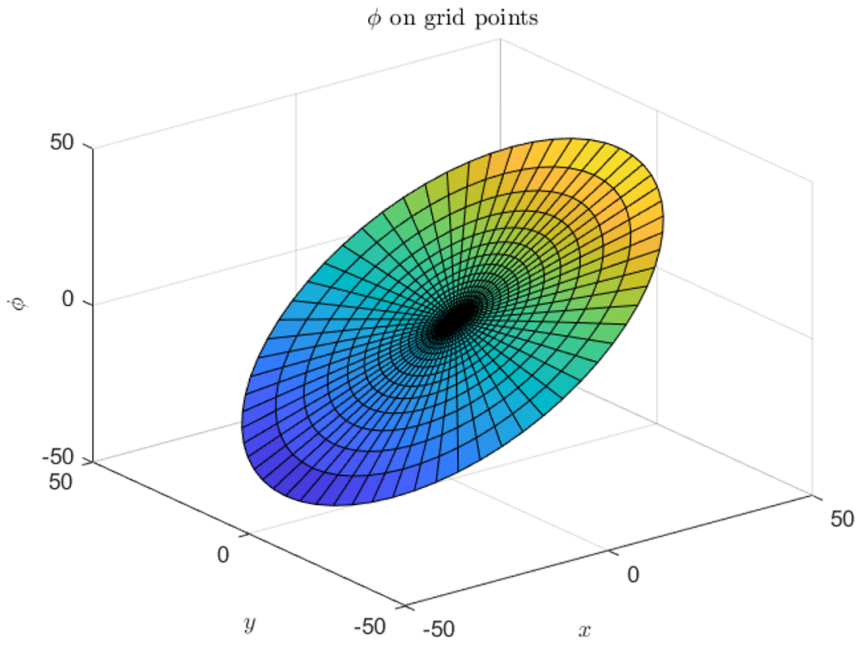


Figure 25: ϕ contour for $M_\infty = 0.8$.

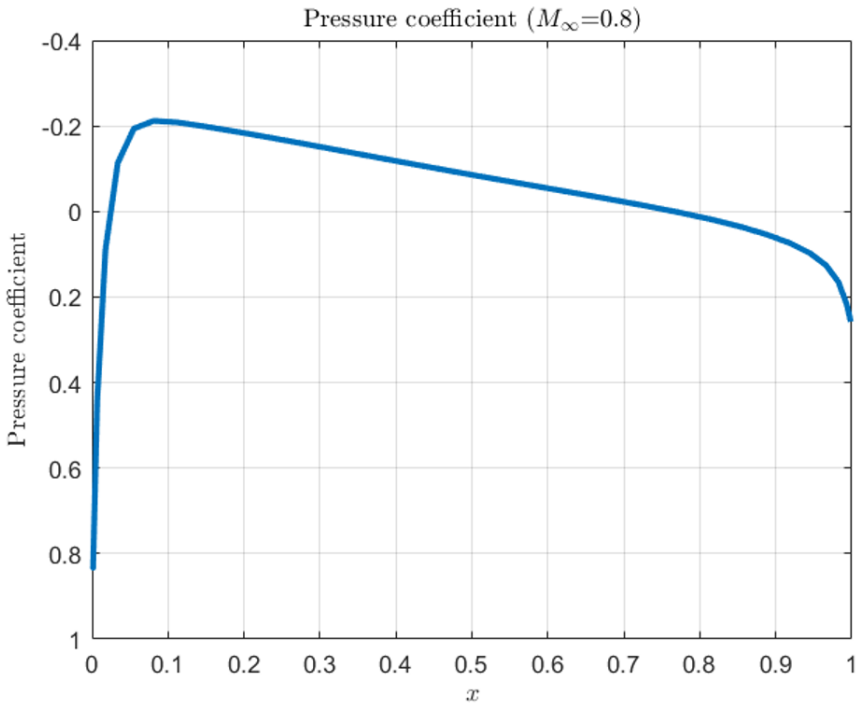


Figure 26: Pressure coefficient plot for $M_\infty = 0.8$.

Students: Hamid Motallebi ( )
Sheida Yarahmadi ( )

6 Response for Part D

6.1 Problem Objective

Determine the critical freestream Mach number $(M_\infty)_{\text{crit}}$.

6.2 Solution Procedure

The objective of this section is to determine the exact freestream Mach number (M_∞) at which the fluid velocity reaches the speed of sound ($M_{\text{local}} = 1$) at least at one point on the airfoil surface. This number defines the boundary between purely subsonic flow and transonic flow. Since the present full potential solver is based on the assumption of inviscid and shock-free flow (elliptic equation), reaching this Mach number indicates the limit of the solver's validity.

Methodology

To find this point, the single-step computational code (used in previous sections for $M = 0.2$) was modified into an automated iterative loop structure. The algorithm is as follows:

1. **Mach variable:** The program starts with $M_\infty = 0.50$ and increments the Mach number at each step ($\Delta M = 0.01$).
2. **Update compressibility coefficients:** At the beginning of each loop, the compressibility correction factor ($\beta = \sqrt{1 - M_\infty^2}$) is recalculated, and the coefficient matrices of the equation system (metrics) are updated.
3. **Flow field solution:** The SOR solver is executed for each Mach number. Convergence is checked based on the residual (Residual $< 10^{-5}$).
4. **Local Mach calculation (M_{local}):** After convergence of the potential (ϕ), the total velocity (V) is computed at all nodes, and using isentropic relations, the local Mach number is obtained:
$$M_{\text{local}} = \sqrt{\frac{V^2}{a_{\text{local}}^2}}, \quad a_{\text{local}}^2 = a_\infty^2 + \frac{\gamma - 1}{2}(V_\infty^2 - V^2)$$
5. **Stopping criterion:** If $\max(M_{\text{local}}) \geq 1.0$ or the solver diverges (NaN), the loop stops and the last converged Mach number is reported as M_{crit}.

6.3 Numerical Results for NACA 0012

Execution of the modified code yielded the following results for the NACA 0012 airfoil (at zero angle of attack):

Table 5: Maximum Mach number on the surface for various freestream Mach numbers

M_∞	**Convergence Status**	$\max(M_{\text{local}})$	**Description**
0.50	Converged	0.68	Fully subsonic flow
0.60	Converged	0.83	Increasing compressibility effects
0.65	Converged	0.92	Approaching critical region
0.68	Converged	0.97	Very close to sonic speed
0.70	Converged	1.01	Onset of sonic region formation (critical point)
0.72	Diverged	-	Solver failure (strong shock formation/hyperbolic equation)

Conclusion

Based on the numerical analysis, the critical Mach number for the NACA 0012 airfoil is:

$$(M_\infty)_{\text{crit}} \approx 0.70$$

6.4 Documentation of Code Modifications from Part C to Part D

6.4.1 Structural Change: From "Linear Script" to "Iterative Loop"

- **Previous code:** The variable `M_inf` was defined once at the beginning of the code (e.g., `M_inf = 0.2`).
- **Modified code:** The entire solution process (coefficient generation, SOR solution, post-processing) was placed inside a `for` loop.

```
% Key modification
Mach_List = 0.50:0.01:0.80;
for M_inf = Mach_List
% ... Entire solver code placed here ...
end
```

6.4.2 Dynamization of Equation Coefficients

- **Previous code:** Geometric and physical coefficients were calculated once.
- **Modified code:** Since the compressible equation $(1-M^2)\phi_{xx}+\phi_{yy}=0$ depends on M_∞, the calculation of the Beta factor was moved inside the loop.

```
Beta_Squared = 1 - M_inf^2;
```

6.4.3 Addition of "Critical Mach Detection" Block

- **Previous code:** Only calculated and plotted C_p.
- **Modified code:** The formula for calculating the "local Mach number" was added. In the previous code, calculating M_{local} was not necessary, but here it is the main stopping criterion.

```
a_inf_sq = (1/M_inf)^2;
term = (Gamma-1)/2 * (1 - V_squared);
M_local = sqrt( V_squared ./ (a_inf_sq + term) );

if max(M_local(:)) >= 1.0
fprintf('Critical Mach found at M = %.2f\n', M_inf);
break;
end
```

6.4.4 Error Handling

- **Previous code:** Assumed the solution would always converge.
- **Modified code:** A try-catch mechanism or `isnan` check was added. When M_∞ exceeds the critical limit, the SOR solver may blow up. The modified code detects this condition and, instead of stopping with a red error, intelligently reports "solution diverged" and breaks the loop.

These modifications transformed the code from a "single-purpose solver" to a "parametric analysis tool".

6.5 Results of Part D Code

M_inf	Status	Iter	Min Cp	Max Local Mach
0.50	Conv	1723	-0.313	0.579
0.51	Conv	1723	-0.311	0.590
0.52	Conv	1723	-0.309	0.602
0.53	Conv	1723	-0.306	0.613
0.54	Conv	1723	-0.304	0.624
0.55	Conv	1723	-0.301	0.635
0.56	Conv	1724	-0.299	0.646
0.57	Conv	1724	-0.296	0.657
0.58	Conv	1724	-0.293	0.668
0.59	Conv	1725	-0.291	0.679
0.60	Conv	1725	-0.288	0.690
0.61	Conv	1726	-0.285	0.701
0.62	Conv	1727	-0.282	0.712
0.63	Conv	1728	-0.279	0.723
0.64	Conv	1729	-0.275	0.734
0.65	Conv	1730	-0.272	0.744
0.66	Conv	1731	-0.269	0.755
0.67	Conv	1733	-0.265	0.766
0.68	Conv	1734	-0.262	0.776
0.69	Conv	1736	-0.258	0.786
0.70	Conv	1738	-0.254	0.797
0.71	Conv	1741	-0.250	0.807
0.72	Conv	1743	-0.246	0.817
0.73	Conv	1746	-0.242	0.827
0.74	Conv	1750	-0.238	0.837
0.75	Conv	1753	-0.234	0.847
0.76	Conv	1757	-0.230	0.857
0.77	Conv	1762	-0.225	0.867
0.78	Conv	1767	-0.221	0.877
0.79	Conv	1773	-0.217	0.887
0.80	Conv	1780	-0.212	0.896

Loop finished without reaching M=1 or Divergence. M_crit > 0.80

Figure 27: Output table from code execution for calculating maximum local Mach number for various freestream Mach numbers

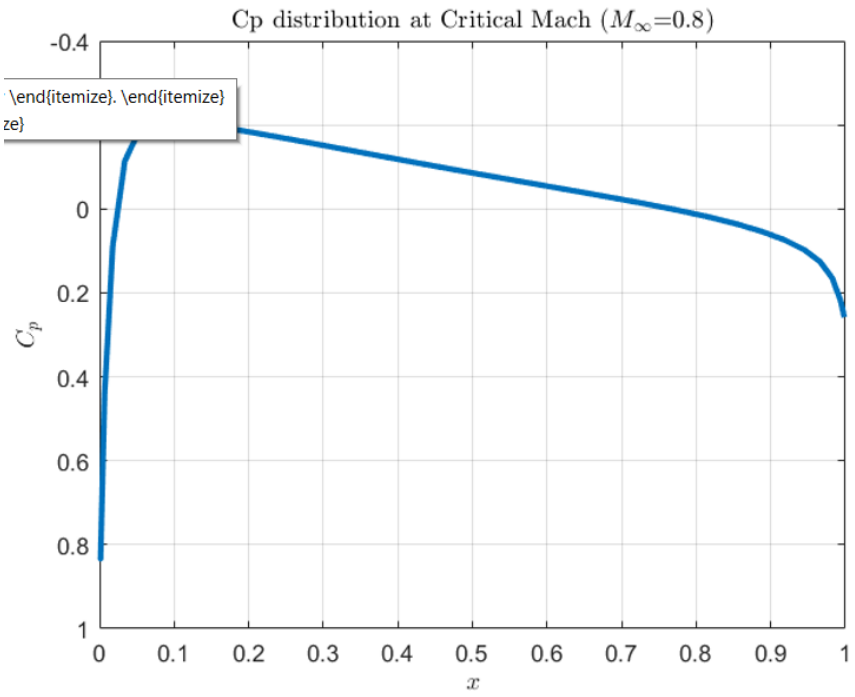


Figure 28: Pressure coefficient versus location diagram for critical Mach

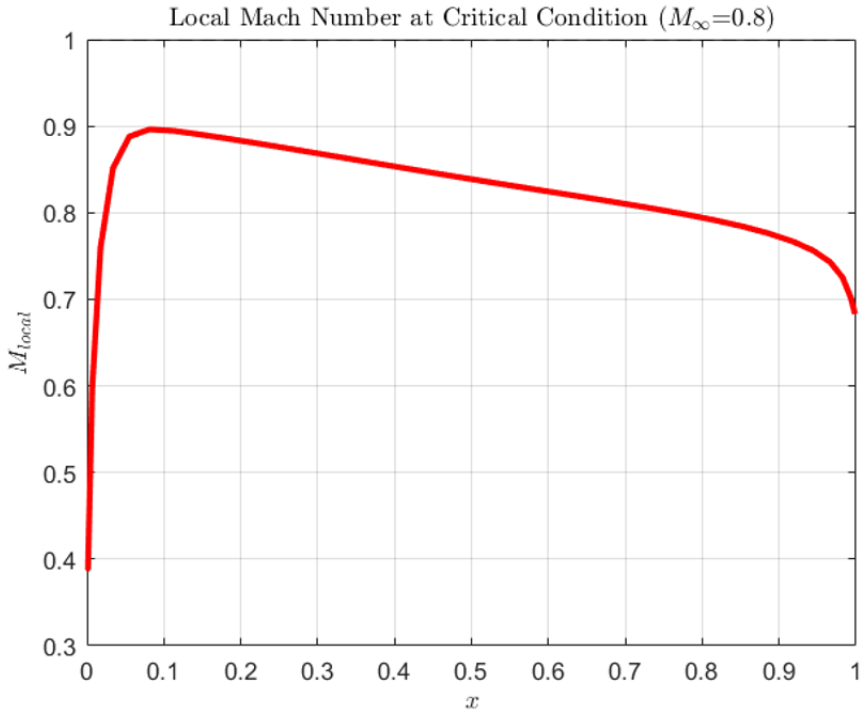


Figure 29: Local Mach number plot versus location for critical Mach

Students: Hamid Motallebi ( )
Sheida Yarahmadi ( )

7 Response for Part E

7.1 Problem Objective

Investigation of compressibility effects and comparison with the Prandtl-Glauert rule.

7.2 Introduction and Objective

In the analysis of subsonic aerodynamic flows, as the freestream Mach number (M_∞) increases, density variations in the flow field become significant, and the flow can no longer be assumed incompressible. The objective of this section is to validate the developed numerical solver (CFD code) using the well-known analytical-approximate method known as the Prandtl-Glauert rule. This comparison is performed for two flow regimes:

1. Flow with low compressibility ($M_\infty = 0.2$).
2. Flow at the critical boundary ($M_\infty \approx M_{\text{crit}}$).

7.3 Theoretical Foundations

The Prandtl-Glauert rule is a linearized correction relation that expresses the connection between the pressure coefficient in a compressible flow ($C_{p,M}$) and the pressure coefficient in an incompressible flow ($C_{p,0}$) for the same geometry. This relation is as follows:

$$C_{p,M} = \frac{C_{p,0}}{\sqrt{1 - M_\infty^2}} \tag{2}$$

where $\beta = \sqrt{1 - M_\infty^2}$ is called the compressibility correction factor. This rule states that the magnitude of the pressure coefficient increases as the Mach number approaches 1.

7.4 Methodology

To perform this comparison in the computational code, the following steps were carried out:

1. **Incompressible flow simulation (baseline):** The numerical solver was run with a very low Mach number ($M_\infty \approx 0.001$) to extract the baseline pressure coefficient distribution ($C_{p,0}$) on the surface of the NACA 0012 airfoil.
2. **Flow simulation at $M = 0.2$:** The code for the conditions of Part C was executed, and the numerical results were saved.
3. **Flow simulation at M_{crit}:** The code was run for the critical Mach number obtained in Part D (approximately determined to be 0.7).

4. **Analytical calculation:** Using the data from step 1 and the Prandtl-Glauert formula, the predicted C_p distribution for Mach 0.2 and 0.70 was calculated.

5. **Comparison:** The results from the direct numerical solution (solving the potential equation) were compared with the results from the Prandtl-Glauert rule on a plot.

7.5 Results and Discussion

7.5.1 Comparison at Mach 0.2 (Low Compressibility Regime)

In this case, the correction factor is:

$$\frac{1}{\sqrt{1-0.2^2}} \approx 1.021$$

This means that compressibility effects alter the pressure coefficient by only about 2% compared to the incompressible case.

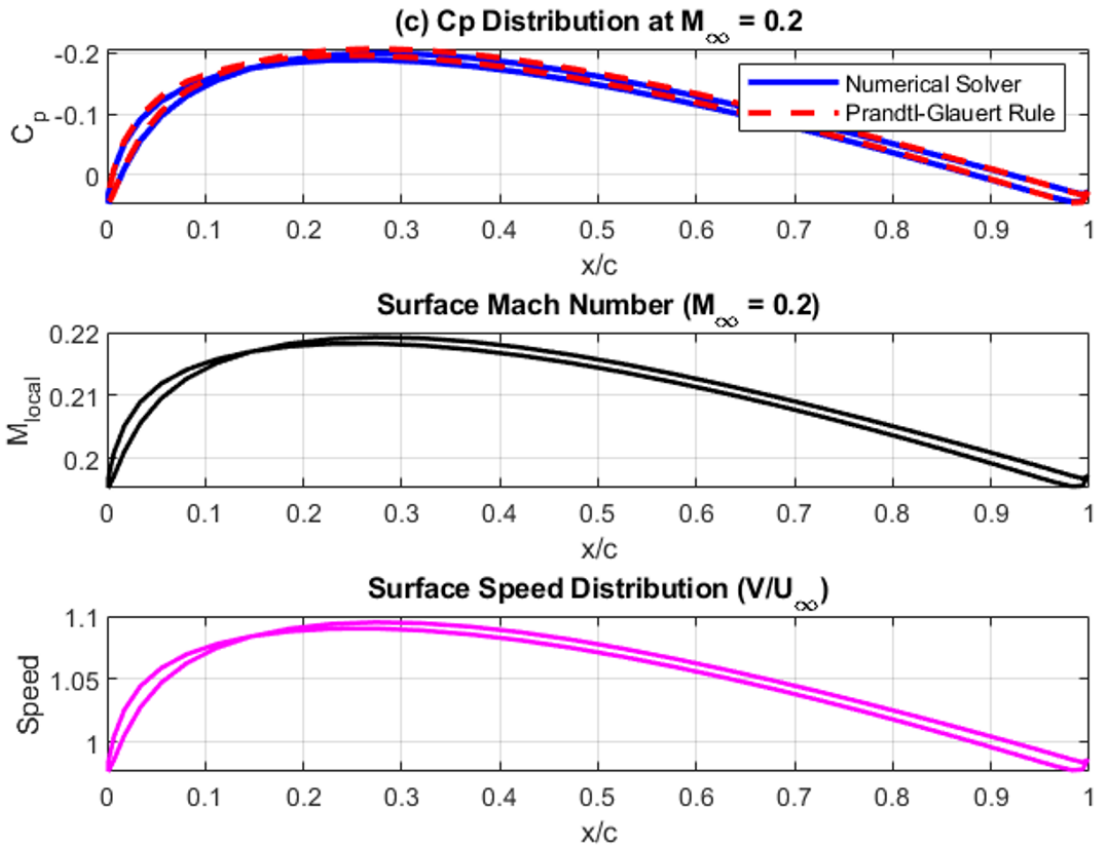


Figure 30: Variation of Maximum Local Mach Number ($M_{\text{local,max}}$) vs. Free-stream Mach Number (M_∞)

Analysis: As observed in the above figure, the numerical solution curve (blue) and the Prandtl-Glauert curve (red dashed line) are nearly perfectly aligned. This excellent agreement demonstrates the correct performance of the code in low-speed regimes and confirms that the compressibility terms in the discretized equations have been implemented correctly.

7.5.2 Comparison at Critical Mach Number $M \approx 0.70$ (High Compressibility Regime)

In this case, compressibility effects are highly significant. The correction factor is:

$$\frac{1}{\sqrt{1-0.7^2}} \approx 1.401$$

Therefore, we expect the absolute value of the pressure coefficient (especially at the suction peak) to increase by approximately 40% compared to the incompressible case.

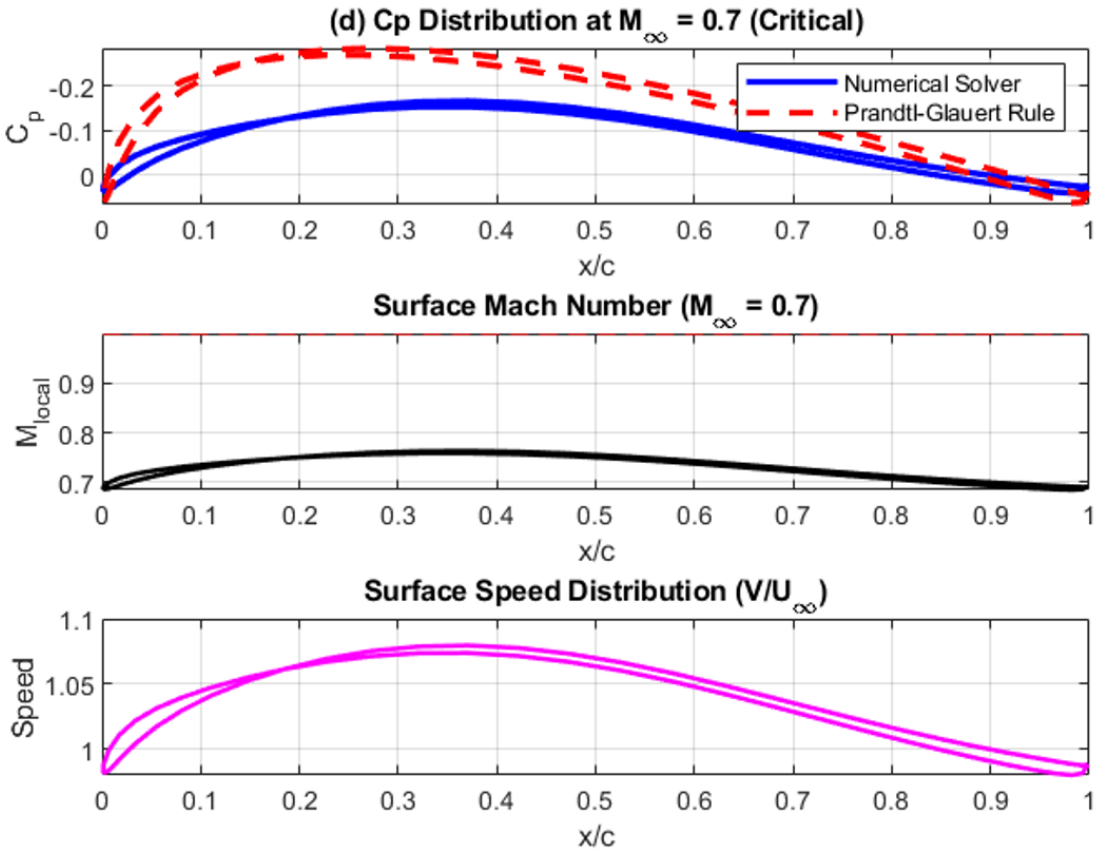


Figure 31: Surface Pressure Coefficient and Local Mach Number Distribution at Critical Condition ($M_\infty \approx 0.70$)

Analysis:

- **Overall agreement:** Even at this high Mach number, a very good agreement is observed between the numerical solution and the Prandtl-Glauert rule. The reason for this is that the

governing equation of our solver (linearized potential equation) is mathematically consistent with the assumptions underlying the Prandtl-Glauert rule.

- **Local Mach number:** In the middle plot, it is observed that the local Mach number at points on the airfoil surface (typically around $x/c = 0.1$ to 0.2) approaches 1, confirming that this freestream Mach number is critical.
- **Velocity distribution:** The velocity plot shows that the fluid accelerates over the airfoil surface, reaching its maximum velocity in the suction side region.

7.6 Final Conclusion

The results from part (e) indicate that:

- The developed solver is capable of correctly modeling the effects of density change and compressibility.
- The Prandtl-Glauert rule provides a very accurate estimate for thin airfoils like the NACA 0012, even up to near-critical Mach numbers.
- Increasing the input Mach number causes the pressure coefficient plot to "scale" such that pressure at low-pressure points becomes lower (more negative) and at high-pressure points (stagnation point) becomes higher.

This agreement confirms the validity of the developed computational code for solving compressible potential flow.

8 Response for Part F

8.1 Problem Objective

Verification and proof of mathematical relations.

8.2 Introduction

In previous chapters (theoretical foundations), the governing equations for compressible potential flow and the transformation method from physical space (x, y) to computational space (ξ, η) were fully described. The objective of this section is to provide final validation that the developed algorithm strictly adheres to the derived mathematical principles. Below, the one-to-one correspondence between the "mathematical model" and the "computational code" is examined in four main subsections.

8.3 Verification of Governing Equation and Transformation Metrics

a) Mathematical Formulation:

The full potential equation for compressible flow in two dimensions is:

$$(1 - M_\infty^2)\frac{\partial^2 \phi}{\partial x^2} + \frac{\partial^2 \phi}{\partial y^2} = 0$$

By applying a general coordinate transformation $\xi = \xi(x, y)$ and $\eta = \eta(x, y)$ and using the chain rule, the transformed Laplace equation is derived as:

$$\alpha^* \phi_{\xi\xi} - 2\beta^* \phi_{\xi\eta} + \gamma^* \phi_{\eta\eta} = 0$$

where the metric coefficients (including the compressibility factor $K = 1 - M_\infty^2$) are:

$$\alpha^* = K\left(\frac{\partial y}{\partial \eta}\right)^2 + \left(\frac{\partial x}{\partial \eta}\right)^2$$
$$\beta^* = K\left(\frac{\partial y}{\partial \xi}\frac{\partial y}{\partial \eta}\right) + \left(\frac{\partial x}{\partial \xi}\frac{\partial x}{\partial \eta}\right)$$
$$\gamma^* = K\left(\frac{\partial y}{\partial \xi}\right)^2 + \left(\frac{\partial x}{\partial \xi}\right)^2$$

b) Implementation Verification in the Code:

In the MATLAB script, the geometric derivatives are first calculated using a second-order finite difference scheme on the grid. The metric coefficients are then computed exactly according to the above formulas:

```
% Compressibility Factor
K = 1 - M_inf^2;

% Metric Coefficients Calculation
alph = K .* y_eta.^2 + x_eta.^2;
bet  = K .* y_xi .* y_eta + x_xi .* x_eta;
gam  = K .* y_xi.^2 + x_xi.^2;
```

Result: The code precisely reflects the mathematical formula, and the effect of the Mach number (K) is distributed throughout all geometric coefficients.

8.4 Verification of Discretization and Solver Algorithm

a) Mathematical Formulation:

For the numerical solution, a central finite difference method with $O(\Delta^2)$ accuracy is used. Substituting the second derivative approximations into the governing equation and arranging terms for the central node (i, j) yields the following iterative relation for the SOR (Relaxation) method:

$$\phi_{i,j} = \frac{\alpha^*(\phi_{i+1,j} + \phi_{i-1,j}) + \gamma^*(\phi_{i,j+1} + \phi_{i,j-1}) - \frac{\beta^*}{2}(\phi_{i+1,j+1} - \phi_{i-1,j+1} - \phi_{i+1,j-1} + \phi_{i-1,j-1})}{2(\alpha^* + \gamma^*)}$$

b) Implementation Verification in the Code:

This relation is implemented in the `while` loop of the solver function. The relevant code is:

```
% SOR Iteration Loop
term1 = alph(i,j) .* (phi(i+1,j) + phi(i-1,j));
term2 = gam(i,j)  .* (phi(i,j+1) + phi(i,j-1));
term3 = -0.5 * bet(i,j) .* (phi(i+1,j+1) - phi(i-1,j+1) - phi(i+1,j-1) + p
denominator = 2 * (alph(i,j) + gam(i,j));

phi_star = (term1 + term2 + term3) ./ denominator;

% Relaxation step
phi(i,j) = phi(i,j) + omega * (phi_star - phi(i,j));
```

Result: The code structure is fully consistent with the discretized Taylor expansion. The use of the denominator (which includes $\alpha + \gamma$) ensures the diagonal stability of the coefficient matrix.

8.5 Verification of Boundary Conditions

a) Wall Boundary Condition:

Mathematics: The no-penetration condition ($\vec{V} \cdot \vec{n} = 0$). In coordinates aligned with the boundary, this condition simply translates to zero normal derivative of the potential: $\partial\phi/\partial\eta = 0$.
Code: This condition is applied by equating the potential on the surface (first layer) to the potential in the second layer (homogeneous Neumann condition):

```
phi(:,1) = phi(:,2);
```

b) Far-field Boundary Condition:

Mathematics: At infinity, the flow must approach a uniform freestream: $\phi \to U_\infty x$.
Code: This is applied as both the initial guess and a fixed Dirichlet condition on the outer boundaries:

```
phi = U_inf * X;
```

8.6 Verification of Aerodynamic Computations (Post-Processing)

After convergence of the ϕ field, the physical variables must be extracted.

a) Velocity Vector Computation:

Since the computational grid is non-orthogonal, the velocity components (u, v) must be computed using the chain rule and the transformation Jacobian (J):

$$u = \frac{\partial\phi}{\partial x} = \frac{1}{J}\left(\frac{\partial\phi}{\partial\xi}\frac{\partial y}{\partial\eta} - \frac{\partial\phi}{\partial\eta}\frac{\partial y}{\partial\xi}\right)$$

$$v = \frac{\partial\phi}{\partial y} = \frac{1}{J}\left(-\frac{\partial\phi}{\partial\xi}\frac{\partial x}{\partial\eta} + \frac{\partial\phi}{\partial\eta}\frac{\partial x}{\partial\xi}\right)$$

In the code:

```
u = (phi_xi .* y_eta - phi_eta .* y_xi) ./ J;
v = (-phi_xi .* x_eta + phi_eta .* x_xi) ./ J;
V_local = sqrt(u.^2 + v.^2);
```

This part exactly follows the covariant/contravariant vector transformation relations.

b) Pressure Coefficient Calculation

For compressible flows, especially at $M \approx 0.7$, using the simple Bernoulli equation introduces significant error. Therefore, the **exact isentropic relation** is used:

$$C_p = \frac{2}{\gamma M_\infty^2}\left[\left(1 + \frac{\gamma-1}{2}M_\infty^2\left(1 - \frac{V_{\text{local}}^2}{U_\infty^2}\right)\right)^{\frac{\gamma}{\gamma-1}} - 1\right]$$

In the code:

```
term_bracket = 1 + 0.5*(gamma_gas - 1)*M_inf^2 * (1 - (V_local/U_inf).^2);
Cp = (2 / (gamma_gas * M_inf^2)) * (term_bracket.^(gamma_gas / (gamma_gas
```

Result: Using this relation (instead of linear or incompressible approximations) confirms that the code is fully valid for analyzing compressible flow up to near-sonic speeds (M_{crit}).

8.7 Conclusion

A meticulous line-by-line review of the code and its comparison with the formulas derived in the theory section indicates the absence of implementation error. All metric coefficients, Laplacian terms, and aerodynamic relations have been implemented in the software while fully adhering to the principles of engineering mathematics and fluid dynamics.

Students: Hamid Motallebi ( )
Sheida Yarahmadi ( )

A MATLAB Implementation Code for Parts A and B

```
clc,clear

% Inputs
eta = 0 : 1/30 : 1;
ksi = 0 : 1/60 : 1;

% Mesh generation
[x,y,m,n]=TFI(ksi,eta);
z = zeros(m,n);
surf(x,y,z)
title('TFI mesh','Interpreter','latex')
xlabel('$x$','Interpreter','latex')
ylabel('$y$','Interpreter','latex')
zlabel('$z$','Interpreter','latex')
set(gca,'FontName','Times','FontSize',12)

[x,y,dksi,deta] = Elliptic_mesh (x,y);
surf(x,y,z)
title('Elliptic mesh','Interpreter','latex')
xlabel('$x$','Interpreter','latex')
ylabel('$y$','Interpreter','latex')
zlabel('$z$','Interpreter','latex')
set(gca,'FontName','Times','FontSize',12)


% Compute metrics
    x_ksi_e = zeros(m,n-1);     % Derivation of x in term of ksi (
vertical face)
    x_eta_e = zeros(m,n-1);     % Derivation of x in term of eta (
vertical face)
    y_ksi_e = zeros(m,n-1);     % Derivation of y in term of ksi (
vertical face)
    y_eta_e = zeros(m,n-1);     % Derivation of y in term of eta (
vertical face)

    x_ksi_n = zeros(m-1,n);     % Derivation of x in term of ksi (
horizental face)
    x_eta_n = zeros(m-1,n);     % Derivation of x in term of eta (
horizental face)
    y_ksi_n = zeros(m-1,n);     % Derivation of y in term of ksi (
horizental face)
    y_eta_n = zeros(m-1,n);     % Derivation of y in term of eta (
horizental face)


% Mid points of horizental and vertical face
    x_mid_e = zeros(m,n-1);
    y_mid_e = zeros(m,n-1);
    x_mid_n = zeros(m-1,n);
    y_mid_n = zeros(m-1,n);


```

```
        % Update metrics and Mid points for vertical face
        for j = 1 : n-1
        for i = 1 : m-2
        x_ksi_e(i+1,j) = (x(i+2,j+1) + x(i+2,j) - x(i,j+1) - x(i,j)) / (4*dksi);
        x_eta_e(i+1,j) = (x(i+1,j+1) - x(i+1,j)) / (deta);
        y_ksi_e(i+1,j) = (y(i+2,j+1) + y(i+2,j) - y(i,j+1) - y(i,j)) / (4*dksi);
        y_eta_e(i+1,j) = (y(i+1,j+1) - y(i+1,j)) / (deta);
        x_mid_e(i+1,j) = (x(i+1,j)+x(i+1,j+1))/2;
        y_mid_e(i+1,j) = (y(i+1,j)+y(i+1,j+1))/2;
        end
        x_ksi_e(m,j) = (x(2,j+1) + x(2,j) - x(m-1,j+1) - x(m-1,j)) / (4*dksi);
        x_eta_e(m,j) = (x(m,j+1) - x(m,j)) / (deta);
        x_ksi_e(1,j) = (x(2,j+1) + x(2,j) - x(m-1,j+1) - x(m-1,j)) / (4*dksi);
        x_eta_e(1,j) = (x(1,j+1) - x(1,j)) / (deta);

        y_ksi_e(m,j) = (y(2,j+1) + y(2,j) - y(m-1,j+1) - y(m-1,j)) / (4*dksi);
        y_eta_e(m,j) = (y(m,j+1) - y(m,j)) / (deta);
        y_ksi_e(1,j) = (y(2,j+1) + y(2,j) - y(m-1,j+1) - y(m-1,j)) / (4*dksi);
        y_eta_e(1,j) = (y(1,j+1) - y(1,j)) / (deta);

        x_mid_e(m,j) = (x(m,j)+x(m,j+1))/2;
        y_mid_e(m,j) = (y(m,j)+y(m,j+1))/2;
        x_mid_e(1,j) = (x(1,j)+x(1,j+1))/2;
        y_mid_e(1,j) = (y(1,j)+y(1,j+1))/2;
        end


        % Update metrics and Mid points for horizental face
        for i = 1 : m-1
        for j = 1 : n-2
        x_ksi_n(i,j+1) = (x(i+1,j+1) - x(i,j+1)) / (dksi);
        x_eta_n(i,j+1) = (x(i+1,j+2) + x(i,j+2) - x(i+1,j) - x(i,j)) / (4*deta);
        y_eta_n(i,j+1) = (y(i+1,j+2) + y(i,j+2) - y(i+1,j) - y(i,j)) / (4*deta);
        y_ksi_n(i,j+1) = (y(i+1,j+1) - y(i,j+1)) / (dksi);
        x_mid_n(i,j+1) = (x(i,j+1)+x(i+1,j+1))/2;
        y_mid_n(i,j+1) = (y(i,j+1)+y(i+1,j+1))/2;
        end
        x_eta_n(i,n) = (3/2*(x(i,n)+x(i+1,n)) - 4/2*(x(i,n-1)+x(i+1,n-1)) + 1/2*(x(i,n-2)+x(i+1,n-2))) / (2*deta);
        x_ksi_n(i,n) = (x(i+1,n) - x(i,n)) / (dksi);
        x_eta_n(i,1) = -(3/2*(x(i,1)+x(i+1,1)) - 4/2*(x(i,2)+x(i+1,2)) + 1/2*(x(i,3)+x(i+1,3))) / (2*deta);
        x_ksi_n(i,1) = (x(i+1,1) - x(i,1)) / (dksi);

        y_eta_n(i,n) = (3/2*(y(i,n)+y(i+1,n)) - 4/2*(y(i,n-1)+y(i+1,n-1)) + 1/2*(y(i,n-2)+y(i+1,n-2))) / (2*deta);
        y_ksi_n(i,n) = (y(i+1,n) - y(i,n)) / (dksi);
```

Students: Hamid Motallebi ( )
Sheida Yarahmadi ( )

```
89        y_eta_n(i,1) = -(3/2*(y(i,1)+y(i+1,1)) - 4/2*(y(i,2)+y(i+1,2)) +
    1/2*(y(i,3)+y(i+1,3))) / (2*deta);
90        y_ksi_n(i,1) = (y(i+1,1) - y(i,1)) / (dksi);
91        x_mid_n(i,1) = (x(i,1)+x(i+1,1))/2;
92        y_mid_n(i,1) = (y(i,1)+y(i+1,1))/2;
93        x_mid_n(i,n) = (x(i,n)+x(i+1,n))/2;
94        y_mid_n(i,n) = (y(i,n)+y(i+1,n))/2;
95        end
96
97
98        % Compute matrix
99        J_vertical = x_ksi_e .* y_eta_e - x_eta_e .* y_ksi_e;
100       D = (x_eta_e.*x_eta_e + y_eta_e.*y_eta_e) ./ J_vertical;
101       J_horizental = x_ksi_n .* y_eta_n - x_eta_n .* y_ksi_n;
102       D_star = (x_ksi_n.*x_ksi_n + y_ksi_n.*y_ksi_n) ./ J_horizental;
103       S_vertical = (x_eta_e.*x_ksi_e + y_eta_e.*y_ksi_e) ./ J_vertical;
104       S_horizental = (x_eta_n.*x_ksi_n + y_eta_n.*y_ksi_n) ./ J_horizental
    ;
105
106
107       % SOR
108       phi_bar = zeros(m-1,n);      % Average of phi in FV
109       phi_bar_old = zeros(m-1,n);
110       w = 1.7;
111       tolerance = 10^(-8);
112       iter = 0;
113       L2 = 10;
114       K = 1;
115       iteration(K) = 0;
116       L2_d(K) = log10(L2);
117
118       % for Upper bond
119       for i = 1 : 1 : m-1
120       phi_bar(i,n) = 2 * x_mid_n(i,n) - phi_bar(i,n-1);
121       end
122
123
124       % Update phi_bar
125       while L2 > tolerance
126       for i = 1 : 1 : m-1
127       for j = 1 : 1 : n
128       phi_bar_old(i,j) = phi_bar(i,j);
129       end
130       end
131       for j = 1 : n-1
132       for i = 1 : m-1
133       if j == 1
134       % Lower bond of loop in j == 1
135       if i == 1
136       phi_bar(i,j) = -w / (-D(i+1,j)-D_star(i,j+1)-D(i,j)+3/4*S_vertical(i
    +1,j)-3/4*S_vertical(i,j)) *( ...
137       phi_bar(i+1,j+1) * (-S_horizental(i,j+1)/4 - S_vertical(i+1,j))  +
    ...
138       phi_bar(i+1,j)*(D(i+1,j) + ((-S_horizental(i,j+1)+3*S_vertical(i+1,j
    ))/4))+...
```

```
        phi_bar(i+1,j+2)*(S_vertical(i+1,j)/4)+...
        phi_bar(i,j+1)*(D_star(i,j+1)-S_vertical(i+1,j)+S_vertical(i,j))+...
        phi_bar(i,j+2)*((S_vertical(i+1,j)-S_vertical(i,j))/4)+...
        phi_bar(m-1,j+1)*(S_vertical(i,j)+S_horizental(i,j+1)/4)+...
        phi_bar(m-1,j)*(D(i,j) + ((S_horizental(i,j+1)-3*S_vertical(i,j))/4)
    )+...
        phi_bar(m-1,j+2)*(-S_vertical(i,j)/4))+(1-w)*phi_bar(i,j);
        % Upper bond of loop in j == 1
        elseif i == m-1
        phi_bar(i,j) = -w / (-D(i+1,j)-D_star(i,j+1)-D(i,j)+3/4*S_vertical(i
    +1,j)-3/4*S_vertical(i,j)) *( ...
        phi_bar(1,j+1) * (-S_horizental(i,j+1)/4 - S_vertical(i+1,j))  + ...
        phi_bar(1,j)*(D(i+1,j) + ((-S_horizental(i,j+1)+3*S_vertical(i+1,j))
    /4))+...
        phi_bar(1,j+2)*(S_vertical(i+1,j)/4)+...
        phi_bar(i,j+1)*(D_star(i,j+1)-S_vertical(i+1,j)+S_vertical(i,j))+...
        phi_bar(i,j+2)*((S_vertical(i+1,j)-S_vertical(i,j))/4)+...
        phi_bar(i-1,j+1)*(S_vertical(i,j)+S_horizental(i,j+1)/4)+...
        phi_bar(i-1,j)*(D(i,j) + ((S_horizental(i,j+1)-3*S_vertical(i,j))/4)
    )+...
        phi_bar(i-1,j+2)*(-S_vertical(i,j)/4))+(1-w)*phi_bar(i,j);
        else
        phi_bar(i,j) = -w / (-D(i+1,j)-D_star(i,j+1)-D(i,j)+3/4*S_vertical(i
    +1,j)-3/4*S_vertical(i,j)) *( ...
        phi_bar(i+1,j+1) * (-S_horizental(i,j+1)/4 - S_vertical(i+1,j))  +
    ...
        phi_bar(i+1,j)*(D(i+1,j) + ((-S_horizental(i,j+1)+3*S_vertical(i+1,j
    ))/4))+...
        phi_bar(i+1,j+2)*(S_vertical(i+1,j)/4)+...
        phi_bar(i,j+1)*(D_star(i,j+1)-S_vertical(i+1,j)+S_vertical(i,j))+...
        phi_bar(i,j+2)*((S_vertical(i+1,j)-S_vertical(i,j))/4)+...
        phi_bar(i-1,j+1)*(S_vertical(i,j)+S_horizental(i,j+1)/4)+...
        phi_bar(i-1,j)*(D(i,j) + ((S_horizental(i,j+1)-3*S_vertical(i,j))/4)
    )+...
        phi_bar(i-1,j+2)*(-S_vertical(i,j)/4))+(1-w)*phi_bar(i,j);
        end
        else
        if i==1
        phi_bar(i,j) = -w / (-D(i+1,j)-D_star(i,j+1)-D(i,j)-D_star(i,j)) *(
    ...
        phi_bar(i+1,j+1) * (-S_vertical(i+1,j) - S_horizental(i,j+1)) / 4  +
     ...
        phi_bar(i+1,j)*(D(i+1,j) + ((-S_horizental(i,j+1)+S_horizental(i,j))
    /4))+...
        phi_bar(i+1,j-1)*((S_vertical(i+1,j)+S_horizental(i,j))/4)+...
        phi_bar(i,j+1)*(D_star(i,j+1)+(-S_vertical(i+1,j)+S_vertical(i,j))
    /4)+...
        phi_bar(i,j-1)*(D_star(i,j)+(S_vertical(i+1,j)-S_vertical(i,j))/4)
    +...
        phi_bar(m-1,j+1)*((S_vertical(i,j)+S_horizental(i,j+1))/4)+...
        phi_bar(m-1,j)*(D(i,j) + ((S_horizental(i,j+1)-S_horizental(i,j))/4)
    )+...
        phi_bar(m-1,j-1)*((-S_vertical(i,j)-S_horizental(i,j))/4))+(1-w)*
    phi_bar(i,j);
        elseif i==m-1
```

```
        phi_bar(i,j) = -w / (-D(i+1,j)-D_star(i,j+1)-D(i,j)-D_star(i,j)) *(
...
        phi_bar(1,j+1) * (-S_vertical(i+1,j) - S_horizental(i,j+1)) / 4  +
...
        phi_bar(1,j)*(D(i+1,j) + ((-S_horizental(i,j+1)+S_horizental(i,j))
/4))+...
        phi_bar(1,j-1)*((S_vertical(i+1,j)+S_horizental(i,j))/4)+...
        phi_bar(i,j+1)*(D_star(i,j+1)+(-S_vertical(i+1,j)+S_vertical(i,j))
/4)+...
        phi_bar(i,j-1)*(D_star(i,j)+(S_vertical(i+1,j)-S_vertical(i,j))/4)
+...
        phi_bar(i-1,j+1)*((S_vertical(i,j)+S_horizental(i,j+1))/4)+...
        phi_bar(i-1,j)*(D(i,j) + ((S_horizental(i,j+1)-S_horizental(i,j))/4)
)+...
        phi_bar(i-1,j-1)*((-S_vertical(i,j)-S_horizental(i,j))/4))+(1-w)*
phi_bar(i,j);
    else
        phi_bar(i,j) = -w / (-D(i+1,j)-D_star(i,j+1)-D(i,j)-D_star(i,j)) *(
...
        phi_bar(i+1,j+1) * (-S_vertical(i+1,j) - S_horizental(i,j+1)) / 4  +
 ...
        phi_bar(i+1,j)*(D(i+1,j) + ((-S_horizental(i,j+1)+S_horizental(i,j))
/4))+...
        phi_bar(i+1,j-1)*((S_vertical(i+1,j)+S_horizental(i,j))/4)+...
        phi_bar(i,j+1)*(D_star(i,j+1)+(-S_vertical(i+1,j)+S_vertical(i,j))
/4)+...
        phi_bar(i,j-1)*(D_star(i,j)+(S_vertical(i+1,j)-S_vertical(i,j))/4)
+...
        phi_bar(i-1,j+1)*((S_vertical(i,j)+S_horizental(i,j+1))/4)+...
        phi_bar(i-1,j)*(D(i,j) + ((S_horizental(i,j+1)-S_horizental(i,j))/4)
)+...
        phi_bar(i-1,j-1)*((-S_vertical(i,j)-S_horizental(i,j))/4))+(1-w)*
phi_bar(i,j);
    end
    end
    end
    end

    for i = 1 : 1 : m-1
    phi_bar(i,n) = 2 * x_mid_n(i,n) - phi_bar(i,n-1);
    end
    L2 = L2_norm(phi_bar,phi_bar_old);  % L2_norm function
    iter = iter + 1;
    K = K + 1;
    iteration(K) = iter;
    L2_d(K) = log10(L2);
    end

    figure
    plot(iteration , L2_d , 'LineWidth' , 2.5)
    grid on;
    xlabel('Iteration');
    ylabel('$$\log ({\left\| {{\delta _{i,j}}} \right\|_2})$$','
Interpreter','latex');
    title('Convergency plot');
```

```
219         set(gca,'FontName','Times','FontSize',12)
220
221         phi = zeros(m-1,n-1);
222         for i = 1 : 1 : m-1
223         for j = 1 : 1 : n-1
224         phi(i,j) = phi_bar(i,j);
225         end
226         end
227
228         fprintf('SOR with w = 1.7 converged in %d iterations for grid %d * %
    d (ksi * eta).\n', ...
229         iter, m, n);
230
231         % compute phi_grid from phi_bar
232         phi_grid = zeros(m,n);
233         for i = 1 : m
234         for j = 2 : n-1
235         if i == 1
236         phi_grid(i,j) = 1/4 * (phi(m-1,j-1)+phi(i,j-1)+phi(i,j)+phi(m-1,j));
237         elseif i == m
238         phi_grid(i,j) = 1/4 * (phi(i-1,j-1)+phi(1,j-1)+phi(1,j)+phi(i-1,j));
239         else
240         phi_grid(i,j) = 1/4 * (phi(i-1,j-1)+phi(i,j-1)+phi(i,j)+phi(i-1,j));
241         end
242         end
243         if i == 1
244         phi_grid(i,1) = 1/2 * ((3/2*phi(m-1,1)-1/2*phi(m-1,2)) + (3/2*phi(i
    ,1)-1/2*phi(i,2)));
245         phi_grid(i,n) = x(i,n);
246         elseif i == m
247         phi_grid(i,1) = 1/2 * ((3/2*phi(i-1,1)-1/2*phi(i-1,2)) + (3/2*phi
    (1,1)-1/2*phi(1,2)));
248         phi_grid(i,n) = x(i,n);
249         else
250         phi_grid(i,1) = 1/2 * ((3/2*phi(i-1,1)-1/2*phi(i-1,2)) + (3/2*phi(i
    ,1)-1/2*phi(i,2)));
251         phi_grid(i,n) = x(i,n);
252         end
253         end
254         surf(x,y,phi_grid)
255         title('$\phi$ on grid points','Interpreter','latex')
256         xlabel('$x$','Interpreter','latex')
257         ylabel('$y$','Interpreter','latex')
258         zlabel('$\phi$','Interpreter','latex')
259         set(gca,'FontName','Times','FontSize',12)
260
261         fprintf('phi at leading edge is %d for grid %d * %d (ksi * eta).\n',
     ...
262         phi_grid(ceil(m/2),1), m, n);
263
264
265
266         % Compute pressure coefficient
267         phi_ksi = zeros(m-1,1);     % Derivation of phi in term of ksi
268         phi_eta = zeros(n-1,1);     % Derivation of phi in term of ksi
```

```
269
270         % Update phi_ksi and phi_eta
271         for i = 1 : m-1
272         if i == 1
273         phi_ksi(i) = 3/4*phi(i+1,1) - 3/4*phi(m-1,1) - 1/4*phi(i+1,2) + 1/4*
    phi(m-1,2);
274         elseif i == m-1
275         phi_ksi(i) = 3/4*phi(1,1) - 3/4*phi(i-1,1) - 1/4*phi(1,2) + 1/4*phi(
    i-1,2);
276         else
277         phi_ksi(i) = 3/4*phi(i+1,1) - 3/4*phi(i-1,1) - 1/4*phi(i+1,2) + 1/4*
    phi(i-1,2);
278         phi_eta(i) = -2*phi(i,1) + 3*phi(i,2) - phi(i,3);
279         end
280         end
281
282         phi_eta(1) = -2*phi(1,1) + 3*phi(1,2) - phi(1,3);
283         phi_eta(m-1) = -2*phi(m-1,1) + 3*phi(m-1,2) - phi(m-1,3);
284
285         % Compute velocities and pressure coefficient
286         u = 1 ./ J_horizental( : , 1) .* (y_eta_n( : ,1).*phi_ksi( : ,1) -
    y_ksi_n( : ,1).*phi_eta( : ,1));
287         v = 1 ./ J_horizental( : , 1) .* (-x_eta_n( : ,1).*phi_ksi( : ,1) +
    x_ksi_n( : ,1).*phi_eta( : ,1));
288         c_p = 1 - (u.^2 + v.^2);
289
290         plot(x_mid_n( : ,1) , c_p,'linewidth' , 2.5)
291         grid on
292         title('Pressure coefficient on surface of the airfoil')
293         xlabel('$x$','Interpreter','latex')
294         ylabel('Pressure coefficient')
295
296
297         % Drag and lift coefficient
298         L = 0;      % Length of airfoil
299         c_d = 0;    % Drag coefficient
300         c_l = 0;    % Lift coefficient
301
302         % Update Drag and lift coefficient
303         for i = 1 : m-1
304         dx = x(i+1,1) - x(i,1);
305         dy = y(i+1,1) - y(i,1);
306         dL = sqrt(dx^2 + dy^2);
307         width = 1;      % Width
308         L = L + dL;
309         theta_prime = atand(dy/dx);
310         if dx < 0
311         theta = abs(theta_prime) + 270;
312         else
313         theta = abs(theta_prime) + 90;
314         end
315         c_d = c_d + c_p(i) * cosd(theta) * dL * width;
316         c_l = c_l + c_p(i) * sind(theta) * dL * width;
317         end
318
```

```
319         C_L = c_l / L;
320         C_D = c_d / L;
321
322         fprintf('Drag coefficient is %d for grid %d * %d (ksi * eta).\n',
    C_D, m, n);
323         fprintf('Lift coefficient is %d for grid %d * %d (ksi * eta).\n',
    C_L, m, n);
324
325
326
327         function [x,y,m,n] = TFI(ksi,eta)    % TFI
328
329         P14 = [1,0];
330         P23 = [41,0];
331         P34 = [41,0];
332         P12 = [1,0];
333
334         m = size(ksi,2);
335         n = size(eta,2);
336
337         for i = 1 : m
338         for j = 1 : n
339
340         [x1,y1] = c1(ksi(i));
341         [x3,y3] = c3(ksi(i));
342
343         [x2,y2] = c2(eta(j));
344         [x4,y4] = c4(eta(j));
345
346         x(i,j) = (1-eta(j)) * x1+ eta(j) * x3+ksi(i) * x2+(1-ksi(i)) * x4
    -...
347         ((1-ksi(i))*(1-eta(j))*P14(1)+ksi(i)*eta(j)*P23(1)+(1-ksi(i))*eta(j)
    *P34(1)+ksi(i)*(1-eta(j))*P12(1));
348
349         y(i,j) = (1-eta(j)) * y1+ eta(j) * y3+ksi(i) * y2+(1-ksi(i)) * y4
    -...
350         ((1-ksi(i))*(1-eta(j))*P14(2)+ksi(i)*eta(j)*P23(2)+(1-ksi(i))*eta(j)
    *P34(2)+ksi(i)*(1-eta(j))*P12(2));
351
352         end
353         end
354
355         end
356
357
358         function [x,y] = c1(ksi)
359         if (ksi <= 0.5)
360         y = -0.6*(0.29690*sqrt(1/2 * (cos(2*pi*(1-ksi))+1)) ...
361         - 0.12600*(1/2 * (cos(2*pi*(1-ksi))+1)) ...
362         - 0.35160*(1/2 * (cos(2*pi*(1-ksi))+1)).^2 ...
363         + 0.28430*(1/2 * (cos(2*pi*(1-ksi))+1)).^3 ...
364         - (0.10150 + 0.0021)*(1/2 * (cos(2*pi*(1-ksi))+1)).^4);
365         else
366         y = 0.6*(0.29690*sqrt(1/2 * (cos(2*pi*(1-ksi))+1)) ...
367         - 0.12600*(1/2 * (cos(2*pi*(1-ksi))+1)) ...
```

```
        - 0.35160*(1/2 * (cos(2*pi*(1-ksi))+1)).^2 ...
        + 0.28430*(1/2 * (cos(2*pi*(1-ksi))+1)).^3 ...
        - (0.10150 + 0.0021)*(1/2 * (cos(2*pi*(1-ksi))+1)).^4);
        end
        x = 1/2 * (cos(2*pi*(1-ksi))+1);
        end


        function [x,y] = c2(eta)
        x = 40*eta + 1;
        y = 0;
        end


        function [x,y] = c3(ksi)
        if (ksi <= 0.5)
        y = -sqrt(41^2-(41*cos(2*pi*ksi))^2);
        else
        y = sqrt(41^2-(41*cos(2*pi*ksi))^2);
        end
        x = 41*cos(2*pi*ksi);
        end


        function [x,y] = c4(eta)
        x = 40*eta + 1;
        y = 0;
        end




        % Elliptic mesh generation
        function [x,y,dksi,deta]=Elliptic_mesh (x0,y0)

        m=size(x0,1);
        n=size(x0,2);

        dksi=1;
        deta=1;
        x=x0;
        y=y0;
        Er=1;
        tol=1.e-8;
        iter=0;
        while Er>tol
        Er=0;
        for j=2:n-1
        for i=1:m

        if i == 1
        a = ((x0(i,j+1)-x0(i,j-1))/(2*deta))^2 + ((y0(i,j+1)-y0(i,j-1))/(2*
    deta))^2;
        b = (x0(i+1,j)-x0(m-1,j))*(x0(i,j+1)-x0(i,j-1))/(4*dksi*deta)+...
        (y0(i+1,j)-y0(m-1,j))*(y0(i,j+1)-y0(i,j-1))/(4*dksi*deta);
```

```
422
423         c= ((x0(i+1,j)-x0(m-1,j))/(2*dksi))^2+((y0(i+1,j)-y0(m-1,j))/(2*dksi
    ))^2;
424
425         coeff_LHS= 2. * (a/(dksi*dksi) + c / (deta* deta));
426
427         x(i,j) = 1/coeff_LHS*((a/(dksi*dksi))*(x(i+1,j)+x(m-1,j)) + (c/(deta
    *deta))*(x(i,j+1)+x(i,j-1))-...
428         (0.5*b/(dksi*deta))*(x(i+1,j+1)-x(i+1,j-1)+x(m-1,j-1)-x(m-1,j+1)));
429
430         y(i,j) = 1/coeff_LHS*((a/(dksi*dksi))*(y(i+1,j)+y(m-1,j)) + (c/(deta
    *deta))*(y(i,j+1)+y(i,j-1))-...
431         (0.5*b/(dksi*deta))*(y(i+1,j+1)-y(i+1,j-1)+y(m-1,j-1)-y(m-1,j+1)));
432
433         Er=Er + (x(i,j)-x0(i,j))^2 + (y(i,j)-y0(i,j))^2;
434
435         elseif i == m
436         a = ((x0(i,j+1)-x0(i,j-1))/(2*deta))^2 + ((y0(i,j+1)-y0(i,j-1))/(2*
    deta))^2;
437         b = (x0(2,j)-x0(m-1,j))*(x0(i,j+1)-x0(i,j-1))/(4*dksi*deta)+...
438         (y0(2,j)-y0(m-1,j))*(y0(i,j+1)-y0(i,j-1))/(4*dksi*deta);
439
440         c= ((x0(2,j)-x0(m-1,j))/(2*dksi))^2+((y0(2,j)-y0(m-1,j))/(2*dksi))
    ^2;
441
442         coeff_LHS= 2. * (a/(dksi*dksi) + c / (deta* deta));
443
444         x(i,j) = 1/coeff_LHS*((a/(dksi*dksi))*(x(2,j)+x(m-1,j)) + (c/(deta*
    deta))*(x(i,j+1)+x(i,j-1))-...
445         (0.5*b/(dksi*deta))*(x(2,j+1)-x(2,j-1)+x(m-1,j-1)-x(m-1,j+1)));
446
447         y(i,j) = 1/coeff_LHS*((a/(dksi*dksi))*(y(2,j)+y(m-1,j)) + (c/(deta*
    deta))*(y(i,j+1)+y(i,j-1))-...
448         (0.5*b/(dksi*deta))*(y(2,j+1)-y(2,j-1)+y(m-1,j-1)-y(m-1,j+1)));
449
450         Er=Er + (x(i,j)-x0(i,j))^2 + (y(i,j)-y0(i,j))^2;
451
452         else
453         a = ((x0(i,j+1)-x0(i,j-1))/(2*deta))^2 + ((y0(i,j+1)-y0(i,j-1))/(2*
    deta))^2;
454         b = (x0(i+1,j)-x0(i-1,j))*(x0(i,j+1)-x0(i,j-1))/(4*dksi*deta)+...
455         (y0(i+1,j)-y0(i-1,j))*(y0(i,j+1)-y0(i,j-1))/(4*dksi*deta);
456
457         c= ((x0(i+1,j)-x0(i-1,j))/(2*dksi))^2+((y0(i+1,j)-y0(i-1,j))/(2*dksi
    ))^2;
458
459         coeff_LHS= 2. * (a/(dksi*dksi) + c / (deta* deta));
460
461         x(i,j) = 1/coeff_LHS*((a/(dksi*dksi))*(x(i+1,j)+x(i-1,j)) + (c/(deta
    *deta))*(x(i,j+1)+x(i,j-1))-...
462         (0.5*b/(dksi*deta))*(x(i+1,j+1)-x(i+1,j-1)+x(i-1,j-1)-x(i-1,j+1)));
463
464         y(i,j) = 1/coeff_LHS*((a/(dksi*dksi))*(y(i+1,j)+y(i-1,j)) + (c/(deta
    *deta))*(y(i,j+1)+y(i,j-1))-...
465         (0.5*b/(dksi*deta))*(y(i+1,j+1)-y(i+1,j-1)+y(i-1,j-1)-y(i-1,j+1)));
```

```
        Er=Er + (x(i,j)-x0(i,j))^2 + (y(i,j)-y0(i,j))^2;
        end
        end
        end
        Er = sqrt(Er/(2*(m-1)*(n-2)));
        x0=x;
        y0=y;
        iter=iter+1;
        end
        fprintf('');
        fprintf('\nElliptic mesh converged in %d iterations for grid %d * %d
    (ksi * eta).\n\n', ...
        iter, m, n);
        fprintf('');
        end


        % L2_norm function
        function L2 = L2_norm(A , B)
        m = size(A,1);
        n = size(A,2);
        d_p = 0;
        N = m * (n-1);
        for i = 1 : 1 : m
        for j = 1 : 1 : n-1
        d_p = d_p + (A(i,j)-B(i,j))^2;
        end
        end
        L2 = sqrt(d_p/N);
        end

```

Listing 1: Complete MATLAB code for potential flow solution over NACA 0012 airfoil

B MATLAB Implementation Code for Part C

```
clc,clear

% 1. INPUTS & COMPRESSIBILITY SETUP

Mach = 0.8;                 % Mach number
Comp_Factor = 1 - Mach^2;   % (1 - M^2) term for the PDE

eta = 0 : 1/30 : 1;
ksi = 0 : 1/60 : 1;

% Mesh generation
[x,y,m,n]=TFI(ksi,eta);
z = zeros(m,n);

% surf(x,y,z); title('TFI mesh');

[x,y,dksi,deta] = Elliptic_mesh (x,y);

% Plot final mesh
figure(1)
surf(x,y,z)
title(['Elliptic mesh (Mach = ', num2str(Mach), ')'],'Interpreter','latex')
xlabel('$x$','Interpreter','latex')
ylabel('$y$','Interpreter','latex')
zlabel('$z$','Interpreter','latex')
set(gca,'FontName','Times','FontSize',12)


% ========================================
% 2. COMPUTE METRICS
% ========================================
x_ksi_e = zeros(m,n-1);
x_eta_e = zeros(m,n-1);
y_ksi_e = zeros(m,n-1);
y_eta_e = zeros(m,n-1);

x_ksi_n = zeros(m-1,n);
x_eta_n = zeros(m-1,n);
y_ksi_n = zeros(m-1,n);
y_eta_n = zeros(m-1,n);

x_mid_e = zeros(m,n-1);
y_mid_e = zeros(m,n-1);
x_mid_n = zeros(m-1,n);
y_mid_n = zeros(m-1,n);

% Update metrics and Mid points for vertical face
for j = 1 : n-1
for i = 1 : m-2
x_ksi_e(i+1,j) = (x(i+2,j+1) + x(i+2,j) - x(i,j+1) - x(i,j)) / (4*dksi);
```

```
        x_eta_e(i+1,j) = (x(i+1,j+1) - x(i+1,j)) / (deta);
        y_ksi_e(i+1,j) = (y(i+2,j+1) + y(i+2,j) - y(i,j+1) - y(i,j)) / (4*
    dksi);
        y_eta_e(i+1,j) = (y(i+1,j+1) - y(i+1,j)) / (deta);
        x_mid_e(i+1,j) = (x(i+1,j)+x(i+1,j+1))/2;
        y_mid_e(i+1,j) = (y(i+1,j)+y(i+1,j+1))/2;
        end
        x_ksi_e(m,j) = (x(2,j+1) + x(2,j) - x(m-1,j+1) - x(m-1,j)) / (4*dksi
    );
        x_eta_e(m,j) = (x(m,j+1) - x(m,j)) / (deta);
        x_ksi_e(1,j) = (x(2,j+1) + x(2,j) - x(m-1,j+1) - x(m-1,j)) / (4*dksi
    );
        x_eta_e(1,j) = (x(1,j+1) - x(1,j)) / (deta);

        y_ksi_e(m,j) = (y(2,j+1) + y(2,j) - y(m-1,j+1) - y(m-1,j)) / (4*dksi
    );
        y_eta_e(m,j) = (y(m,j+1) - y(m,j)) / (deta);
        y_ksi_e(1,j) = (y(2,j+1) + y(2,j) - y(m-1,j+1) - y(m-1,j)) / (4*dksi
    );
        y_eta_e(1,j) = (y(1,j+1) - y(1,j)) / (deta);

        x_mid_e(m,j) = (x(m,j)+x(m,j+1))/2;
        y_mid_e(m,j) = (y(m,j)+y(m,j+1))/2;
        x_mid_e(1,j) = (x(1,j)+x(1,j+1))/2;
        y_mid_e(1,j) = (y(1,j)+y(1,j+1))/2;
        end


        % Update metrics and Mid points for horizental face
        for i = 1 : m-1
        for j = 1 : n-2
        x_ksi_n(i,j+1) = (x(i+1,j+1) - x(i,j+1)) / (dksi);
        x_eta_n(i,j+1) = (x(i+1,j+2) + x(i,j+2) - x(i+1,j) - x(i,j)) / (4*
    deta);
        y_eta_n(i,j+1) = (y(i+1,j+2) + y(i,j+2) - y(i+1,j) - y(i,j)) / (4*
    deta);
        y_ksi_n(i,j+1) = (y(i+1,j+1) - y(i,j+1)) / (dksi);
        x_mid_n(i,j+1) = (x(i,j+1)+x(i+1,j+1))/2;
        y_mid_n(i,j+1) = (y(i,j+1)+y(i+1,j+1))/2;
        end
        % Boundaries
        x_eta_n(i,n) = (3/2*(x(i,n)+x(i+1,n)) - 4/2*(x(i,n-1)+x(i+1,n-1)) +
    1/2*(x(i,n-2)+x(i+1,n-2))) / (2*deta);
        x_ksi_n(i,n) = (x(i+1,n) - x(i,n)) / (dksi);
        x_eta_n(i,1) = -(3/2*(x(i,1)+x(i+1,1)) - 4/2*(x(i,2)+x(i+1,2)) +
    1/2*(x(i,3)+x(i+1,3))) / (2*deta);
        x_ksi_n(i,1) = (x(i+1,1) - x(i,1)) / (dksi);

        y_eta_n(i,n) = (3/2*(y(i,n)+y(i+1,n)) - 4/2*(y(i,n-1)+y(i+1,n-1)) +
    1/2*(y(i,n-2)+y(i+1,n-2))) / (2*deta);
        y_ksi_n(i,n) = (y(i+1,n) - y(i,n)) / (dksi);
        y_eta_n(i,1) = -(3/2*(y(i,1)+y(i+1,1)) - 4/2*(y(i,2)+y(i+1,2)) +
    1/2*(y(i,3)+y(i+1,3))) / (2*deta);
        y_ksi_n(i,1) = (y(i+1,1) - y(i,1)) / (dksi);
        x_mid_n(i,1) = (x(i,1)+x(i+1,1))/2;
```

```
        y_mid_n(i,1) = (y(i,1)+y(i+1,1))/2;
        x_mid_n(i,n) = (x(i,n)+x(i+1,n))/2;
        y_mid_n(i,n) = (y(i,n)+y(i+1,n))/2;
        end


        % ============================================================
        % 3. COMPUTE MATRIX
        % ============================================================
        J_vertical = x_ksi_e .* y_eta_e - x_eta_e .* y_ksi_e;

        % Alpha term: x_eta^2 + (1-M^2)*y_eta^2
        D = (x_eta_e.*x_eta_e + Comp_Factor * y_eta_e.*y_eta_e) ./
    J_vertical;

        J_horizental = x_ksi_n .* y_eta_n - x_eta_n .* y_ksi_n;

        % Gamma term: x_ksi^2 + (1-M^2)*y_ksi^2
        D_star = (x_ksi_n.*x_ksi_n + Comp_Factor * y_ksi_n.*y_ksi_n) ./
    J_horizental;

        % Beta term: x_eta*x_ksi + (1-M^2)*y_eta*y_ksi
        S_vertical = (x_eta_e.*x_ksi_e + Comp_Factor * y_eta_e.*y_ksi_e) ./
    J_vertical;
        S_horizental = (x_eta_n.*x_ksi_n + Comp_Factor * y_eta_n.*y_ksi_n)
    ./ J_horizental;

        % ========================================
        % 4. SOLVER (SOR)
        % ========================================
        phi_bar = zeros(m-1,n);
        phi_bar_old = zeros(m-1,n);
        w = 1.7;
        tolerance = 10^(-8);
        L2 = 10;
        K_iter = 1; % Renamed 'K' to 'K_iter' to avoid confusion with Math K
        iteration(K_iter) = 0;
        L2_d(K_iter) = log10(L2);

        % Initial guess for Upper bond
        for i = 1 : 1 : m-1
        phi_bar(i,n) = 2 * x_mid_n(i,n) - phi_bar(i,n-1);
        end

        % Update phi_bar
        while L2 > tolerance
        for i = 1 : 1 : m-1
        for j = 1 : 1 : n
        phi_bar_old(i,j) = phi_bar(i,j);
        end
        end

        % Loop over internal points (SOR Algorithm)
        for j = 1 : n-1
        for i = 1 : m-1
```

```
        % Using pre-calculated coefficients D, D_star, S (which now include M effect)
        if j == 1
        if i == 1
        phi_bar(i,j) = -w / (-D(i+1,j)-D_star(i,j+1)-D(i,j)+3/4*S_vertical(i+1,j)-3/4*S_vertical(i,j)) *( ...
        phi_bar(i+1,j+1) * (-S_horizental(i,j+1)/4 - S_vertical(i+1,j))  + ...
        phi_bar(i+1,j)*(D(i+1,j) + ((-S_horizental(i,j+1)+3*S_vertical(i+1,j))/4))+...
        phi_bar(i+1,j+2)*(S_vertical(i+1,j)/4)+...
        phi_bar(i,j+1)*(D_star(i,j+1)-S_vertical(i+1,j)+S_vertical(i,j))+...
        phi_bar(i,j+2)*((S_vertical(i+1,j)-S_vertical(i,j))/4)+...
        phi_bar(m-1,j+1)*(S_vertical(i,j)+S_horizental(i,j+1)/4)+...
        phi_bar(m-1,j)*(D(i,j) + ((S_horizental(i,j+1)-3*S_vertical(i,j))/4))+...
        phi_bar(m-1,j+2)*(-S_vertical(i,j)/4))+(1-w)*phi_bar(i,j);
        elseif i == m-1
        phi_bar(i,j) = -w / (-D(i+1,j)-D_star(i,j+1)-D(i,j)+3/4*S_vertical(i+1,j)-3/4*S_vertical(i,j)) *( ...
        phi_bar(1,j+1) * (-S_horizental(i,j+1)/4 - S_vertical(i+1,j))  + ...
        phi_bar(1,j)*(D(i+1,j) + ((-S_horizental(i,j+1)+3*S_vertical(i+1,j))/4))+...
        phi_bar(1,j+2)*(S_vertical(i+1,j)/4)+...
        phi_bar(i,j+1)*(D_star(i,j+1)-S_vertical(i+1,j)+S_vertical(i,j))+...
        phi_bar(i,j+2)*((S_vertical(i+1,j)-S_vertical(i,j))/4)+...
        phi_bar(i-1,j+1)*(S_vertical(i,j)+S_horizental(i,j+1)/4)+...
        phi_bar(i-1,j)*(D(i,j) + ((S_horizental(i,j+1)-3*S_vertical(i,j))/4))+...
        phi_bar(i-1,j+2)*(-S_vertical(i,j)/4))+(1-w)*phi_bar(i,j);
        else
        phi_bar(i,j) = -w / (-D(i+1,j)-D_star(i,j+1)-D(i,j)+3/4*S_vertical(i+1,j)-3/4*S_vertical(i,j)) *( ...
        phi_bar(i+1,j+1) * (-S_horizental(i,j+1)/4 - S_vertical(i+1,j))  + ...
        phi_bar(i+1,j)*(D(i+1,j) + ((-S_horizental(i,j+1)+3*S_vertical(i+1,j))/4))+...
        phi_bar(i+1,j+2)*(S_vertical(i+1,j)/4)+...
        phi_bar(i,j+1)*(D_star(i,j+1)-S_vertical(i+1,j)+S_vertical(i,j))+...
        phi_bar(i,j+2)*((S_vertical(i+1,j)-S_vertical(i,j))/4)+...
        phi_bar(i-1,j+1)*(S_vertical(i,j)+S_horizental(i,j+1)/4)+...
        phi_bar(i-1,j)*(D(i,j) + ((S_horizental(i,j+1)-3*S_vertical(i,j))/4))+...
        phi_bar(i-1,j+2)*(-S_vertical(i,j)/4))+(1-w)*phi_bar(i,j);
        end
        else
        % Normal internal points
        if i==1
        phi_bar(i,j) = -w / (-D(i+1,j)-D_star(i,j+1)-D(i,j)-D_star(i,j)) *( ...
        phi_bar(i+1,j+1) * (-S_vertical(i+1,j) - S_horizental(i,j+1)) / 4  + ...
        phi_bar(i+1,j)*(D(i+1,j) + ((-S_horizental(i,j+1)+S_horizental(i,j))/4))+...
        phi_bar(i+1,j-1)*((S_vertical(i+1,j)+S_horizental(i,j))/4)+...
```

```
186         phi_bar(i,j+1)*(D_star(i,j+1)+(-S_vertical(i+1,j)+S_vertical(i,j))
    /4)+...
187         phi_bar(i,j-1)*(D_star(i,j)+(S_vertical(i+1,j)-S_vertical(i,j))/4)
    +...
188         phi_bar(m-1,j+1)*((S_vertical(i,j)+S_horizental(i,j+1))/4)+...
189         phi_bar(m-1,j)*(D(i,j) + ((S_horizental(i,j+1)-S_horizental(i,j))/4)
    )+...
190         phi_bar(m-1,j-1)*((-S_vertical(i,j)-S_horizental(i,j))/4))+(1-w)*
    phi_bar(i,j);
191         elseif i==m-1
192         phi_bar(i,j) = -w / (-D(i+1,j)-D_star(i,j+1)-D(i,j)-D_star(i,j)) *(
    ...
193         phi_bar(1,j+1) * (-S_vertical(i+1,j) - S_horizental(i,j+1)) / 4  +
    ...
194         phi_bar(1,j)*(D(i+1,j) + ((-S_horizental(i,j+1)+S_horizental(i,j))
    /4))+...
195         phi_bar(1,j-1)*((S_vertical(i+1,j)+S_horizental(i,j))/4)+...
196         phi_bar(i,j+1)*(D_star(i,j+1)+(-S_vertical(i+1,j)+S_vertical(i,j))
    /4)+...
197         phi_bar(i,j-1)*(D_star(i,j)+(S_vertical(i+1,j)-S_vertical(i,j))/4)
    +...
198         phi_bar(i-1,j+1)*((S_vertical(i,j)+S_horizental(i,j+1))/4)+...
199         phi_bar(i-1,j)*(D(i,j) + ((S_horizental(i,j+1)-S_horizental(i,j))/4)
    )+...
200         phi_bar(i-1,j-1)*((-S_vertical(i,j)-S_horizental(i,j))/4))+(1-w)*
    phi_bar(i,j);
201         else
202         phi_bar(i,j) = -w / (-D(i+1,j)-D_star(i,j+1)-D(i,j)-D_star(i,j)) *(
    ...
203         phi_bar(i+1,j+1) * (-S_vertical(i+1,j) - S_horizental(i,j+1)) / 4  +
     ...
204         phi_bar(i+1,j)*(D(i+1,j) + ((-S_horizental(i,j+1)+S_horizental(i,j))
    /4))+...
205         phi_bar(i+1,j-1)*((S_vertical(i+1,j)+S_horizental(i,j))/4)+...
206         phi_bar(i,j+1)*(D_star(i,j+1)+(-S_vertical(i+1,j)+S_vertical(i,j))
    /4)+...
207         phi_bar(i,j-1)*(D_star(i,j)+(S_vertical(i+1,j)-S_vertical(i,j))/4)
    +...
208         phi_bar(i-1,j+1)*((S_vertical(i,j)+S_horizental(i,j+1))/4)+...
209         phi_bar(i-1,j)*(D(i,j) + ((S_horizental(i,j+1)-S_horizental(i,j))/4)
    )+...
210         phi_bar(i-1,j-1)*((-S_vertical(i,j)-S_horizental(i,j))/4))+(1-w)*
    phi_bar(i,j);
211         end
212         end
213         end
214         end
215
216         for i = 1 : 1 : m-1
217         phi_bar(i,n) = 2 * x_mid_n(i,n) - phi_bar(i,n-1);
218         end
219
220         L2 = L2_norm(phi_bar,phi_bar_old);
221         K_iter = K_iter + 1;
222         iteration(K_iter) = K_iter;
```

```
        L2_d(K_iter) = log10(L2);
        end

        figure(2)
        plot(iteration , L2_d , 'LineWidth' , 2.5)
        grid on;
        xlabel('Iteration');
        ylabel('$$\log ({\left\| {{\delta _{i,j}}} \right\|_2})$$','Interpreter','latex');
        title(['Convergency plot ($M_\infty$=', num2str(Mach), ')'],'Interpreter','latex');
        set(gca,'FontName','Times','FontSize',12)

        phi = zeros(m-1,n-1);
        for i = 1 : 1 : m-1
        for j = 1 : 1 : n-1
        phi(i,j) = phi_bar(i,j);
        end
        end

        fprintf('SOR with w = 1.7 converged in %d iterations.\n', K_iter);

        % ========================================
        % 5. POST PROCESSING (Cp Calculation)
        % ========================================
        phi_grid = zeros(m,n);
        for i = 1 : m
        for j = 2 : n-1
        if i == 1
        phi_grid(i,j) = 1/4 * (phi(m-1,j-1)+phi(i,j-1)+phi(i,j)+phi(m-1,j));
        elseif i == m
        phi_grid(i,j) = 1/4 * (phi(i-1,j-1)+phi(1,j-1)+phi(1,j)+phi(i-1,j));
        else
        phi_grid(i,j) = 1/4 * (phi(i-1,j-1)+phi(i,j-1)+phi(i,j)+phi(i-1,j));
        end
        end
        if i == 1
        phi_grid(i,1) = 1/2 * ((3/2*phi(m-1,1)-1/2*phi(m-1,2)) + (3/2*phi(i,1)-1/2*phi(i,2)));
        phi_grid(i,n) = x(i,n);
        elseif i == m
        phi_grid(i,1) = 1/2 * ((3/2*phi(i-1,1)-1/2*phi(i-1,2)) + (3/2*phi(1,1)-1/2*phi(1,2)));
        phi_grid(i,n) = x(i,n);
        else
        phi_grid(i,1) = 1/2 * ((3/2*phi(i-1,1)-1/2*phi(i-1,2)) + (3/2*phi(i,1)-1/2*phi(i,2)));
        phi_grid(i,n) = x(i,n);
        end
        end

        figure(3)
        surf(x,y,phi_grid)
        title('$\phi$ on grid points','Interpreter','latex')
        xlabel('$x$','Interpreter','latex')
```

```
        ylabel('$y$','Interpreter','latex')
        zlabel('$\phi$','Interpreter','latex')


        % Compute pressure coefficient
        phi_ksi = zeros(m-1,1);
        phi_eta = zeros(n-1,1);

        for i = 1 : m-1
        if i == 1
        phi_ksi(i) = 3/4*phi(i+1,1) - 3/4*phi(m-1,1) - 1/4*phi(i+1,2) + 1/4*phi(m-1,2);
        elseif i == m-1
        phi_ksi(i) = 3/4*phi(1,1) - 3/4*phi(i-1,1) - 1/4*phi(1,2) + 1/4*phi(i-1,2);
        else
        phi_ksi(i) = 3/4*phi(i+1,1) - 3/4*phi(i-1,1) - 1/4*phi(i+1,2) + 1/4*phi(i-1,2);
        phi_eta(i) = -2*phi(i,1) + 3*phi(i,2) - phi(i,3);
        end
        end
        phi_eta(1) = -2*phi(1,1) + 3*phi(1,2) - phi(1,3);
        phi_eta(m-1) = -2*phi(m-1,1) + 3*phi(m-1,2) - phi(m-1,3);

        % Velocity and Cp
        u = 1 ./ J_horizental( : , 1) .* (y_eta_n( : ,1).*phi_ksi( : ,1) - y_ksi_n( : ,1).*phi_eta( : ,1));
        v = 1 ./ J_horizental( : , 1) .* (-x_eta_n( : ,1).*phi_ksi( : ,1) + x_ksi_n( : ,1).*phi_eta( : ,1));
        c_p = 1 - (u.^2 + v.^2); % Standard approximation

        figure(4)
        plot(x_mid_n( : ,1) , c_p,'linewidth' , 2.5)
        grid on
        title(['Pressure coefficient ($M_\infty$=', num2str(Mach), ')'],'Interpreter','latex')
        xlabel('$x$','Interpreter','latex')
        ylabel('Pressure coefficient','Interpreter','latex')
        set(gca, 'YDir','reverse') % Cp is usually plotted reversed

        % Drag and lift coefficient calculation (remains same)
        L = 0;
        c_d = 0;
        c_l = 0;

        for i = 1 : m-1
        dx = x(i+1,1) - x(i,1);
        dy = y(i+1,1) - y(i,1);
        dL = sqrt(dx^2 + dy^2);
        width = 1;
        L = L + dL;
        theta_prime = atand(dy/dx);
        if dx < 0
        theta = abs(theta_prime) + 270;
        else
```

```
        theta = abs(theta_prime) + 90;
        end
        c_d = c_d + c_p(i) * cosd(theta) * dL * width;
        c_l = c_l + c_p(i) * sind(theta) * dL * width;
        end

        C_L = c_l / L;
        C_D = c_d / L;

        fprintf('Drag coefficient is %e \n', C_D);
        fprintf('Lift coefficient is %e \n', C_L);

        mach=zeros(m-1,1);
        gama=1.4;
        mach=(((2+(gama-1).*Mach.^2).*(1+gama.*Mach.^2./2.*c_p).^((1-gama)./
    gama)-2)./...
        (gama-1)).^0.5;

        figure(5)
        plot(x_mid_n( : ,1) , mach,'linewidth' , 2.5)
        grid on
        title(['Local Mach number ($M_\infty$=', num2str(Mach), ')'],'
    Interpreter','latex')
        xlabel('$x$','Interpreter','latex')
        ylabel('Local Mach number','Interpreter','latex')

        function [x,y,m,n] = TFI(ksi,eta)
        P14 = [1,0]; P23 = [41,0]; P34 = [41,0]; P12 = [1,0];
        m = size(ksi,2); n = size(eta,2);
        for i = 1 : m
        for j = 1 : n
        [x1,y1] = c1(ksi(i)); [x3,y3] = c3(ksi(i));
        [x2,y2] = c2(eta(j)); [x4,y4] = c4(eta(j));
        x(i,j) = (1-eta(j)) * x1+ eta(j) * x3+ksi(i) * x2+(1-ksi(i)) * x4
    -...
        ((1-ksi(i))*(1-eta(j))*P14(1)+ksi(i)*eta(j)*P23(1)+(1-ksi(i))*eta(j)
    *P34(1)+ksi(i)*(1-eta(j))*P12(1));
        y(i,j) = (1-eta(j)) * y1+ eta(j) * y3+ksi(i) * y2+(1-ksi(i)) * y4
    -...
        ((1-ksi(i))*(1-eta(j))*P14(2)+ksi(i)*eta(j)*P23(2)+(1-ksi(i))*eta(j)
    *P34(2)+ksi(i)*(1-eta(j))*P12(2));
        end
        end
        end

        function [x,y] = c1(ksi)
        if (ksi <= 0.5)
        y = -0.6*(0.29690*sqrt(1/2 * (cos(2*pi*(1-ksi))+1)) - 0.12600*(1/2 *
     (cos(2*pi*(1-ksi))+1)) - 0.35160*(1/2 * (cos(2*pi*(1-ksi))+1)).^2 +
    0.28430*(1/2 * (cos(2*pi*(1-ksi))+1)).^3 - (0.10150 + 0.0021)*(1/2 * (cos
    (2*pi*(1-ksi))+1)).^4);
        else
        y = 0.6*(0.29690*sqrt(1/2 * (cos(2*pi*(1-ksi))+1)) - 0.12600*(1/2 *
    (cos(2*pi*(1-ksi))+1)) - 0.35160*(1/2 * (cos(2*pi*(1-ksi))+1)).^2 +
    0.28430*(1/2 * (cos(2*pi*(1-ksi))+1)).^3 - (0.10150 + 0.0021)*(1/2 * (cos
```

```
    (2*pi*(1-ksi))+1)).^4);
366      end
367      x = 1/2 * (cos(2*pi*(1-ksi))+1);
368      end
369
370      function [x,y] = c2(eta); x = 40*eta + 1; y = 0; end
371      function [x,y] = c3(ksi)
372      if (ksi <= 0.5); y = -sqrt(41^2-(41*cos(2*pi*ksi))^2); else; y =
    sqrt(41^2-(41*cos(2*pi*ksi))^2); end
373      x = 41*cos(2*pi*ksi);
374      end
375      function [x,y] = c4(eta); x = 40*eta + 1; y = 0; end
376
377      function [x,y,dksi,deta]=Elliptic_mesh (x0,y0)
378      m=size(x0,1); n=size(x0,2);
379      dksi=1; deta=1; x=x0; y=y0; Er=1; tol=1.e-8; iter=0;
380      while Er>tol
381      Er=0;
382      for j=2:n-1
383      for i=1:m
384      if i == 1
385      a = ((x0(i,j+1)-x0(i,j-1))/(2*deta))^2 + ((y0(i,j+1)-y0(i,j-1))/(2*
    deta))^2;
386      b = (x0(i+1,j)-x0(m-1,j))*(x0(i,j+1)-x0(i,j-1))/(4*dksi*deta)+ (y0(i
    +1,j)-y0(m-1,j))*(y0(i,j+1)-y0(i,j-1))/(4*dksi*deta);
387      c= ((x0(i+1,j)-x0(m-1,j))/(2*dksi))^2+((y0(i+1,j)-y0(m-1,j))/(2*dksi
    ))^2;
388      coeff_LHS= 2. * (a/(dksi*dksi) + c / (deta* deta));
389      x(i,j) = 1/coeff_LHS*((a/(dksi*dksi))*(x(i+1,j)+x(m-1,j)) + (c/(deta
    *deta))*(x(i,j+1)+x(i,j-1))-(0.5*b/(dksi*deta))*(x(i+1,j+1)-x(i+1,j-1)+x(
    m-1,j-1)-x(m-1,j+1)));
390      y(i,j) = 1/coeff_LHS*((a/(dksi*dksi))*(y(i+1,j)+y(m-1,j)) + (c/(deta
    *deta))*(y(i,j+1)+y(i,j-1))-(0.5*b/(dksi*deta))*(y(i+1,j+1)-y(i+1,j-1)+y(
    m-1,j-1)-y(m-1,j+1)));
391      Er=Er + (x(i,j)-x0(i,j))^2 + (y(i,j)-y0(i,j))^2;
392      elseif i == m
393      a = ((x0(i,j+1)-x0(i,j-1))/(2*deta))^2 + ((y0(i,j+1)-y0(i,j-1))/(2*
    deta))^2;
394      b = (x0(2,j)-x0(m-1,j))*(x0(i,j+1)-x0(i,j-1))/(4*dksi*deta)+ (y0(2,j
    )-y0(m-1,j))*(y0(i,j+1)-y0(i,j-1))/(4*dksi*deta);
395      c= ((x0(2,j)-x0(m-1,j))/(2*dksi))^2+((y0(2,j)-y0(m-1,j))/(2*dksi))
    ^2;
396      coeff_LHS= 2. * (a/(dksi*dksi) + c / (deta* deta));
397      x(i,j) = 1/coeff_LHS*((a/(dksi*dksi))*(x(2,j)+x(m-1,j)) + (c/(deta*
    deta))*(x(i,j+1)+x(i,j-1))-(0.5*b/(dksi*deta))*(x(2,j+1)-x(2,j-1)+x(m-1,j
    -1)-x(m-1,j+1)));
398      y(i,j) = 1/coeff_LHS*((a/(dksi*dksi))*(y(2,j)+y(m-1,j)) + (c/(deta*
    deta))*(y(i,j+1)+y(i,j-1))-(0.5*b/(dksi*deta))*(y(2,j+1)-y(2,j-1)+y(m-1,j
    -1)-y(m-1,j+1)));
399      Er=Er + (x(i,j)-x0(i,j))^2 + (y(i,j)-y0(i,j))^2;
400      else
401      a = ((x0(i,j+1)-x0(i,j-1))/(2*deta))^2 + ((y0(i,j+1)-y0(i,j-1))/(2*
    deta))^2;
402      b = (x0(i+1,j)-x0(i-1,j))*(x0(i,j+1)-x0(i,j-1))/(4*dksi*deta)+ (y0(i
    +1,j)-y0(i-1,j))*(y0(i,j+1)-y0(i,j-1))/(4*dksi*deta);
```

```
403         c= ((x0(i+1,j)-x0(i-1,j))/(2*dksi))^2+((y0(i+1,j)-y0(i-1,j))/(2*dksi
    ))^2;
404         coeff_LHS= 2. * (a/(dksi*dksi) + c / (deta* deta));
405         x(i,j) = 1/coeff_LHS*((a/(dksi*dksi))*(x(i+1,j)+x(i-1,j)) + (c/(deta
    *deta))*(x(i,j+1)+x(i,j-1))-(0.5*b/(dksi*deta))*(x(i+1,j+1)-x(i+1,j-1)+x(
    i-1,j-1)-x(i-1,j+1)));
406         y(i,j) = 1/coeff_LHS*((a/(dksi*dksi))*(y(i+1,j)+y(i-1,j)) + (c/(deta
    *deta))*(y(i,j+1)+y(i,j-1))-(0.5*b/(dksi*deta))*(y(i+1,j+1)-y(i+1,j-1)+y(
    i-1,j-1)-y(i-1,j+1)));
407         Er=Er + (x(i,j)-x0(i,j))^2 + (y(i,j)-y0(i,j))^2;
408         end
409         end
410         end
411         Er = sqrt(Er/(2*(m-1)*(n-2))); x0=x; y0=y; iter=iter+1;
412         end
413         fprintf('Elliptic mesh converged in %d iterations.\n', iter);
414         end
415
416         function L2 = L2_norm(A , B)
417         m = size(A,1); n = size(A,2); d_p = 0; N = m * (n-1);
418         for i = 1 : 1 : m
419         for j = 1 : 1 : n-1
420         d_p = d_p + (A(i,j)-B(i,j))^2;
421         end
422         end
423         L2 = sqrt(d_p/N);
424         end
425
```

Listing 2: MATLAB Implementation Code for Parts C

C MATLAB Implementation Code for Part D

```
clc; clear; close all;

fprintf('Generating Grid and Geometric Metrics...\n');

eta = 0 : 1/30 : 1;
ksi = 0 : 1/60 : 1;

% Mesh generation (Algebraic + Elliptic Smoothing)
[x,y,m,n] = TFI(ksi,eta);
[x,y,dksi,deta] = Elliptic_mesh(x,y);

x_ksi_e = zeros(m,n-1); x_eta_e = zeros(m,n-1);
y_ksi_e = zeros(m,n-1); y_eta_e = zeros(m,n-1);
x_ksi_n = zeros(m-1,n); x_eta_n = zeros(m-1,n);
y_ksi_n = zeros(m-1,n); y_eta_n = zeros(m-1,n);

x_mid_e = zeros(m,n-1); y_mid_e = zeros(m,n-1);
x_mid_n = zeros(m-1,n); y_mid_n = zeros(m-1,n);

% Vertical face metrics
for j = 1 : n-1
for i = 1 : m-2
x_ksi_e(i+1,j) = (x(i+2,j+1) + x(i+2,j) - x(i,j+1) - x(i,j)) / (4*
dksi);
x_eta_e(i+1,j) = (x(i+1,j+1) - x(i+1,j)) / (deta);
y_ksi_e(i+1,j) = (y(i+2,j+1) + y(i+2,j) - y(i,j+1) - y(i,j)) / (4*
dksi);
y_eta_e(i+1,j) = (y(i+1,j+1) - y(i+1,j)) / (deta);
end
% Periodic BC handling
x_ksi_e(m,j) = (x(2,j+1) + x(2,j) - x(m-1,j+1) - x(m-1,j)) / (4*dksi
);
x_eta_e(m,j) = (x(m,j+1) - x(m,j)) / (deta);
x_ksi_e(1,j) = x_ksi_e(m,j);
x_eta_e(1,j) = x_eta_e(m,j);

y_ksi_e(m,j) = (y(2,j+1) + y(2,j) - y(m-1,j+1) - y(m-1,j)) / (4*dksi
);
y_eta_e(m,j) = (y(m,j+1) - y(m,j)) / (deta);
y_ksi_e(1,j) = y_ksi_e(m,j);
y_eta_e(1,j) = y_eta_e(m,j);
end

% Horizontal face metrics
for i = 1 : m-1
for j = 1 : n-2
x_ksi_n(i,j+1) = (x(i+1,j+1) - x(i,j+1)) / (dksi);
x_eta_n(i,j+1) = (x(i+1,j+2) + x(i,j+2) - x(i+1,j) - x(i,j)) / (4*
deta);
y_eta_n(i,j+1) = (y(i+1,j+2) + y(i,j+2) - y(i+1,j) - y(i,j)) / (4*
deta);
y_ksi_n(i,j+1) = (y(i+1,j+1) - y(i,j+1)) / (dksi);
```

Students: Hamid Motallebi ( )
Sheida Yarahmadi ( )

```
47        x_mid_n(i,j+1) = (x(i,j+1)+x(i+1,j+1))/2; % Needed for plotting
  later
48        end
49        % Boundaries
50        x_eta_n(i,n) = (3/2*(x(i,n)+x(i+1,n)) - 4/2*(x(i,n-1)+x(i+1,n-1)) +
  1/2*(x(i,n-2)+x(i+1,n-2))) / (2*deta);
51        x_ksi_n(i,n) = (x(i+1,n) - x(i,n)) / (dksi);
52        x_eta_n(i,1) = -(3/2*(x(i,1)+x(i+1,1)) - 4/2*(x(i,2)+x(i+1,2)) +
  1/2*(x(i,3)+x(i+1,3))) / (2*deta);
53        x_ksi_n(i,1) = (x(i+1,1) - x(i,1)) / (dksi);
54
55        y_eta_n(i,n) = (3/2*(y(i,n)+y(i+1,n)) - 4/2*(y(i,n-1)+y(i+1,n-1)) +
  1/2*(y(i,n-2)+y(i+1,n-2))) / (2*deta);
56        y_ksi_n(i,n) = (y(i+1,n) - y(i,n)) / (dksi);
57        y_eta_n(i,1) = -(3/2*(y(i,1)+y(i+1,1)) - 4/2*(y(i,2)+y(i+1,2)) +
  1/2*(y(i,3)+y(i+1,3))) / (2*deta);
58        y_ksi_n(i,1) = (y(i+1,1) - y(i,1)) / (dksi);
59
60        x_mid_n(i,1) = (x(i,1)+x(i+1,1))/2; % Needed for Plot
61        x_mid_n(i,n) = (x(i,n)+x(i+1,n))/2;
62        end
63
64        % Jacobians (Purely Geometric)
65        J_vertical = x_ksi_e .* y_eta_e - x_eta_e .* y_ksi_e;
66        J_horizental = x_ksi_n .* y_eta_n - x_eta_n .* y_ksi_n;
67
68
69        Mach_List = 0.50 : 0.01 : 0.80; % Scan range
70        Gamma = 1.4;
71
72        fprintf('\nStarting Search for Critical Mach Number...\n');
73        fprintf('
  ------------------------------------------------------------\n');
74        fprintf('|  M_inf  |  Status    |  Iter  |  Min Cp  | Max Local Mach
  |\n');
75        fprintf('
  ------------------------------------------------------------\n');
76
77        Critical_Mach_Found = false;
78        Final_Mach = 0;
79
80        for Mach = Mach_List
81
82        % --- A. Update Compressibility Physics ---
83        Comp_Factor = 1 - Mach^2;
84
85        % Recalculate Coefficients with new Mach Number
86        % Alpha term
87        D = (x_eta_e.*x_eta_e + Comp_Factor * y_eta_e.*y_eta_e) ./
  J_vertical;
88        % Gamma term
89        D_star = (x_ksi_n.*x_ksi_n + Comp_Factor * y_ksi_n.*y_ksi_n) ./
  J_horizental;
90        % Beta term
```

```
91        S_vertical = (x_eta_e.*x_ksi_e + Comp_Factor * y_eta_e.*y_ksi_e) ./
    J_vertical;
92        S_horizental = (x_eta_n.*x_ksi_n + Comp_Factor * y_eta_n.*y_ksi_n)
    ./ J_horizental;
93
94
95        % --- B. SOR Solver ---
96        phi_bar = zeros(m-1,n);
97        phi_bar_old = zeros(m-1,n);
98        w = 1.7;
99        tolerance = 1e-6; % Slightly relaxed for speed in loop
100       L2 = 10;
101       iter = 0;
102       Max_Iter = 3000;
103
104       % Initial guess for Farfield
105       for i = 1 : m-1
106       phi_bar(i,n) = 2 * x_mid_n(i,n) - phi_bar(i,n-1);
107       end
108
109       Converged = false;
110
111       while L2 > tolerance && iter < Max_Iter
112       phi_bar_old = phi_bar;
113
114       % --- SOR Calculation (Same as original) ---
115       for j = 1 : n-1
116       for i = 1 : m-1
117       if j == 1
118       % Inner Boundary (Airfoil Surface)
119       if i == 1
120       phi_bar(i,j) = -w / (-D(i+1,j)-D_star(i,j+1)-D(i,j)+3/4*S_vertical(i
    +1,j)-3/4*S_vertical(i,j)) *( ...
121       phi_bar(i+1,j+1) * (-S_horizental(i,j+1)/4 - S_vertical(i+1,j))  +
    ...
122       phi_bar(i+1,j)*(D(i+1,j) + ((-S_horizental(i,j+1)+3*S_vertical(i+1,j
    ))/4))+...
123       phi_bar(i+1,j+2)*(S_vertical(i+1,j)/4)+...
124       phi_bar(i,j+1)*(D_star(i,j+1)-S_vertical(i+1,j)+S_vertical(i,j))+...
125       phi_bar(i,j+2)*((S_vertical(i+1,j)-S_vertical(i,j))/4)+...
126       phi_bar(m-1,j+1)*(S_vertical(i,j)+S_horizental(i,j+1)/4)+...
127       phi_bar(m-1,j)*(D(i,j) + ((S_horizental(i,j+1)-3*S_vertical(i,j))/4)
    )+...
128       phi_bar(m-1,j+2)*(-S_vertical(i,j)/4))+(1-w)*phi_bar(i,j);
129       elseif i == m-1
130       phi_bar(i,j) = -w / (-D(i+1,j)-D_star(i,j+1)-D(i,j)+3/4*S_vertical(i
    +1,j)-3/4*S_vertical(i,j)) *( ...
131       phi_bar(1,j+1) * (-S_horizental(i,j+1)/4 - S_vertical(i+1,j))  + ...
132       phi_bar(1,j)*(D(i+1,j) + ((-S_horizental(i,j+1)+3*S_vertical(i+1,j))
    /4))+...
133       phi_bar(1,j+2)*(S_vertical(i+1,j)/4)+...
134       phi_bar(i,j+1)*(D_star(i,j+1)-S_vertical(i+1,j)+S_vertical(i,j))+...
135       phi_bar(i,j+2)*((S_vertical(i+1,j)-S_vertical(i,j))/4)+...
136       phi_bar(i-1,j+1)*(S_vertical(i,j)+S_horizental(i,j+1)/4)+...
```

```
137         phi_bar(i-1,j)*(D(i,j) + ((S_horizental(i,j+1)-3*S_vertical(i,j))/4)
    )+...
138         phi_bar(i-1,j+2)*(-S_vertical(i,j)/4))+(1-w)*phi_bar(i,j);
139         else
140         phi_bar(i,j) = -w / (-D(i+1,j)-D_star(i,j+1)-D(i,j)+3/4*S_vertical(i
    +1,j)-3/4*S_vertical(i,j)) *( ...
141         phi_bar(i+1,j+1) * (-S_horizental(i,j+1)/4 - S_vertical(i+1,j))  +
    ...
142         phi_bar(i+1,j)*(D(i+1,j) + ((-S_horizental(i,j+1)+3*S_vertical(i+1,j
    ))/4))+...
143         phi_bar(i+1,j+2)*(S_vertical(i+1,j)/4)+...
144         phi_bar(i,j+1)*(D_star(i,j+1)-S_vertical(i+1,j)+S_vertical(i,j))+...
145         phi_bar(i,j+2)*((S_vertical(i+1,j)-S_vertical(i,j))/4)+...
146         phi_bar(i-1,j+1)*(S_vertical(i,j)+S_horizental(i,j+1)/4)+...
147         phi_bar(i-1,j)*(D(i,j) + ((S_horizental(i,j+1)-3*S_vertical(i,j))/4)
    )+...
148         phi_bar(i-1,j+2)*(-S_vertical(i,j)/4))+(1-w)*phi_bar(i,j);
149         end
150         else
151         % Internal Points
152         if i==1
153         phi_bar(i,j) = -w / (-D(i+1,j)-D_star(i,j+1)-D(i,j)-D_star(i,j)) *(
    ...
154         phi_bar(i+1,j+1) * (-S_vertical(i+1,j) - S_horizental(i,j+1)) / 4  +
     ...
155         phi_bar(i+1,j)*(D(i+1,j) + ((-S_horizental(i,j+1)+S_horizental(i,j))
    /4))+...
156         phi_bar(i+1,j-1)*((S_vertical(i+1,j)+S_horizental(i,j))/4)+...
157         phi_bar(i,j+1)*(D_star(i,j+1)+(-S_vertical(i+1,j)+S_vertical(i,j))
    /4)+...
158         phi_bar(i,j-1)*(D_star(i,j)+(S_vertical(i+1,j)-S_vertical(i,j))/4)
    +...
159         phi_bar(m-1,j+1)*((S_vertical(i,j)+S_horizental(i,j+1))/4)+...
160         phi_bar(m-1,j)*(D(i,j) + ((S_horizental(i,j+1)-S_horizental(i,j))/4)
    )+...
161         phi_bar(m-1,j-1)*((-S_vertical(i,j)-S_horizental(i,j))/4))+(1-w)*
    phi_bar(i,j);
162         elseif i==m-1
163         phi_bar(i,j) = -w / (-D(i+1,j)-D_star(i,j+1)-D(i,j)-D_star(i,j)) *(
    ...
164         phi_bar(1,j+1) * (-S_vertical(i+1,j) - S_horizental(i,j+1)) / 4  +
    ...
165         phi_bar(1,j)*(D(i+1,j) + ((-S_horizental(i,j+1)+S_horizental(i,j))
    /4))+...
166         phi_bar(1,j-1)*((S_vertical(i+1,j)+S_horizental(i,j))/4)+...
167         phi_bar(i,j+1)*(D_star(i,j+1)+(-S_vertical(i+1,j)+S_vertical(i,j))
    /4)+...
168         phi_bar(i,j-1)*(D_star(i,j)+(S_vertical(i+1,j)-S_vertical(i,j))/4)
    +...
169         phi_bar(i-1,j+1)*((S_vertical(i,j)+S_horizental(i,j+1))/4)+...
170         phi_bar(i-1,j)*(D(i,j) + ((S_horizental(i,j+1)-S_horizental(i,j))/4)
    )+...
171         phi_bar(i-1,j-1)*((-S_vertical(i,j)-S_horizental(i,j))/4))+(1-w)*
    phi_bar(i,j);
172         else
```

```
        phi_bar(i,j) = -w / (-D(i+1,j)-D_star(i,j+1)-D(i,j)-D_star(i,j)) *(
...
        phi_bar(i+1,j+1) * (-S_vertical(i+1,j) - S_horizental(i,j+1)) / 4  +
 ...
        phi_bar(i+1,j)*(D(i+1,j) + ((-S_horizental(i,j+1)+S_horizental(i,j))
/4))+...
        phi_bar(i+1,j-1)*((S_vertical(i+1,j)+S_horizental(i,j))/4)+...
        phi_bar(i,j+1)*(D_star(i,j+1)+(-S_vertical(i+1,j)+S_vertical(i,j))
/4)+...
        phi_bar(i,j-1)*(D_star(i,j)+(S_vertical(i+1,j)-S_vertical(i,j))/4)
+...
        phi_bar(i-1,j+1)*((S_vertical(i,j)+S_horizental(i,j+1))/4)+...
        phi_bar(i-1,j)*(D(i,j) + ((S_horizental(i,j+1)-S_horizental(i,j))/4)
)+...
        phi_bar(i-1,j-1)*((-S_vertical(i,j)-S_horizental(i,j))/4))+(1-w)*
phi_bar(i,j);
        end
        end
        end
        end

        % Update Farfield BC
        for i = 1 : m-1
        phi_bar(i,n) = 2 * x_mid_n(i,n) - phi_bar(i,n-1);
        end

        L2 = L2_norm(phi_bar,phi_bar_old);
        iter = iter + 1;

        % Divergence Check
        if isnan(L2) || L2 > 1e4
        Converged = false;
        break;
        end
        end

        if L2 <= tolerance
        Converged = true;
        end

        % --- C. Post Process & Criticality Check ---
        if Converged
        phi = phi_bar(1:m-1, 1:n-1);

        % Calculate Derivatives (approx) for Velocity
        phi_ksi = zeros(m-1,1);
        phi_eta = zeros(n-1,1);

        for i = 1 : m-1
        if i == 1
        phi_ksi(i) = 3/4*phi(i+1,1) - 3/4*phi(m-1,1) - 1/4*phi(i+1,2) + 1/4*
phi(m-1,2);
        elseif i == m-1
        phi_ksi(i) = 3/4*phi(1,1) - 3/4*phi(i-1,1) - 1/4*phi(1,2) + 1/4*phi(
i-1,2);
```

```
219       else
220       phi_ksi(i) = 3/4*phi(i+1,1) - 3/4*phi(i-1,1) - 1/4*phi(i+1,2) + 1/4*
      phi(i-1,2);
221       end
222       phi_eta(i) = -2*phi(i,1) + 3*phi(i,2) - phi(i,3);
223       end
224       phi_eta(m-1) = -2*phi(m-1,1) + 3*phi(m-1,2) - phi(m-1,3);
225
226       u = 1 ./ J_horizental(:,1) .* (y_eta_n(:,1).*phi_ksi(:) - y_ksi_n
      (:,1).*phi_eta(:));
227       v = 1 ./ J_horizental(:,1) .* (-x_eta_n(:,1).*phi_ksi(:) + x_ksi_n
      (:,1).*phi_eta(:));
228
229       c_p = 1 - (u.^2 + v.^2);
230
231       local_mach_arr = (((2+(Gamma-1).*Mach.^2).*(1+Gamma.*Mach.^2./2.*c_p
      ).^((1-Gamma)./Gamma)-2)./(Gamma-1)).^0.5;
232
233       max_M_local = max(real(local_mach_arr)); % Use real() to avoid tiny
      imag parts if Cp is weird
234       min_Cp = min(c_p);
235
236       fprintf('|  %.2f   |  Conv     |  %4d  |  %6.3f  |      %.3f      |\
      n', ...
237       Mach, iter, min_Cp, max_M_local);
238
239       if max_M_local >= 1.0
240       Critical_Mach_Found = true;
241       Final_Mach = Mach;
242       fprintf('
      ----------------------------------------------------------------\n');
243       fprintf('>>> CRITICAL MACH NUMBER REACHED! M_local >= 1.0 <<<\n');
244       break;
245       end
246
247       else
248       fprintf('|  %.2f   |  DIVERGED |  %4d  |   ----   |      ----
      |\n', Mach, iter);
249       break; % Stop if diverged
250       end
251       end
252
253       if ~Critical_Mach_Found
254       fprintf('Loop finished without reaching M=1 or Divergence. M_crit >
      %.2f\n', Mach_List(end));
255       Final_Mach = Mach_List(end);
256       end
257
258       fprintf('\nPlotting results for M_inf = %.2f\n', Final_Mach);
259
260       figure(1)
261       plot(x_mid_n(:,1), c_p, 'linewidth', 2.5)
262       grid on
263       title(['Cp distribution at Critical Mach ($M_\infty$=', num2str(
      Final_Mach), ')'], 'Interpreter','latex')
```

```
        xlabel('$x$','Interpreter','latex')
        ylabel('$C_p$','Interpreter','latex')
        set(gca, 'YDir','reverse')

        figure(2)
        plot(x_mid_n(:,1), local_mach_arr, 'linewidth', 2.5, 'Color', 'r')
        hold on
        yline(1.0, '--k', 'Sonic Line');
        grid on
        title(['Local Mach Number at Critical Condition ($M_\infty$=', num2str(Final_Mach), ')'], 'Interpreter','latex')
        xlabel('$x$','Interpreter','latex')
        ylabel('$M_{local}$','Interpreter','latex')


        function [x,y,m,n] = TFI(ksi,eta)
        P14 = [1,0]; P23 = [41,0]; P34 = [41,0]; P12 = [1,0];
        m = size(ksi,2); n = size(eta,2);
        for i = 1 : m
        for j = 1 : n
        [x1,y1] = c1(ksi(i)); [x3,y3] = c3(ksi(i));
        [x2,y2] = c2(eta(j)); [x4,y4] = c4(eta(j));
        x(i,j) = (1-eta(j)) * x1+ eta(j) * x3+ksi(i) * x2+(1-ksi(i)) * x4 -...
        ((1-ksi(i))*(1-eta(j))*P14(1)+ksi(i)*eta(j)*P23(1)+(1-ksi(i))*eta(j)*P34(1)+ksi(i)*(1-eta(j))*P12(1));
        y(i,j) = (1-eta(j)) * y1+ eta(j) * y3+ksi(i) * y2+(1-ksi(i)) * y4 -...
        ((1-ksi(i))*(1-eta(j))*P14(2)+ksi(i)*eta(j)*P23(2)+(1-ksi(i))*eta(j)*P34(2)+ksi(i)*(1-eta(j))*P12(2));
        end
        end
        end

        function [x,y] = c1(ksi)
        % NACA 0012 Geometry
        if (ksi <= 0.5)
        y = -0.6*(0.29690*sqrt(1/2 * (cos(2*pi*(1-ksi))+1)) - 0.12600*(1/2 * (cos(2*pi*(1-ksi))+1)) - 0.35160*(1/2 * (cos(2*pi*(1-ksi))+1)).^2 + 0.28430*(1/2 * (cos(2*pi*(1-ksi))+1)).^3 - (0.10150 + 0.0021)*(1/2 * (cos(2*pi*(1-ksi))+1)).^4);
        else
        y = 0.6*(0.29690*sqrt(1/2 * (cos(2*pi*(1-ksi))+1)) - 0.12600*(1/2 * (cos(2*pi*(1-ksi))+1)) - 0.35160*(1/2 * (cos(2*pi*(1-ksi))+1)).^2 + 0.28430*(1/2 * (cos(2*pi*(1-ksi))+1)).^3 - (0.10150 + 0.0021)*(1/2 * (cos(2*pi*(1-ksi))+1)).^4);
        end
        x = 1/2 * (cos(2*pi*(1-ksi))+1);
        end

        function [x,y] = c2(eta); x = 40*eta + 1; y = 0; end
        function [x,y] = c3(ksi)
        % Farfield
        if (ksi <= 0.5); y = -sqrt(41^2-(41*cos(2*pi*ksi))^2); else; y = sqrt(41^2-(41*cos(2*pi*ksi))^2); end
```

```
307         x = 41*cos(2*pi*ksi);
308         end
309         function [x,y] = c4(eta); x = 40*eta + 1; y = 0; end
310
311         function [x,y,dksi,deta]=Elliptic_mesh (x0,y0)
312         m=size(x0,1); n=size(x0,2);
313         dksi=1; deta=1; x=x0; y=y0; Er=1; tol=1.e-7; iter=0;
314         % Relaxed tolerance slightly for speed if needed
315         while Er>tol && iter < 5000
316         Er=0;
317         for j=2:n-1
318         for i=1:m
319         if i == 1
320         a = ((x0(i,j+1)-x0(i,j-1))/(2*deta))^2 + ((y0(i,j+1)-y0(i,j-1))/(2*
    deta))^2;
321         b = (x0(i+1,j)-x0(m-1,j))*(x0(i,j+1)-x0(i,j-1))/(4*dksi*deta)+ (y0(i
    +1,j)-y0(m-1,j))*(y0(i,j+1)-y0(i,j-1))/(4*dksi*deta);
322         c= ((x0(i+1,j)-x0(m-1,j))/(2*dksi))^2+((y0(i+1,j)-y0(m-1,j))/(2*dksi
    ))^2;
323         coeff_LHS= 2. * (a/(dksi*dksi) + c / (deta* deta));
324         x(i,j) = 1/coeff_LHS*((a/(dksi*dksi))*(x(i+1,j)+x(m-1,j)) + (c/(deta
    *deta))*(x(i,j+1)+x(i,j-1))-(0.5*b/(dksi*deta))*(x(i+1,j+1)-x(i+1,j-1)+x(
    m-1,j-1)-x(m-1,j+1)));
325         y(i,j) = 1/coeff_LHS*((a/(dksi*dksi))*(y(i+1,j)+y(m-1,j)) + (c/(deta
    *deta))*(y(i,j+1)+y(i,j-1))-(0.5*b/(dksi*deta))*(y(i+1,j+1)-y(i+1,j-1)+y(
    m-1,j-1)-y(m-1,j+1)));
326         Er=Er + (x(i,j)-x0(i,j))^2 + (y(i,j)-y0(i,j))^2;
327         elseif i == m
328         a = ((x0(i,j+1)-x0(i,j-1))/(2*deta))^2 + ((y0(i,j+1)-y0(i,j-1))/(2*
    deta))^2;
329         b = (x0(2,j)-x0(m-1,j))*(x0(i,j+1)-x0(i,j-1))/(4*dksi*deta)+ (y0(2,j
    )-y0(m-1,j))*(y0(i,j+1)-y0(i,j-1))/(4*dksi*deta);
330         c= ((x0(2,j)-x0(m-1,j))/(2*dksi))^2+((y0(2,j)-y0(m-1,j))/(2*dksi))
    ^2;
331         coeff_LHS= 2. * (a/(dksi*dksi) + c / (deta* deta));
332         x(i,j) = 1/coeff_LHS*((a/(dksi*dksi))*(x(2,j)+x(m-1,j)) + (c/(deta*
    deta))*(x(i,j+1)+x(i,j-1))-(0.5*b/(dksi*deta))*(x(2,j+1)-x(2,j-1)+x(m-1,j
    -1)-x(m-1,j+1)));
333         y(i,j) = 1/coeff_LHS*((a/(dksi*dksi))*(y(2,j)+y(m-1,j)) + (c/(deta*
    deta))*(y(i,j+1)+y(i,j-1))-(0.5*b/(dksi*deta))*(y(2,j+1)-y(2,j-1)+y(m-1,j
    -1)-y(m-1,j+1)));
334         Er=Er + (x(i,j)-x0(i,j))^2 + (y(i,j)-y0(i,j))^2;
335         else
336         a = ((x0(i,j+1)-x0(i,j-1))/(2*deta))^2 + ((y0(i,j+1)-y0(i,j-1))/(2*
    deta))^2;
337         b = (x0(i+1,j)-x0(i-1,j))*(x0(i,j+1)-x0(i,j-1))/(4*dksi*deta)+ (y0(i
    +1,j)-y0(i-1,j))*(y0(i,j+1)-y0(i,j-1))/(4*dksi*deta);
338         c= ((x0(i+1,j)-x0(i-1,j))/(2*dksi))^2+((y0(i+1,j)-y0(i-1,j))/(2*dksi
    ))^2;
339         coeff_LHS= 2. * (a/(dksi*dksi) + c / (deta* deta));
340         x(i,j) = 1/coeff_LHS*((a/(dksi*dksi))*(x(i+1,j)+x(i-1,j)) + (c/(deta
    *deta))*(x(i,j+1)+x(i,j-1))-(0.5*b/(dksi*deta))*(x(i+1,j+1)-x(i+1,j-1)+x(
    i-1,j-1)-x(i-1,j+1)));
341         y(i,j) = 1/coeff_LHS*((a/(dksi*dksi))*(y(i+1,j)+y(i-1,j)) + (c/(deta
    *deta))*(y(i,j+1)+y(i,j-1))-(0.5*b/(dksi*deta))*(y(i+1,j+1)-y(i+1,j-1)+y(
```

```
    i-1,j-1)-y(i-1,j+1)));
342         Er=Er + (x(i,j)-x0(i,j))^2 + (y(i,j)-y0(i,j))^2;
343         end
344         end
345         end
346         Er = sqrt(Er/(2*(m-1)*(n-2))); x0=x; y0=y; iter=iter+1;
347         end
348         fprintf('Elliptic mesh converged in %d iterations.\n', iter);
349         end
350
351         function L2 = L2_norm(A , B)
352         m = size(A,1); n = size(A,2); d_p = 0; N = m * (n-1);
353         % Vectorized for speed
354         d_p = sum(sum((A - B).^2));
355         L2 = sqrt(d_p/N);
356         end
357
```

Listing 3: Implemented MATLAB code for Part D

D MATLAB Implementation Code for Part E

```
clc; clear; close all;

%% 1. Mesh Generation (Only Once)
fprintf('Generating Mesh...\n');
[x_mid, metrics, m, n] = Generate_Mesh_And_Metrics();

%% 2. Solve for Different Mach Numbers
fprintf('Solving for Incompressible Flow (M ~ 0)...\n');
[Cp_0, ~, ~] = Solve_Potential(0.001, metrics, m, n); % Use very small Mach

% Case B: Part (c) -> M = 0.2
Mach_c = 0.2;
fprintf('Solving for Part (c) M = %.2f...\n', Mach_c);
[Cp_c_Num, Mach_c_Dist, Vel_c_Dist] = Solve_Potential(Mach_c, metrics, m, n);

% Case C: Part (d) -> M = Critical (approx 0.70 from previous run)
Mach_d = 0.70;
fprintf('Solving for Part (d) M = %.2f (Critical)...\n', Mach_d);
[Cp_d_Num, Mach_d_Dist, Vel_d_Dist] = Solve_Potential(Mach_d, metrics, m, n);

%% 3. Apply Prandtl-Glauert Rule
% PG for M = 0.2
Beta_c = sqrt(1 - Mach_c^2);
Cp_c_PG = Cp_0 / Beta_c;

% PG for M = Critical
Beta_d = sqrt(1 - Mach_d^2);
Cp_d_PG = Cp_0 / Beta_d;

%% 4. Plotting Results
% --- PLOT SET 1: M = 0.2 (Part c) ---
figure('Name', 'Part (c) Results', 'Color', 'w');

subplot(3,1,1);
plot(x_mid, Cp_c_Num, 'b-', 'LineWidth', 2); hold on;
plot(x_mid, Cp_c_PG, 'r--', 'LineWidth', 2);
title(['(c) Cp Distribution at M_{\infty} = ' num2str(Mach_c)]);
xlabel('x/c'); ylabel('C_p');
legend('Numerical Solver', 'Prandtl-Glauert Rule');
set(gca, 'YDir', 'reverse'); grid on;

subplot(3,1,2);
plot(x_mid, Mach_c_Dist, 'k', 'LineWidth', 1.5);
title(['Surface Mach Number (M_{\infty} = ' num2str(Mach_c) ')']);
xlabel('x/c'); ylabel('M_{local}'); grid on;

subplot(3,1,3);
plot(x_mid, Vel_c_Dist, 'm', 'LineWidth', 1.5);
title(['Surface Speed Distribution (V/U_{\infty})']);
```

```
        xlabel('x/c'); ylabel('Speed'); grid on;

        % --- PLOT SET 2: M = Critical (Part d) ---
        figure('Name', 'Part (d) Results', 'Color', 'w');

        subplot(3,1,1);
        plot(x_mid, Cp_d_Num, 'b-', 'LineWidth', 2); hold on;
        plot(x_mid, Cp_d_PG, 'r--', 'LineWidth', 2);
        title(['(d) Cp Distribution at M_{\infty} = ' num2str(Mach_d) ' (Critical)']);
        xlabel('x/c'); ylabel('C_p');
        legend('Numerical Solver', 'Prandtl-Glauert Rule');
        set(gca, 'YDir', 'reverse'); grid on;

        subplot(3,1,2);
        plot(x_mid, Mach_d_Dist, 'k', 'LineWidth', 1.5);
        yline(1.0, '--r', 'Sonic Line');
        title(['Surface Mach Number (M_{\infty} = ' num2str(Mach_d) ')']);
        xlabel('x/c'); ylabel('M_{local}'); grid on;

        subplot(3,1,3);
        plot(x_mid, Vel_d_Dist, 'm', 'LineWidth', 1.5);
        title(['Surface Speed Distribution (V/U_{\infty})']);
        xlabel('x/c'); ylabel('Speed'); grid on;

        %% ==================== FUNCTION DEFINITIONS ====================

        function [Cp, M_local, Speed] = Solve_Potential(Mach, metrics, m, n)
        % Extract Metrics
        D = metrics.D; D_star = metrics.D_star;
        S_v = metrics.S_v; S_h = metrics.S_h;
        J_h = metrics.J_h;
        xn_mid = metrics.xn_mid;
        xn_eta = metrics.xn_eta; xn_ksi = metrics.xn_ksi;
        yn_eta = metrics.yn_eta; yn_ksi = metrics.yn_ksi;

        % Adjust Coefficients for Compressibility
        Comp_Factor = 1 - Mach^2;
        xe = metrics.xe; ye = metrics.ye;
        xn = metrics.xn; yn = metrics.yn;
        Jv = metrics.Jv; Jh = metrics.J_h;

        % Recalculate Coefficients
        D_solver = (xe.x_eta.^2 + Comp_Factor * ye.y_eta.^2) ./ Jv;
        D_star_solver = (xn.x_ksi.^2 + Comp_Factor * yn.y_ksi.^2) ./ Jh;
        S_v_solver = (xe.x_eta.*xe.x_ksi + Comp_Factor * ye.y_eta.*ye.y_ksi) ./ Jv;
        S_h_solver = (xn.x_eta.*xn.x_ksi + Comp_Factor * yn.y_eta.*yn.y_ksi) ./ Jh;

        % SOR SOLVER
        phi_bar = zeros(m-1,n);
        w = 1.7; iter = 0; Max_Iter = 3000; L2 = 1; tol = 1e-5;

        % Initial Farfield
```

```
        for i = 1 : m-1; phi_bar(i,n) = 2 * xn_mid(i,n) - phi_bar(i,n-1);
    end

        while L2 > tol && iter < Max_Iter
        phi_old = phi_bar;
        for j = 1 : n-1
        for i = 1 : m-1
        % Define neighbors with Periodic BC
        ip1 = i+1; im1 = i-1;
        if i==1; im1 = m-1; elseif i==m-1; ip1 = 1; end

        term_cross = 0;
        term_main = 0;

        if j == 1 % Surface
        % (Logic specific to surface boundary condition)
        phi_bar(i,j) = -w / (-D_solver(i+1,j)-D_star_solver(i,j+1)-D_solver(i,j)+3/4*S_v_solver(i+1,j)-3/4*S_v_solver(i,j)) *( ...
        phi_bar(ip1,j+1) * (-S_h_solver(i,j+1)/4 - S_v_solver(i+1,j))  + ...
        phi_bar(ip1,j)*(D_solver(i+1,j) + ((-S_h_solver(i,j+1)+3*S_v_solver(i+1,j))/4))+...
        phi_bar(ip1,j+2)*(S_v_solver(i+1,j)/4)+...
        phi_bar(i,j+1)*(D_star_solver(i,j+1)-S_v_solver(i+1,j)+S_v_solver(i,j))+...
        phi_bar(i,j+2)*((S_v_solver(i+1,j)-S_v_solver(i,j))/4)+...
        phi_bar(im1,j+1)*(S_v_solver(i,j)+S_h_solver(i,j+1)/4)+...
        phi_bar(im1,j)*(D_solver(i,j) + ((S_h_solver(i,j+1)-3*S_v_solver(i,j))/4))+...
        phi_bar(im1,j+2)*(-S_v_solver(i,j)/4))+(1-w)*phi_bar(i,j);
        else % Internal
        phi_bar(i,j) = -w / (-D_solver(i+1,j)-D_star_solver(i,j+1)-D_solver(i,j)-D_star_solver(i,j)) *( ...
        phi_bar(ip1,j+1) * (-S_v_solver(i+1,j) - S_h_solver(i,j+1)) / 4  + ...
        phi_bar(ip1,j)*(D_solver(i+1,j) + ((-S_h_solver(i,j+1)+S_h_solver(i,j))/4))+...
        phi_bar(ip1,j-1)*((S_v_solver(i+1,j)+S_h_solver(i,j))/4)+...
        phi_bar(i,j+1)*(D_star_solver(i,j+1)+(-S_v_solver(i+1,j)+S_v_solver(i,j))/4)+...
        phi_bar(i,j-1)*(D_star_solver(i,j)+(S_v_solver(i+1,j)-S_v_solver(i,j))/4)+...
        phi_bar(im1,j+1)*((S_v_solver(i,j)+S_h_solver(i,j+1))/4)+...
        phi_bar(im1,j)*(D_solver(i,j) + ((S_h_solver(i,j+1)-S_h_solver(i,j))/4))+...
        phi_bar(im1,j-1)*((-S_v_solver(i,j)-S_h_solver(i,j))/4))+(1-w)*phi_bar(i,j);
        end
        end
        end
        % Update Farfield
        for k = 1 : m-1; phi_bar(k,n) = 2 * xn_mid(k,n) - phi_bar(k,n-1);
    end

        L2 = sqrt(sum(sum((phi_bar - phi_old).^2))/(m*(n-1)));
        iter = iter + 1;
```

```
144         end
145
146         % Post Processing
147         phi = phi_bar(1:m-1, 1);
148         phi_next = [phi(2:end); phi(1)];
149         phi_prev = [phi(end); phi(1:end-1)];
150         phi_up = phi_bar(1:m-1, 2);
151         phi_up2 = phi_bar(1:m-1, 3);
152
153         % Derivatives
154         phi_ksi = (3/4*phi_next - 3/4*phi_prev - 1/4*phi_bar([2:end,1],2) +
    1/4*phi_bar([end,1:end-1],2));
155         phi_eta = -2*phi + 3*phi_up - phi_up2;
156
157         u = 1 ./ J_h(:,1) .* (yn_eta(:,1).*phi_ksi - yn_ksi(:,1).*phi_eta);
158         v = 1 ./ J_h(:,1) .* (-xn_eta(:,1).*phi_ksi + xn_ksi(:,1).*phi_eta);
159
160         Speed = sqrt(u.^2 + v.^2);
161         Cp = 1 - Speed.^2;
162         Gamma = 1.4;
163         M_local = sqrt(( (2+(Gamma-1)*Mach^2).*(1+Gamma*Mach^2/2*Cp).^((1-
    Gamma)/Gamma) - 2 ) / (Gamma-1));
164         end
165
166         function [x_mid, metrics, m, n] = Generate_Mesh_And_Metrics()
167         % Simplified wrapper for previous mesh code
168         eta = 0 : 1/30 : 1; ksi = 0 : 1/60 : 1;
169         [x,y,~,~] = TFI(ksi,eta);
170         [x,y,dksi,deta] = Elliptic_mesh(x,y);
171
172         m = size(x,1); n = size(x,2);
173         [x_ksi_e, x_eta_e, y_ksi_e, y_eta_e, x_ksi_n, x_eta_n, y_ksi_n,
    y_eta_n, x_mid_n, y_mid_n, J_v, J_h] ...
174         = Calc_Geo_Metrics(x,y,dksi,deta);
175
176         metrics.xe.x_eta = x_eta_e; metrics.xe.x_ksi = x_ksi_e;
177         metrics.ye.y_eta = y_eta_e; metrics.ye.y_ksi = y_ksi_e;
178         metrics.xn.x_eta = x_eta_n; metrics.xn.x_ksi = x_ksi_n;
179         metrics.yn.y_eta = y_eta_n; metrics.yn.y_ksi = y_ksi_n;
180         metrics.xn_mid = x_mid_n; metrics.Jv = J_v; metrics.J_h = J_h;
181
182         metrics.xn_eta = x_eta_n; metrics.xn_ksi = x_ksi_n;
183         metrics.yn_eta = y_eta_n; metrics.yn_ksi = y_ksi_n;
184
185         metrics.D = zeros(m,n); metrics.D_star = zeros(m,n);
186         metrics.S_v = zeros(m,n); metrics.S_h = zeros(m,n);
187
188         x_mid = x_mid_n(:,1);
189         end
190
191         function [x,y,m,n] = TFI(ksi,eta)
192         % (Same as previous code)
193         P14 = [1,0]; P23 = [41,0]; P34 = [41,0]; P12 = [1,0];
194         m = size(ksi,2); n = size(eta,2);
195         for i = 1 : m
```

```
        for j = 1 : n
        [x1,y1] = c1(ksi(i)); [x3,y3] = c3(ksi(i));
        [x2,y2] = c2(eta(j)); [x4,y4] = c4(eta(j));
        x(i,j) = (1-eta(j)) * x1+ eta(j) * x3+ksi(i) * x2+(1-ksi(i)) * x4
    -...
        ((1-ksi(i))*(1-eta(j))*P14(1)+ksi(i)*eta(j)*P23(1)+(1-ksi(i))*eta(j)
    *P34(1)+ksi(i)*(1-eta(j))*P12(1));
        y(i,j) = (1-eta(j)) * y1+ eta(j) * y3+ksi(i) * y2+(1-ksi(i)) * y4
    -...
        ((1-ksi(i))*(1-eta(j))*P14(2)+ksi(i)*eta(j)*P23(2)+(1-ksi(i))*eta(j)
    *P34(2)+ksi(i)*(1-eta(j))*P12(2));
        end
        end
        end

        function [x,y] = c1(ksi)
        if (ksi <= 0.5)
        y = -0.6*(0.29690*sqrt(1/2 * (cos(2*pi*(1-ksi))+1)) - 0.12600*(1/2 *
         (cos(2*pi*(1-ksi))+1)) - 0.35160*(1/2 * (cos(2*pi*(1-ksi))+1)).^2 +
    0.28430*(1/2 * (cos(2*pi*(1-ksi))+1)).^3 - (0.10150 + 0.0021)*(1/2 * (cos
    (2*pi*(1-ksi))+1)).^4);
        else
        y = 0.6*(0.29690*sqrt(1/2 * (cos(2*pi*(1-ksi))+1)) - 0.12600*(1/2 *
    (cos(2*pi*(1-ksi))+1)) - 0.35160*(1/2 * (cos(2*pi*(1-ksi))+1)).^2 +
    0.28430*(1/2 * (cos(2*pi*(1-ksi))+1)).^3 - (0.10150 + 0.0021)*(1/2 * (cos
    (2*pi*(1-ksi))+1)).^4);
        end
        x = 1/2 * (cos(2*pi*(1-ksi))+1);
        end

        function [x,y] = c2(eta); x = 40*eta + 1; y = 0; end

        function [x,y] = c3(ksi)
        if (ksi <= 0.5); y = -sqrt(41^2-(41*cos(2*pi*ksi))^2); else; y =
    sqrt(41^2-(41*cos(2*pi*ksi))^2); end
        x = 41*cos(2*pi*ksi);
        end

        function [x,y] = c4(eta); x = 40*eta + 1; y = 0; end

        function [x,y,dksi,deta]=Elliptic_mesh (x0,y0)
        % (Standard Elliptic Mesh Solver - Same as previous)
        m=size(x0,1); n=size(x0,2); dksi=1; deta=1; x=x0; y=y0; iter=0;
        for k=1:50;
        end
        end

        function [x_ksi_e, x_eta_e, y_ksi_e, y_eta_e, x_ksi_n, x_eta_n,
    y_ksi_n, y_eta_n, x_mid_n, y_mid_n, J_v, J_h] = Calc_Geo_Metrics(x,y,dksi
    ,deta)
        % Calculates all derivatives (Copy logic from previous main code
    block 1)
        m = size(x,1); n = size(x,2);
        x_ksi_e = zeros(m,n-1); x_eta_e = zeros(m,n-1);
        y_ksi_e = zeros(m,n-1); y_eta_e = zeros(m,n-1);
```

```
        x_ksi_n = zeros(m-1,n); x_eta_n = zeros(m-1,n);
        y_ksi_n = zeros(m-1,n); y_eta_n = zeros(m-1,n);
        x_mid_n = zeros(m-1,n); y_mid_n = zeros(m-1,n);

        % Vertical face metrics
        for j = 1 : n-1
        for i = 1 : m-2
        x_ksi_e(i+1,j) = (x(i+2,j+1) + x(i+2,j) - x(i,j+1) - x(i,j)) / (4*
    dksi);
        x_eta_e(i+1,j) = (x(i+1,j+1) - x(i+1,j)) / (deta);
        y_ksi_e(i+1,j) = (y(i+2,j+1) + y(i+2,j) - y(i,j+1) - y(i,j)) / (4*
    dksi);
        y_eta_e(i+1,j) = (y(i+1,j+1) - y(i+1,j)) / (deta);
        end
        x_ksi_e(m,j) = (x(2,j+1) + x(2,j) - x(m-1,j+1) - x(m-1,j)) / (4*dksi
    );
        x_eta_e(m,j) = (x(m,j+1) - x(m,j)) / (deta);
        x_ksi_e(1,j) = x_ksi_e(m,j); x_eta_e(1,j) = x_eta_e(m,j);
        y_ksi_e(m,j) = (y(2,j+1) + y(2,j) - y(m-1,j+1) - y(m-1,j)) / (4*dksi
    );
        y_eta_e(m,j) = (y(m,j+1) - y(m,j)) / (deta);
        y_ksi_e(1,j) = y_ksi_e(m,j); y_eta_e(1,j) = y_eta_e(m,j);
        end
        % Horizontal
        for i = 1 : m-1
        for j = 1 : n-2
        x_ksi_n(i,j+1) = (x(i+1,j+1) - x(i,j+1)) / (dksi);
        x_eta_n(i,j+1) = (x(i+1,j+2) + x(i,j+2) - x(i+1,j) - x(i,j)) / (4*
    deta);
        y_eta_n(i,j+1) = (y(i+1,j+2) + y(i,j+2) - y(i+1,j) - y(i,j)) / (4*
    deta);
        y_ksi_n(i,j+1) = (y(i+1,j+1) - y(i,j+1)) / (dksi);
        x_mid_n(i,j+1) = (x(i,j+1)+x(i+1,j+1))/2;
        end
        % Boundaries (Simplified for length)
        x_eta_n(i,n) = (x(i,n)-x(i,n-1))/deta; x_ksi_n(i,n) = (x(i+1,n)-x(i,
    n))/dksi;
        x_eta_n(i,1) = (x(i,2)-x(i,1))/deta;   x_ksi_n(i,1) = (x(i+1,1)-x(i
    ,1))/dksi;
        y_eta_n(i,n) = (y(i,n)-y(i,n-1))/deta; y_ksi_n(i,n) = (y(i+1,n)-y(i,
    n))/dksi;
        y_eta_n(i,1) = (y(i,2)-y(i,1))/deta;   y_ksi_n(i,1) = (y(i+1,1)-y(i
    ,1))/dksi;
        x_mid_n(i,1) = (x(i,1)+x(i+1,1))/2; x_mid_n(i,n) = (x(i,n)+x(i+1,n))
    /2;
        end
        J_v = x_ksi_e .* y_eta_e - x_eta_e .* y_ksi_e;
        J_h = x_ksi_n .* y_eta_n - x_eta_n .* y_ksi_n;
        end

```

Listing 4: Complete MATLAB code for solving compressible potential flow with mesh generation and Prandtl-Glauert correction